Phase-Based Planning Model for Software Work in a Human–Agent Cell

Aleksandr Rodionov
Yandex, Saint Petersburg, Russia
rexarrior@yandex.ru
ORCID: 0009-0006-6710-923X

Abstract

Reduced active human effort per task does not by itself determine project duration: autonomous intervals may overlap, while dependencies, integration, and repeated developer attention constrain elapsed time. We compare one developer using up to P agent streams with sequential no-AI completion of the same fixed interdependent workload and acceptance criterion.

We propose a deterministic phase-based model linking task- and mode-specific durations, a task DAG, agent streams, integration overhead, and capacity-one human attention. Its M0–M4 hierarchy progresses from divisible work through indivisibility, precedence, and overhead to exact phase schedules with local slot retention. M0 gives an exact aggregate criterion; under identical-stream, zero-overhead assumptions, M1–M2 give sufficient speedup guarantees for indivisibility and precedence.

At M4, the structural lower bound B_4 combines stream load, critical-path length, and total human attention; minimizing the maximum-weight path length over feasible resource-augmented phase graphs gives T^* exactly. A reproducible package combines CP-SAT optimization on frozen exact grids with larger synthetic fixed-policy grids and targeted exact-solver audits. In the frozen 90-scenario EXP-01 grid, a strict certified gap $T^* > B_4$ occurred in 30 cases. On examined finite grids, configuration-dependent overhead produced interior minima over P, while positive decomposition overhead produced U-shaped granularity profiles; more streams or finer decomposition need not reduce makespan.

Within the model, achievable speedup is a property of the fully specified human–agent process, not of the AI tool or nominal agent count alone. These results check internal logic rather than estimate a particular tool or team; practical use requires local calibration.

1 Introduction

Coding agents change the unit at which software-development productivity must be analyzed. Unlike autocomplete or a single-turn chat assistant, an agent may inspect a repository, plan several actions, edit multiple files, run tools and tests, and return to the developer only after an autonomous interval. Recent human and production evidence makes the resulting distinction between human effort and elapsed process time explicit. In a controlled comparison published at CHI 2026, an agent increased the correct-completion rate and, among correctly completed tasks, roughly halved the study's measured user-effort

metric relative to a copilot; total agent-workflow time was comparable once autonomous actions were included. Many participants also reported understanding copilot outputs better, although that contrast was not statistically decisive [11]. Production traces from millions of GitHub Copilot sessions likewise show sparse user turns unfolding into long chains of model calls and tool execution [25]. These observations describe a process that alternates autonomous execution with human specification, inspection, and correction, rather than a tool that multiplies developer speed by a single constant.

The outcomes of such a process are intrinsically multidimensional. In a randomized study, direct agentic editing improved initial task completion relative to a constrained chatbot but reduced code comprehension, with no statistically distinguishable advantage in accuracy or time on the later extension task [2]. Large-scale observational logs also contain recurrent divergences from developer instructions and intentions; among the small subset of identified misalignment episodes with a visible resolution, correction usually followed explicit developer pushback rather than autonomous self-correction [31]. Neither study estimates the effect of several concurrent agents, and neither converts directly into a project-duration coefficient. Together with evidence that task selection changes when developers can use multiple agents in parallel [5], they show why elapsed time, active human time, correctness, comprehension, and acceptance must remain separate quantities. Any duration coefficient used for planning must therefore refer to a specified task, tool, operating mode, baseline, and acceptance criterion.

Even a reliable task-level coefficient is not yet a project plan. The makespan of a fixed set of interdependent tasks depends on which autonomous intervals can overlap, how tasks occupy a limited number of agent streams, when predecessors release their successors, and how often the same developer must specify, clarify, review, or accept work. Existing asynchronous coding-agent systems already expose dependency-management, merge, test, and integration costs, and increasing the configured agent count need not reduce runtime [16]. A local task speedup may therefore leave project makespan unchanged if it does not relax the active constraint. Conversely, a mode that takes longer for one isolated task may still reduce total elapsed time when its autonomous phase overlaps other work. A scheduling problem intervenes between an observed task-level AI effect and the elapsed time of an accepted workload.

This article studies one *human–agent cell*: one developer completes a prespecified AI-assisted set of software tasks using up to P agent streams, continuing each task until its result meets a fixed acceptance criterion. The no-AI baseline is sequential completion of the same workload by one developer. Interactive, delegated, and fully autonomous operating modes are distinguished. In interactive mode, the human and tool work jointly in one stream. In delegated mode, mandatory human-attention phases surround or interrupt autonomous agent phases. Fully autonomous mode is admitted as the limiting case with no mandatory human phase, but automatic discovery of new work and automatic acceptance of results are not modeled.

For delegated tasks, the model adopts a local retention rule: once started, a task retains its assigned agent slot until acceptance. If its agent requests a busy developer, that task and slot wait while other agents may continue. A new task can start only in an available slot, and a dependent task can start only after all predecessors have been accepted. This is one explicit process architecture, not a claim about agent systems in general. Slot reuse during a human wait, preemption, multiple developers, additional constrained CI or API resources, dynamic revelation of the work graph, and interproject scheduling remain outside the formulation. The choice between manual and AI-assisted

execution is also external: the model schedules an already selected AI-assisted task set.

The research question is: *when does a fully specified configuration in which one developer works with multiple AI agents reduce the makespan of a fixed, interdependent workload relative to sequential work without AI, and which constraints determine the attainable speedup?* A complete comparison must specify not only P and the operating mode of each task, but also the task decomposition, dependency graph, phase profiles, acceptance criterion, and overhead functions. The optimization objective is the makespan of an accepted result. Quality and computational cost enter only insofar as they alter review, correction, or completion time; they are not separate objectives in the present model.

The central thesis is that, for the human–agent cell modeled here, **achievable speedup is a property of a fully specified human–agent process, not the product of "AI speed" and a nominal agent count**. Makespan can be constrained by aggregate stream load, an indivisible task, technological precedence, the sequential human resource, or a mixed resource chain created by phase ordering and the developer-attention queue. Additional available capacity cannot worsen the optimum when durations and overhead are fixed; increasing configured P, however, may raise integration or switching overhead and need not improve makespan. Decomposition is equally conditional: it can expose useful parallelism while adding new boundaries for specification, review, and integration.

To make these constraints explicit, the article develops a deterministic phase-based hierarchy M0–M4. The normalized duration $k_i^{(r)}$ belongs to a "task—tool—mode" triplet and compares the complete cycle required for an accepted result with the no-AI duration of the same task. It is descriptive, not a causal estimate without a separate identification design. At M4 it is decomposed into autonomous-agent and human-attention components $a_i^{(r)}$ and $h_i^{(r)}$, whose internal order is represented by phases. M0 starts from ideally divisible aggregate work; M1 adds indivisible tasks; M2 adds the dependency graph; M3 adds cross-stream integration overhead; and M4 constructs an exact phase schedule with a shared human resource and local blocking.

The article makes four contributions. First, it defines the operational unit of comparison as a complete "tasks—modes—resources—acceptance criterion" scenario. This separates observed task duration from a forecast, available capacity from configured parallelism, and an autonomous phase from a fully autonomous task.

Second, it derives conditions for speedup in M0–M2. At M0, speedup is equivalent to $P > \bar{k}_w$. With indivisible tasks and precedence, this remains necessary, while a classical list-scheduling bound yields a sufficient criterion involving the relative longest task or critical path. The hierarchy therefore distinguishes an idealized estimate, a structural lower bound, and an attainable schedule.

Third, for M4 it introduces the structural lower bound

$$B_4 = \max\{W_4/P, L_4, \gamma(P)H\},$$

and a speedup ceiling imposed by sequential human work. The bound combines stream workload, the original critical path, and total human attention, but it need not equal the optimal makespan T^*. Minimizing the maximum-weight path length over feasible resource-augmented phase graphs gives T^* exactly and shows how the service order of human phases and the task order on agent slots create mixed paths absent from the original task DAG.

Fourth, a reproducible computational package evaluates the internal logic of the model in series EXP-00–EXP-08. An exact solver and a fixed deterministic policy are used for different estimands. The exact solver reproduces the illustrative scenarios and provides

certified counterexamples to universal attainability of B_4; the fixed policy supports the larger synthetic sensitivity studies. Together, they separate available capacity from configuration-dependent overhead, test granularity and scale robustness, and examine sensitivity to synthetic DAG structure and the placement of human work. These are computational verification and sensitivity analyses of the formal model, not empirical estimates for a particular team or AI product.

The underlying scheduling objects—indivisible jobs, critical paths, capacity-constrained resources, and resource-ordering arcs—are established. The contribution is their systematic mapping to a software task cycle with a task- and mode-specific coefficient, alternating agent and human phases, repeated requests for developer attention, slot retention, integration overhead, and an explicit acceptance criterion. Empirical calibration of this mapping remains open. The remainder of the article positions the model against empirical AI-productivity, orchestration, scheduling, and human-attention research; defines M0–M4; reports the computational method and results; and then discusses practical interpretation, validity threats, and future work.

2 Problem Setting and Compact Notation

This section provides a concise conceptual overview of the formulation and its key notation; formal definitions and deterministic assumptions are introduced in the *Formal Model* section. The model describes a fixed software-development workload that is completed to a predefined acceptance criterion. The no-AI baseline consists of sequential completion of the same workload by a single developer. A complete glossary of terms and the full notation system are provided in Appendix A.

Task and mode.
Task i has size Z_i and is performed in operating mode r: interactive, delegated, or fully autonomous. The mode determines the sequence of task specification, agent execution, review, and correction of the result. The main M4 analysis concerns the interactive and delegated modes; the fully autonomous mode, which has no mandatory human-attention phase, serves as a limiting case.

Normalized AI-assisted duration.
The quantity $k_i^{(r)} = T_i^{ai}(r)/(Z_i X)$ relates the duration required to obtain an accepted result to the baseline time for the same task. It is specific to the "task–tool–mode" triplet, not to the AI tool in general. This is a descriptive normalization; a causal interpretation requires a separate identification design.

Phases.
The normalized duration is decomposed as $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$. Autonomous agent phases require an agent slot; human-attention phases simultaneously require the developer's attention and retention of the assigned slot.

Configured parallelism.
The parameter P denotes the number of agent streams configured in the given configuration and available to the schedule; coordination overhead is evaluated at this value. Additional unused capacity under an unchanged configuration does not itself generate overhead.

Configuration.
The notation $\sigma = (P, r)$ or $\sigma = (P, \rho)$ is shorthand when the AI tool and acceptance rules are fixed. If the tool or the acceptance criterion for an accepted result changes, the change must be stated explicitly in the comparison, although these elements are not added as dimensions of σ.

Schedule and makespan.
A feasible schedule respects the task-DAG dependencies, phase order, the unit capacity of the human resource, and local slot retention. Its makespan is denoted by $T(\pi)$, and the minimum makespan over feasible schedules by T^*.

Notation	Meaning
X	baseline effort per unit of work without AI
Z_i	size of task i in units of X
$k_i = a_i + h_i$	normalized duration and its agent and human components
P	configured number of agent streams in the configuration
$G = (V, E)$	task-dependency DAG
$W = X \sum_i Z_i k_i$	total work when $C = \gamma = 1$
$A,\ H$	total autonomous agent work and human work
$C(P),\ \gamma(P)$	multipliers for cross-stream integration overhead and human attention-switching overhead
$T(\pi),\ T^*$	makespan of a particular schedule and the optimal makespan

3 Related Work

3.1 Empirical Estimates of AI Productivity

Recent agent studies make the measurement problem visible within a single workflow. Chen et al. compared GitHub Copilot with OpenHands in a randomized-order, within-participant study of 20 regular Copilot users performing data-analysis, feature-addition, and bug-fixing tasks [11]. Mean correct completion was 25% with the copilot and 60% with the agent. Among correctly completed tasks, measured user effort fell from 25.1 to 12.5 minutes, but mean total agent-workflow time was 27.9 minutes after autonomous actions were added. User effort was operationalized differently by condition: start-to-finish work for the copilot, but instruction and evaluation intervals excluding autonomous actions for the agent. The contrast is therefore evidence that a delegated workflow can reduce active human time without an equal reduction in elapsed time, not a twofold task speedup. The study had no no-AI arm, used one agent at a time, included only 20 student participants who were novice agent users, and compared treatment packages with different interfaces and model choices.

The distinction extends beyond time. In the randomized study by Balepur et al., an agent accelerated an initial web-development task and improved its accuracy relative to a constrained chatbot, but reduced code comprehension; no sustained advantage in time or accuracy appeared on a later extension performed without the editing agent [2]. Because the comparator was another AI workflow rather than no AI, the study does not identify k. Together, the two controlled studies show that correctness, active human effort, total elapsed time, comprehension, and later task performance are separate outcomes.

Production observations describe still other units of analysis. Telemetry from millions of GitHub Copilot agent sessions reveals multi-step chains of model and tool calls, retries,

long-tailed workloads, and limited internal parallelism, but does not link sessions to fixed tasks completed to an accepted result or to human actions [25]. Tang et al. extracted 16,118 evidence-grounded misalignment episodes from 20,574 public or opt-in IDE and CLI sessions [31]. Among the 9.33% with a visible resolution, 91.49% were resolved by the agent after explicit developer pushback. This conditional result supports the presence of corrective human work; it is not the share of all agent tasks requiring intervention, and the study measures neither correction time nor productivity.

Earlier copilot studies remain useful historical causal contrasts. In the experiment by Peng et al., participants who completed a bounded JavaScript task using GitHub Copilot spent, on average, 55.8% less elapsed time than control-group completers [27]. The result pertains to one task, an early Copilot version, and a complete-case sample. By contrast, Becker et al. found that access to heterogeneous tools available in early 2025 increased adjusted active implementation time by 19% among experienced developers working on mature open-source projects [4]. The estimates concern different participants, tasks, tools, inclusion rules, and outcomes and do not contradict one another; neither studies several concurrent agents or complete project makespan.

Heterogeneity in effects is also observed outside software development. When a generative assistant was introduced in customer support, Brynjolfsson et al. estimated a 15% average increase in throughput, measured as the number of resolved cases per hour [9]. The effect depended on employee experience and case type, and the throughput of a dynamic flow cannot be converted directly into the duration of a fixed software-development task. In terms of the model, estimates of $k_i^{(r)}$ are comparable only when task boundaries, the baseline configuration, and acceptance criteria are the same. Elapsed time, active time, throughput, success, quality, and comprehension remain distinct outcome measures.

Earlier interaction studies explain additional heterogeneity. Vaithilingam et al. found no statistically significant Copilot benefit in either time or the task-completion rate across three short tasks, but identified costs associated with understanding, reviewing, editing, debugging, and rewriting generated code [33]. Barke et al. showed that programmers alternate between using AI to accelerate their work and using it to explore a possible solution [3]. The operating mode r therefore describes the entire process of task specification, execution, review, and correction, while $h_i^{(r)}$ includes deferred review and rework.

Finally, METR notes that selecting tasks based on AI availability and working with multiple agents in parallel biases estimates at the individual-task level [5]. These findings motivate separate measurement of autonomous and human-attention intervals and comparison of complete workflow configurations rather than extrapolation from one task or session metric.

3.2 Effort Estimation as an External Model Layer

Effort estimation addresses a related but distinct problem. Boehm, Abts, and Chulani systematize parametric, regression-based, analogy-based, expert, and combined methods for forecasting effort, cost, and duration [6]. These methods estimate inputs that are not yet known, but do not by themselves assign indivisible tasks to streams, verify resource feasibility, or minimize the makespan of a fixed workload. Comparisons of formal models and expert estimates identify no universally superior approach [21]; a systematic map of 304 journal publications also documents the predominance of retrospective validation and insufficient attention to uncertainty and prospective validation [22]. An observed post-task value of $k_i^{(r)}$ therefore does not itself constitute a forecast. Before a project begins, an

external, locally calibrated estimate $\hat{k}_i^{(r)}$ with explicitly stated uncertainty is required; the scheduling model translates these estimates into elapsed-time constraints.

3.3 Scaling, Orchestration, and Coding-Agent Architectures

For a fixed workload, Amdahl's law relates the limiting speedup to the sequential fraction [1]. Only a structural analogy is used here: the sequential resource is operationally defined human involvement, not a segment of a machine program. Gunther's Universal Scalability Law permits saturation and negative returns due to resource contention and state coherency [19]; its denominator is a hypothesis for $C(P)$, not an empirically established law for coding agents. Brooks's reasoning about communication and conceptual integrity similarly motivates integration overhead [7], but the number of communication links in a human team cannot be transferred literally to the "one developer—multiple agents" topology.

In general-purpose benchmarks of multi-agent systems (MAS), Kim et al. compared 260 configurations and found that success depended on task decomposability, sequential dependencies, the base model, and topology [23]. The outcomes, however, were accuracy and success. No human participated, and the number of agents varied together with the architecture. This supports the structural dependence of coordination overhead on system and task structure, but does not estimate $C(P)$ or the makespan of the human–agent cell considered here.

Repository-level systems provide closer but still incomplete analogues. MAGIS separates the roles of planner, repository custodian, developer, and quality assurance and permits automated review-and-revision cycles [32]. Its primary outcome is the proportion of SWE-bench tasks solved; four roles do not imply four concurrently operating streams, and automated quality assurance is not a human resource. CAID, by contrast, constructs a dependency-aware plan and does execute developer agents concurrently in isolated Git worktrees, followed by merging, testing, and final review [16]. In each of the six primary "model family—task set" pairs, CAID increased the quality metric but simultaneously increased elapsed completion time and cost. Comparisons of configurations with 2, 4, and 8 agents showed a nonmonotonic relationship between quality and agent count, together with increasing overhead. CAID therefore supports the distinction among the configured number of agents, actual concurrency, and useful parallelism, but does not demonstrate speedup and lacks human-attention phases, a no-AI condition, and decomposition into a_i/h_i.

Thus, internal parallel tool use by a single agent, concurrent execution of independent tasks, the number of named roles, and the coordination architecture must be distinguished. Actual concurrency can be established only from observed overlapping intervals under an unchanged comparison condition; success and quality metrics do not replace measurement of makespan.

3.4 Scheduling Theory and Repeated Shared Service

The mathematical core of levels M1–M2 is standard. For identical parallel machines, the conventional quantities are makespan, total workload, the duration of the longest job, and the critical path [29]; list scheduling under precedence constraints and the LPT rule for independent jobs have classical guarantees [17, 18]. Disjunctive arcs, or resource-ordering arcs, are likewise a standard way to represent competition among machines [29]. The article therefore does not claim novelty for makespan, a directed acyclic graph (DAG), the

critical path, a resource-augmented graph, or a shared resource.

The resource-constrained project scheduling problem (RCPSP) combines a DAG of precedence relations with renewable resources of finite capacity, while multi-mode formulations permit alternative duration and resource profiles [8]. Within such a core, attention can be represented as a resource of capacity 1 and phases as separate operations. The standard model, however, does not specify a coefficient k for a software task and operating mode, the meaning of an accepted result, an operational decomposition a_i/h_i, or internal integration and attention-switching overhead.

Still closer analogues are parallel-machine models with a common server. In setup-only formulations, the server sequentially prepares jobs before their parallel processing [20, 24]. In a loading/unloading formulation, each job uses the same server before and after machine processing; a requirement for immediate unloading may keep a machine occupied after processing until the server becomes available [15]. This is a formal analogue of the sequence "human task specification—agent work—human review" and a partial analogue of slot retention, but it lacks a DAG and a repeated "agent rework—human review" cycle.

An even closer reentrant-scheduling formulation routes a job through its assigned primary machine, a remote server, and then back to the same primary machine [10]. It reproduces the alternation "worker—shared service—the same worker," but includes only one repeated visit to the server, independent jobs, and a prespecified route. Importantly, the primary machine is released during remote service; this model therefore does not reproduce M4 local blocking. Neither common-server models nor the reentrant formulation by themselves include a coefficient k for software work, human acceptance, or integration overhead.

This comparison clarifies the contribution of M4: established scheduling mechanisms are used as the core, while the proposed mapping to the software process links the complete cycle "human task specification—agent work—human review—agent rework—human acceptance" to the task's operating mode, separate phase durations, retention of the assigned agent slot, and the fact that the result has been accepted. If a human-attention phase does not retain the slot, A and H should be scheduled as separate resources; this corresponds to a different resource-use rule.

3.5 Human Attention in Supervising Autonomous Workers

Research on human–robot interaction (HRI) provides a substantive structural analogue for one operator and multiple autonomous workers. The fan-out measure, the number of robots per operator, is related to the durations of interaction and autonomous work without operator attention [26]; models of multitask control require service for one robot to fit within the autonomous intervals of the others [13]. State-aware attention allocation shows that the outcome depends on the selection rule and system state, not only on the average fraction of involvement [12]. Threaded-cognition theory further cautions that not all human activity is uniformly sequential: interference arises at specific cognitive resources [30].

Cummings and Mitchell tested an extended robots-per-operator model in a simulation experiment with 12 participants and multiple unmanned aerial vehicles (UAVs) [14]. Accounting for waiting for interaction, waiting in the queue, waiting to regain situational awareness, and cognitive reorientation reduced the model-based estimate of operator throughput by 36–67% under the conditions examined; the authors explicitly emphasize that the result depends on the task and context. The experiment supports a sequential

service queue and a finite useful number of autonomous workers only as a structural analogy. It does not calibrate the number of coding agents, $\gamma(P)$, or the makespan of software work: the vehicles were homogeneous and independent, and the outcome was a model-based estimate of throughput, not the completion time of an accepted software task.

3.6 Synthesis and Bounded Empirical Gap

Table 1 shows that the article's distinction does not reduce to adding a DAG or a shared resource. The closest partial analogues are distributed across several research areas: RCPSP provides precedence relations and resource-capacity constraints; common-server and reentrant models provide repeated service and different machine-retention rules; HRI provides a queue for, and switching by, a single operator; general-purpose MAS provide evidence on the effect of coordination topology; MAGIS provides an automated review-and-revision cycle; and CAID provides concurrently executed branches, merging, and testing. Each of these elements is known separately.

Within the core corpus assembled under the search protocol, with a cutoff date of 2026-08-09, no study was identified that jointly calibrated, on software tasks, a task- and mode-specific coefficient k, autonomous and human-attention intervals, a dependency DAG, actual concurrency, repeated visits to a single human, local retention of an agent slot, integration and switching overhead, and the elapsed-time makespan of an accepted result. This statement applies only to the reviewed corpus and the search channels specified in the protocol; it is not a universal claim about the entire literature.

CAID, the closest software system, observes concurrency, elapsed completion time, and integration, but does not include a human or a no-AI condition. MAGIS includes automated review and revision but does not establish actual concurrency or human service. Common-server and reentrant models formalize repeated service but do not map it to the software process; HRI quantifies waiting associated with human attention but not software work; and general-purpose MAS evaluate primarily success and quality rather than the makespan of a human–agent cell.

The formal contribution of the present work is the joint representation of these quantities and the verification of model properties on synthetic scenarios. The article itself does not perform empirical calibration of a real-world workflow. Collecting observations on software tasks completed to an accepted result, and prospectively validating whether a jointly estimated M4 predicts elapsed-time makespan better than the simpler levels, remain future work.

Table 1: Closest formulation classes and the scope of the present model. The values "yes," "partial," and "no" indicate whether a property is present in a formulation class, not empirical equivalence or novelty.

Formulation class	DAG	Agent streams	Unit-capacity service	Repeated service	Slot retention	Task- and mode-specific k	a_i/h_i phase split	Coordination and integration overhead	Primary outcome	Real-world software-process calibration
RCPSP and multi-mode RCPSP [8][a]	yes	partial	yes	partial	no	no	no	partial	makespan, cost, feasibility	no
Parallel machines with a common server [20, 24, 15, 10][b]	no	yes	yes	partial	partial	no	partial	no	makespan, algorithmic bounds	no
HRI: robots per operator and attention allocation [26, 13, 12, 14][c]	no	yes	yes	yes	partial	no	partial	partial	throughput, mission performance	no
General-purpose MAS and centralized coordination [23][d]	no	partial	no	partial	no	no	no	partial	success, quality; sometimes cost	no
Coding agents: MAGIS and CAID [32, 16][e]	partial	partial	no	partial	no	no	no	partial	success, quality; CAID also time and cost	no
Present work[f]	yes	yes	yes	yes	yes	yes	yes	yes	makespan of an accepted result	no

[a] An RCPSP renewable resource need not represent identical agent streams; repeated service and overhead require explicit operations, and the standard formulation does not introduce automatic slot retention while another resource is awaited.

[b] Repeated service is present in loading/unloading and reentrant variants, but not in setup-only formulations. Immediate unloading may retain a machine, whereas the reentrant model with a remote server releases it; machine-processing and service durations are not calibrated as agent and human components of software work.

[c] An autonomous system waits for the operator or loses performance, but this is not equivalent to retaining a software-development slot. Interaction and autonomous intervals are not software-specific a_i/h_i components; queues, loss of situational awareness, and reorientation do not include code integration.

[d] Sequential interdependence is measured, but no explicit task DAG is constructed; the number of agents or roles does not establish that their intervals overlap. A centralized coordinator is not equivalent to observed unit-capacity service, repeated messages are not a human service cycle, and differences in outcomes do not constitute calibration of M4 overhead.

[e] CAID constructs a dependency-aware plan and exhibits overlapping branches; these properties are not established for MAGIS or for the class as a whole. Automated planning, quality assurance, and revision are not human service; waiting for dependencies or merging is not equivalent to local blocking. CAID observes merging, testing, time, and cost, but does not calibrate a human–agent process. Evaluation on software task suites is not empirical calibration of a real-world workflow with observed human-attention phases.

[f] The article uses synthetic scenarios; empirical calibration and prospective validation remain future work.

The article does not claim novelty for DAGs, makespan, a shared resource, repeated service, return to the same worker, resource retention, resource-ordering arcs, or the number of autonomous workers per operator.

4 Formal Model

4.1 Formal Definitions and Deterministic Assumptions

Definition 1. Let the following parameters be given:

- $X > 0$ — the baseline effort required for one unit of work (the time required to implement an elementary unit of functionality);
- $Z_i \in \mathbb{R}^+$ — the size of task i, measured in units of X. Fractional values are permitted: when a project is decomposed, a subtask may account for a fraction of the baseline unit of work (e.g., $Z_i = 0.5$). In applied calculations, values are typically selected from a finite discrete scale $D \subset \mathbb{Q}^+$;
- $r \in R$ — the **operating mode** used to organize AI-assisted work, that is, the way in which the developer interacts with the tool (e.g., interactive work within a single dialogue; delegated execution with an autonomous agent phase followed by review; or fully autonomous execution without a mandatory human-attention phase). The main M4 analysis concerns the interactive and delegated modes; the fully autonomous mode serves as a limiting case. The finite set of modes R is specified when the model is applied;
- $k_i^{(r)} \in \mathbb{R}^+$ — the normalized duration of task i when AI is used in mode r. This quantity is a property of the "task × tool × mode" triplet and is defined operationally as

$$k_i^{(r)} = \frac{T_i^{ai}(r)}{Z_i X},$$

  where $T_i^{ai}(r)$ is the actual time required to complete task i with AI *in a single work stream operating in mode r when the developer is fully available*, that is, without competition from other streams for the developer's attention. It includes formulating prompts, waiting for generation, reviewing the result, and correcting it. A value of $k_i^{(r)} < 1$ corresponds to a shorter duration than the baseline estimate $Z_i X$, $k_i^{(r)} = 1$ to the same duration, and $k_i^{(r)} > 1$ to a longer duration. This is a descriptive normalization, not a causal estimate of the effect of AI in the absence of a separate identification design. If the mode is fixed or clear from context, the superscript is omitted: $k_i := k_i^{(r)}$;
- $h_i^{(r)}$, $a_i^{(r)}$ — the **human** and **agent** components of the coefficient $k_i^{(r)}$. To define $k_i^{(r)}$, the time $T_i^{ai}(r)$ is measured through two channels. The first comprises intervals during which the developer is engaged with the task: formulating prompts, reading and reviewing results, and directing corrections. The second comprises intervals of autonomous agent work during which the developer is free: generation, self-review, and test execution. These intervals may alternate within a task; their order is determined by the M4-level phases. Normalizing both components by $Z_i X$ yields, by construction,

$$k_i^{(r)} = a_i^{(r)} + h_i^{(r)}, \qquad 0 \le h_i^{(r)} \le k_i^{(r)}.$$

  The present model assigns every interval included in the analysis to exactly one of these two channels; separate constrained resources for CI, APIs, and external infrastructure are outside its scope. When $P = 1$, this decomposition does not affect

task duration: in the absence of a queue for developer attention, the duration is $k_i^{(r)} Z_i X$ for any relative magnitudes of the components. The distinction becomes material when $P > 1$: agent phases of different tasks may proceed in parallel, whereas all human-attention phases are handled by a single developer (level M4, Remark 10);

- $P \geq 1$ — the configured number of concurrently available agent streams; for the no-AI baseline, $P_h = 1$ is assumed. At levels M1–M4, where tasks and phases are indivisible, P is an integer. Only in the aggregate M0 formulas may P be interpreted as effective parallelism averaged over the period. It is important to distinguish additional *available* capacity from the configuration actually selected: an unused stream does not by itself delay the schedule, whereas switching to a configuration with a larger P may change $C(P)$ and $\gamma(P)$;

- σ — the **process configuration**: the pair $\sigma = (P, r)$ or, in the general case, the pair (P, ρ), where $\rho\colon \{1, \ldots, N\} \to R$ assigns modes to tasks (different work streams may operate in different modes). This notation is shorthand when the AI tool and acceptance rules are fixed: if either the tool or the acceptance criterion for an accepted result changes, the change must be stated explicitly in a comparison, although these elements are not added as dimensions of σ. The model's derived quantities—total work W, the weighted-average coefficient $\bar{k}_w$, time T_{ai}, and speedup S_E—are functions of the configuration. In the model, comparing "work-organization variants" means comparing entire configurations: the parameters P and $\{k_i^{(r)}\}$ are not varied independently (see Remark 3);

- $N \in \mathbb{N}$, $N \geq 1$ — the total number of tasks in the project;

- T_h, T_{ai} — the total time required to complete all tasks without and with AI, respectively.

Remark 1 (Task Independence)**.** Initially, all tasks $i = \overline{1, N}$ are assumed to be **disjoint**: they do not share resources, do not block one another, and may be executed in parallel without additional synchronization costs. This assumption permits an additive time model. Task dependencies and the critical path are introduced subsequently at level M2.

4.2 Model Hierarchy Overview

Remark 2 (Model Hierarchy M0–M4)**.** Levels M0–M4 form a sequence of refinements to the same formulation. Each successive level removes some of the relaxations and introduces one or more related aspects of the model; this does not imply that the transition between every pair of adjacent levels adds exactly one constraint.

Level	Primary refinement	Lower bound
M0	ideally divisible aggregate work	$B_0 = W/P$
M1	task indivisibility	$B_1 = \max\{W/P,\ M_N\}$
M2	task-dependency DAG	$B_2 = \max\{W/P,\ L\}$
M3	effective weights and integration overhead	$B_3 = \max\{W_3/P,\ L_3\}$
M4	phases, local slot retention, queueing, and a single shared developer	$B_4 = \max\{W_4/P,\ L_4,\ \gamma H\}$

The formal refinements are introduced, respectively, in Assumption 1 and Remarks 5–10.

4.3 Level M0: Ideally Divisible Work

Lemma 1 (Baseline Time without AI)**.** When the tasks are performed sequentially by a developer without using AI, the total time is

$$T_h = \sum_{i=1}^{N} Z_i X. \tag{1}$$

Assumption 1 (Idealized AI-Assisted Time Model (Level M0))**.** In the core analytical model, the work is assumed to be ideally divisible, with parallelism distributed uniformly. Define the total AI-assisted workload as

$$W := X \sum_{i=1}^{N} Z_i k_i.$$

When AI is used with parallelism factor P, the total task-completion time is taken to be

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{1}{P} \sum_{i=1}^{N} Z_i k_i X. \tag{2}$$

All subsequent theoretical results are based on this assumption unless another level of the model hierarchy is explicitly specified (Remark 2).

Observation 1 (Speedup Criterion and Factor at Level M0)**.** Define the weighted-average normalized duration, with task sizes Z_i as weights, as

$$\bar{k}_w := \frac{\sum_{i=1}^{N} Z_i k_i}{\sum_{i=1}^{N} Z_i}.$$

Substitution from Lemma 1 and Assumption 1 directly yields the chain of equivalences

$$T_{ai}^{(0)} < T_h \quad \Longleftrightarrow \quad \sum_{i=1}^{N} Z_i k_i < P \sum_{i=1}^{N} Z_i \quad \Longleftrightarrow \quad P > \bar{k}_w. \tag{3}$$

This is not an independent theoretical result, but an algebraic restatement of the definition of time at Level M0.

The speedup factor is

$$S_E = \frac{T_h}{T_{ai}^{(0)}} = P \cdot \frac{\sum_{i=1}^{N} Z_i}{\sum_{i=1}^{N} Z_i k_i} = \frac{P}{\bar{k}_w}. \tag{4}$$

In particular, if $k_i \leq 1$ for every i and $P > 1$, then $S_E \geq P > 1$; that is, speedup is guaranteed.

Furthermore, $\min_j k_j \leq \bar{k}_w \leq \max_j k_j$ implies the two-sided bound

$$\frac{P}{\max_i k_i} \;\leq\; S_E \;\leq\; \frac{P}{\min_i k_i},$$

which provides a quick estimate of the speedup range without weighting by Z_i.

Remark 3 (Speedup Factorization and Baseline Choice)**.** The formula in Observation 1 admits the factorization

$$S_E = \underbrace{P}_{\text{organizational factor}} \cdot \underbrace{\frac{1}{\bar{k}_w}}_{\text{tool factor}}.$$

The first factor captures the effect of organizing work in parallel, whereas the second captures the effect of AI on task effort. Sequential work by a single developer ($P_h = 1$) is deliberately chosen as the no-AI baseline T_h. The model therefore addresses what a single developer gains by transitioning to AI agents, rather than comparing AI with a team of P people. If $k_i \equiv 1$, then $S_E = P$: this speedup is due entirely to process organization and can, in principle, also be attained by a team of P people. The central assumption is that the P parallel streams are provided by AI agents supervised by the same developer, without involving additional specialists. Supervision overhead is accounted for at Levels M3–M4: the function $C(P)$ represents cross-stream integration overhead (Remark 9), whereas the term $\gamma(P)\,H$ represents the human resource and its associated ceiling $S_E^{\text{ach}} \leq 1/\bar{h}_w$ (Remark 10, Proposition 4). For comparison with a team of q developers, the baseline time should be replaced, under the idealized approximation, by $T_h(q) = T_h/q$; the condition under which the AI-assisted process is superior then becomes $P/\bar{k}_w > q$.

The factorization is valid *for each fixed configuration* $\sigma = (P, r)$, but its factors cannot be varied independently. In practice, changing P is often accompanied by a change in operating mode r, for example, a transition from interactive work to autonomous agents. The change in mode changes the entire vector $\{k_i^{(r)}\}$ and hence $\bar{k}_w$. Consequently, $\bar{k}_w$ must not be estimated in one mode and combined with the parallelism of another; for example, one must not measure k_i during interactive work and substitute $P = 4$ for autonomous agents. Configurations must be compared as a whole: $S_E(\sigma_1)$ and $S_E(\sigma_2)$.

Remark 4 (Critical Parallelism Threshold for $k_i > 1$)**.** The identity in Observation 1,

$$\sum_{i=1}^{N} Z_i k_i < P \sum_{i=1}^{N} Z_i,$$

implies that a speedup $S_E > 1$ is achieved if and only if

$$P > \bar{k}_w,$$

where $\bar{k}_w$ is the weighted-average normalized duration introduced in Observation 1.

We emphasize that the condition $P > \bar{k}_w$ is necessary and sufficient only at Level M0. At Levels M1–M2, it remains necessary but is no longer sufficient: the achievable time is also bounded below by the longest task (M_N, Remark 5) and by the critical path (L, Remark 6); Theorem 1 provides a substantive sufficient result for these levels. At Level M3, the necessary condition based on the aggregate workload of the streams is $C(P)\bar{a}_w + \bar{h}_w < P$ (Remark 9). At Level M4, all three inequalities specified by $B_4 < T_h$ are simultaneously necessary: $W_4/P < T_h$, $L_4 < T_h$, and $\gamma(P)H < T_h$. They are not sufficient because there exist instances in which $T^* > B_4$ due to local blocking and the queue for developer attention.

- If $k_i \equiv k > 1$ for every i (AI uniformly slows execution), the critical threshold simplifies to

  $$P_{\text{crit}} = k.$$

  Thus, parallelism must **strictly exceed** the slowdown factor; otherwise, using AI provides no elapsed-time advantage.

- If the coefficients k_i are heterogeneous, the threshold is determined by the weighted average: tasks with larger task sizes contribute more to $\bar{k}_w$. Thus, even when some tasks have $k_i \gg 1$, speedup is possible if their contribution to the total task size is small and parallelism is sufficient to compensate for it.
- In practical applications, $\bar{k}_w$ characterizes the duration of the selected AI-assisted configuration for a given project. The estimation procedure is presented in Remark 14. If $\bar{k}_w$ is no smaller than the configured parallelism P, this configuration provides no elapsed-time gain. In that case, one should
  1. reconsider the set of tasks selected for automation, beginning with those having the largest k_i;
  2. increase the permissible process parallelism, for example through asynchronous execution;
  3. reduce k_i by changing how AI is used, including prompt formulation, context composition, and postprocessing.

Thus, rather than providing a binary assessment of the usefulness of AI, the model specifies a **quantitative decision criterion** that relates durations in the selected mode ($k_i^{(r)}$), project structure (Z_i), and configured parallelism (P). The criterion is applied to the configuration $\sigma = (P, r)$ as a whole: the values of $\bar{k}_w$ and P must refer to the same operating mode (Remark 3). The choice between fully manual and AI-assisted execution of individual tasks is outside the scope of the formulation; once that choice has been made, the model schedules only the included AI-assisted task set.

4.4 Level M1: Task Indivisibility

Remark 5 (Workload Lower Bound B_1)**.** The level-M0 formula assumes a perfectly even distribution of the workload. In practice, when tasks are indivisible, the completion time cannot be shorter than the duration of the longest task, however large P may be. Define

$$M_N := X \max_{i \in \overline{1,N}} (Z_i k_i).$$

The tighter bound is given by the inequality

$$T_{ai} \geq B_1 := \max \left\{ \frac{W}{P},\ M_N \right\}.$$

Here, B_1 is a lower bound on the makespan; the exact value depends on the assignment of tasks to an integer number of processors and may strictly exceed B_1 even under an optimal schedule.

4.5 Level M2: Dependencies and the Critical Path

Remark 6 (Critical-Path Lower Bound B_2)**.** When logical or resource dependencies are present, a graph-based model is used in place of the additive model. Let $G = (V, E)$ be a directed acyclic graph whose vertices V correspond to tasks and whose arcs E encode precedence relations. The total completion time with AI then satisfies

$$T_{ai} \geq B_2 := \max \left\{ \frac{W}{P},\ L \right\},$$

where L is the length of the critical path in G under the weights $Z_i k_i X$. Every individual task constitutes a path in G; hence, $L \geq M_N$, and level M2 generalizes M1: when the arc set is empty ($E = \varnothing$), $L = M_N$ and $B_2 = B_1$. Equality $T_{ai} = B_2$ is not guaranteed in general because scheduling on a limited number of processors is NP-hard.

4.5.1 Attainability and Speedup Guarantees for M1–M2

Remark 7 (Attainability Guarantee: Graham's Bound)**.** At levels M1–M2, with an integer number P of identical processors and $C(P) \equiv 1$, the lower bounds B_1 and B_2 can be supplemented with an attainability guarantee. The classical result of Graham for fixed-priority list scheduling, under which a processor does not remain idle whenever a ready task is available, yields the inequality [17]

$$T_{ai} \;\leq\; \frac{W}{P} + \left(1 - \frac{1}{P}\right) L \;\leq\; \left(2 - \frac{1}{P}\right) B_2.$$

For independent tasks (level M1), the same relations hold with L replaced by M_N and B_2 by B_1. Thus, the optimal makespan T^* lies in the interval

$$B_j \;\leq\; T^* \;\leq\; \left(2 - \frac{1}{P}\right) B_j, \qquad j \in \{1, 2\}.$$

Consequently, the achievable speedup differs from the upper bound $\bar{S}^{(j)} = T_h / B_j$ by at most a factor of $2 - 1/P$:

$$\frac{\bar{S}^{(j)}}{2 - 1/P} \;\leq\; S_E^{\text{ach}} \;\leq\; \bar{S}^{(j)}.$$

For independent tasks ordered by nonincreasing duration (the LPT rule), the stronger guarantee $T_{\text{LPT}} \leq (4/3 - 1/(3P))\, T^*$ is known [18]. This constant, however, is relative to the optimum T^* rather than to the lower bound B_j and therefore does not carry over to the interval above. The ratio T^*/B_1 may exceed $4/3 - 1/(3P)$: for example, with three equal-duration tasks and $P = 2$, $T^*/B_1 = 4/3 > 4/3 - 1/6$. These guarantees do not extend directly to levels M3–M4 because Graham's formulation does not include coordination overhead or a shared human resource. Such resources are considered in more general scheduling models, including the RCPSP and parallel-machine problems with a single shared server [8, 20, 24].

Theorem 1 (Sufficient Speedup Criterion at Levels M1–M2)**.** Suppose that $C(P) \equiv 1$ and that P is an integer number of identical processors. Define the relative bottleneck length as

$$\mu := \frac{M_N}{T_h} \quad \text{(level M1)}, \qquad \mu := \frac{L}{T_h} \quad \text{(level M2)}.$$

If

$$\bar{k}_w < P - (P - 1)\, \mu, \tag{5}$$

then speedup is guaranteed: every greedy schedule satisfies $T_{ai} < T_h$. More generally, a target speedup of $S_E^{\text{ach}} \geq S_{\text{target}} \geq 1$ is guaranteed if

$$\bar{k}_w \;\leq\; \frac{P}{S_{\text{target}}} - (P - 1)\, \mu. \tag{6}$$

When $S_{\text{target}} = 1$, the non-strict version of condition (6) guarantees no slowdown, $S_E^{\text{ach}} \geq 1$, whereas its strict version coincides with (5) and guarantees a genuine speedup, $S_E^{\text{ach}} > 1$. As $\mu \to 0$ (in the absence of a dominant bottleneck), the strict sufficient criterion converges to the necessary condition $\bar{k}_w < P$ from Remark 4. The gap between the necessary and sufficient conditions is $(P-1)\,\mu$—the "cost of indivisibility" at M1 or the "cost of dependencies" at M2.

Proof. By Remark 7, every greedy schedule satisfies $T_{ai} \leq W/P + (1 - 1/P)\,L$ (at level M1, with M_N in place of L). Dividing both sides by T_h and using $W = \bar{k}_w\,T_h$ gives

$$\frac{T_{ai}}{T_h} \;\leq\; \frac{\bar{k}_w}{P} + \left(1 - \frac{1}{P}\right)\mu.$$

The right-hand side is at most $1/S_{\text{target}}$ precisely under condition (6) (after multiplication by P and rearrangement), which yields $S_E^{\text{ach}} = T_h/T_{ai} \geq S_{\text{target}}$; condition (5) is obtained by setting $S_{\text{target}} = 1$ and imposing a strict inequality. □

Remark 8 (Equivalent Form via Bottleneck Dominance)**.** Define relative bottleneck dominance as the ratio of the longest task (or, at M2, the critical path) to the mean workload per processor:

$$b := \frac{M_N}{W/P} \quad \text{(M1)}, \qquad b := \frac{L}{W/P} \quad \text{(M2)}.$$

The parameters satisfy the exact relation $\mu = b\,\bar{k}_w/P$; hence, criterion (5) can equivalently be written as

$$\bar{k}_w \left(1 + \left(1 - \frac{1}{P}\right) b\right) < P.$$

One inequality holds if and only if the other does: this is the same criterion expressed in coordinates $(\bar{k}_w, b)$ rather than $(\bar{k}_w, \mu)$.

The form involving b yields two simplified practical conditions. They are *not equivalent* to the original criterion but provide a more conservative bound: each condition is strictly stronger than (5), so satisfying it still guarantees speedup.

1. **Coefficient simplification.** Replacing the factor $1 - 1/P < 1$ by one gives the condition
$$\bar{k}_w\,(1 + b) < P,$$
whose correction term is independent of P for fixed b. It implies (5), but not conversely: criterion (5) may hold even when $\bar{k}_w(1 + b) \geq P$. When configurations with different values of P are compared, $b = b(P)$ must be recomputed together with W/P.

2. **Fixing b as a decomposition rule.** If the decomposition guarantees $b \leq b_0$, meaning that no task and no dependency path exceeds b_0 times the mean workload per processor, then the condition
$$\bar{k}_w\,(1 + b_0) < P$$
is sufficient for speedup regardless of the details of the particular task set. It is not necessary: the actual bottleneck dominance may be lower than b_0, and speedup may be possible even when the condition is violated. This yields a practical recommendation: decompose the work until $b \leq b_0$ and maintain a parallelism margin such that $P > \bar{k}_w(1 + b_0)$.

4.6 Level M3: Coordination Overhead

Remark 9 (Effective Weights and Lower Bound B_3)**.** To account for cross-stream integration and rework arising when shared artifacts are modified in parallel, we introduce a function $C(P) \geq 1$. We assume that $C(1) = 1$ and that $C(P)$ is nondecreasing in P; for any particular process, this assumption requires empirical verification. The function applies only to autonomous agent work, not to human-attention phases. For a fixed configuration, $C(P)$ is treated as an exogenous constant: it does not depend on the task assignment, resource ordering, or realized queues. Define

$$A := X \sum_{i=1}^{N} Z_i a_i, \qquad H := X \sum_{i=1}^{N} Z_i h_i, \qquad W = A + H.$$

Under a mixed assignment of modes, here and below a_i and h_i denote $a_i^{(\rho(i))}$ and $h_i^{(\rho(i))}$, respectively. The effective task duration and total agent-stream workload at level M3 are

$$w_i^{(3)}(P) := Z_i X\big(C(P)a_i + h_i\big), \qquad W_3(P) := \sum_i w_i^{(3)}(P) = C(P)A + H.$$

Let $L_3(P)$ be the length of the critical path in G under the weights $w_i^{(3)}(P)$. Then

$$T_{ai} \geq B_3 := \max\left\{\frac{W_3(P)}{P},\ L_3(P)\right\}.$$

This definition eliminates the asymmetry of the previous aggregate formulation: the additional integration overhead increases not only the total workload but also the paths on which that overhead actually arises.

The speedup condition for the aggregate-workload term in the model with coordination overhead is

$$\frac{C(P)A + H}{P} < T_h \iff C(P)\,\bar{a}_w + \bar{h}_w < P,$$

where $\bar{a}_w := \sum_i Z_i a_i / \sum_i Z_i$ and $\bar{h}_w := \sum_i Z_i h_i / \sum_i Z_i$.

To distinguish $k_i^{(r)}$ from parallelism overhead, we adopt the following accounting rule. The coefficient $k_i^{(r)}$ includes everything that can be measured at $P = 1$ in mode r: prompt formulation, waiting for generation, and reviewing and correcting the result within a single stream. Parallelism overhead includes only effects that arise when $P > 1$. Without this rule, the same costs may be counted twice—both in k and in the multipliers used when $P > 1$—or, conversely, may remain unaccounted for if coordination overhead is implicitly included in the measured k while the additional multipliers are set to one.

Given the decomposition $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$ (Definition 1) and the star topology "one developer—P agents," the overhead is divided into three groups:

1. coordination overhead *at the individual-task level*—preparing the specification, transferring context, and reviewing the result—does not depend on P and is already included in $k_i^{(r)}$; it therefore increases task weights, including those on the critical path;

2. *developer* overhead that grows with P—switching between streams and restoring context when handling requests—is attributed to the human resource and is accounted for by applying the multiplier $\gamma(P)$ to H at level M4 (Remark 10);

3. the function $C(P)$ represents *cross-stream integration overhead* that increases with the number of parallel streams: merge conflicts, interface inconsistencies, and rework caused by concurrent changes.

As a working parametric form, one may use the denominator of Gunther's Universal Scalability Law: $C(P) = 1 + \alpha(P-1) + \beta P(P-1)$, where $\alpha, \beta \geq 0$, α characterizes resource contention, and β characterizes coordination costs [19]. Applying this relationship to coding agents is a hypothesis of the present model, not an empirical finding of the cited work. Brooks's formulation, with $\sim P(P-1)/2$ pairwise communication channels, pertains to a human team and does not literally describe the star topology. Nevertheless, indirect coupling through shared code, interfaces, and integration persists even in the absence of direct communication among agents [7].

4.7 Level M4: Phase Semantics and the Exact Schedule

Remark 10 (Local Blocking and Lower Bound B_4)**.** At level M4, the decomposition $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$ (Definition 1) is augmented by the ordering of phases. Task i is represented by a finite sequence $\mathcal{F}_i = (f_{i1}, \ldots, f_{im_i})$. Agent phases include autonomous implementation, self-review, and rework; human-attention phases include task specification, requirements clarification, review, and acceptance. At $P = 1$, the sums of their durations are $Z_i X a_i$ and $Z_i X h_i$, respectively. The phases of a task follow a prescribed order, so the model permits multiple requests for developer attention and repeated "review—correction—re-review" cycles.

Once the first phase has begun, the task remains assigned to a single agent slot until its result is accepted. A human-attention phase simultaneously requires the developer's attention and retention of the allocated agent slot. If the developer is busy, the request enters the queue, while the corresponding agent stops working and remains blocked. The other agents continue their tasks independently until they themselves request the developer's attention. Blocking is therefore *local*, not global.

At any time, unfinished assigned tasks occupy no more than P agent slots, and the developer handles no more than one human-attention phase. The model does not permit a blocked slot to be reassigned to another task or an additional priority slot to be launched beyond the limit P. A new task may be assigned only to an available agent. If $(i, j) \in E$, the first phase of task j may begin only after task i has been accepted. Agent and human-attention phases are assumed to be nonpreemptive; the order in which simultaneously pending requests are handled is part of the schedule π.

For $P > 1$, the aggregate effective duration of the human-attention phases is taken to be $\gamma(P)H$, where $\gamma(P) \geq 1$ and $\gamma(1) = 1$. The multiplier captures the additional service time due to attention switching and context recovery, but excludes time spent waiting in the queue. Like $C(P)$, it is treated as exogenous for a fixed configuration and does not depend on the particular schedule. The reduced-form approximation $\gamma(P) = 1 + \delta(P-1)$, $\delta \geq 0$, is used; in empirical calibration, it should be related to the actual number of streams supervised concurrently and the frequency of interventions.

Define the effective weights and aggregates at level M4 as follows:

$$w_i^{(4)}(P) := Z_i X \big(C(P) a_i + \gamma(P) h_i\big),$$

$$W_4(P) := \sum_i w_i^{(4)}(P) = C(P)A + \gamma(P)H,$$

and let $L_4(P)$ denote the critical-path length under the weights $w_i^{(4)}(P)$. The resulting fixed lower bound on time is

$$T^* \;\geq\; B_4 := \max\left\{\frac{W_4(P)}{P},\; L_4(P),\; \gamma(P)H\right\},$$

where $T^* := \min_\pi T(\pi)$ is the optimal makespan over feasible schedules and $T(\pi)$ is the makespan of a particular schedule π.

Let $Q(\pi)$ denote the total time for which already assigned agents are blocked while waiting in the queue for developer attention. This time is induced by the schedule itself and is included in neither k_i nor H. Every feasible schedule additionally satisfies

$$T(\pi) \geq \frac{W_4(P) + Q(\pi)}{P}.$$

Consequently, the equality $T^* = B_4$ is not guaranteed: simultaneous requests for developer attention can increase the makespan even when H is small.

For example, suppose that, of two agents working in parallel, Agent A requests assistance. It stops and retains its slot until the developer handles the request. Agent B continues working. If Agent B subsequently also requires assistance, both agents will be waiting in the queue for the same developer; the resulting systemwide idle time is a consequence of coincident requests, not a rule imposing global blocking.

The aggregates A, H, W_4, L_4, and B_4 characterize the workload and the three necessary constraints, but do not encode the relative placement of the human-attention phases. Hence, two instances with identical values of these aggregates can have different values of T^*: their lower bound is the same, whereas queueing time and attainability of the bound are determined by the full phase profile. A computational example of this discrepancy is presented in Section 6.

Definition 2 (Resource-Augmented Phase Graph)**.** Fix the assignment of each task to one of the P agent slots, the order of tasks within each slot, and the service order of all human-attention phases. If these orders are compatible with the original dependencies, construct a directed graph $G_R(\omega)$ whose vertices are the phases and whose vertex weights are their effective durations, incorporating the multiplier $C(P)$ for an agent phase or $\gamma(P)$ for a human-attention phase. Four types of arcs are added to the graph:

1. between consecutive phases of the same task;
2. from the final phase of a predecessor to the first phase of a dependent task;
3. between consecutive human-attention phases in the selected service order;
4. from the final phase of one task to the first phase of the next task assigned to the same agent slot.

Let ω denote the collection of assignment and resource-ordering decisions. Only those ω for which $G_R(\omega)$ is acyclic are considered feasible. Let $\Lambda(\omega)$ denote the length of a maximum-weight path in this graph.

Proposition 1 (Exact Representation of the M4 Optimum)**.** For a fixed feasible resource order ω, the earliest-start schedule has makespan $\Lambda(\omega)$ and is feasible. Conversely, every feasible schedule π induces an order $\omega(\pi)$ such that $\Lambda(\omega(\pi)) \leq T(\pi)$. Therefore,

$$T^* = \min_{\omega\in\Omega} \Lambda(\omega),$$

where Ω is the finite set of feasible assignments and resource orders.

Proof. For a fixed acyclic graph, the earliest start time of each phase equals the maximum path length leading to that phase. Arcs of the first two types enforce the technological precedence constraints, those of the third type prevent human-attention phases from overlapping, and those of the fourth type prevent tasks assigned to the same slot from overlapping. The resulting schedule is therefore feasible and completes in $\Lambda(\omega)$. Conversely, the nonoverlapping operations in any schedule can be ordered on each resource; all added arcs are consistent with the actual timing and therefore create no cycle, and the length of any path does not exceed $T(\pi)$. Minimizing the two representations yields the equality. □

This representation serves as a *resource-augmented critical path*: the original critical path is augmented with arcs capturing competition for the developer and agent slots. It does not introduce a new level M5 because it adds no constraint to M4; rather, it exposes the exact combinatorial structure of the schedule already specified. This construction is closely related to disjunctive graphs and resource-constrained scheduling models [8].

Corollary 1 (Attainability Certificate for the Lower Bound)**.** For every M4 instance, $B_4 \leq T^*$. The equality $T^* = B_4$ holds if and only if there exists a feasible schedule π with $T(\pi) = B_4$; equivalently, there exists an $\omega \in \Omega$ with $\Lambda(\omega) = B_4$. An explicit schedule of this length constitutes an attainability certificate. If the solver proves that no schedule with horizon B_4 exists, the bound is unattainable and $T^* > B_4$.

Corollary 2 (Simple Tightness Cases)**.** If $P = 1$ or the task-DAG is a single precedence chain containing all tasks, then $T^* = B_4 = W_4$.

Proof. When $P = 1$, we have $C(1) = \gamma(1) = 1$, and all tasks sequentially occupy the sole slot. They can be executed in any topological order without idle time, so the makespan equals the sum W_4 of their effective phase durations; the other terms defining B_4 do not exceed this sum. In a single precedence chain, no two tasks can overlap for any P, its length equals the sum of the weights, $L_4 = W_4$, and a sequential schedule attains this bound. □

Attainability of the horizon B_4 can be checked as a schedule-feasibility problem, whereas T^* can be obtained by directly minimizing the makespan. The specific computational formulation and verification protocol are presented in Section 5.

Proposition 2 (Monotonicity of Available Capacity)**.** If phase durations and the parameters C and γ are fixed and do not change when an agent stream is added, then

$$T^*(P+1) \leq T^*(P).$$

Proof. Any schedule for P agent streams remains feasible with $P+1$ streams: the additional stream may be left unused. Therefore, the set of feasible schedules can only expand. □

Proposition 3 (Existence of a Finite Optimal Configuration)**.** Consider configurations with

$$C(P) = 1 + \alpha(P-1) + \beta P(P-1), \qquad \gamma(P) = 1 + \delta(P-1),$$

where $P \in \mathbb{N}$. If either $A > 0$ and $\beta > 0$, or $H > 0$ and $\delta > 0$, then $T^*(P) \to \infty$ as $P \to \infty$; hence, the minimum of $T^*(P)$ is attained at a finite value of P.

Proof. In the first case, $W_4(P)/P \geq C(P)A/P \sim \beta AP$; in the second, $B_4 \geq \gamma(P)H \sim \delta HP$. In both cases, $T^*(P) \geq B_4(P) \to \infty$. A sequence indexed by the natural numbers that tends to infinity attains its global minimum on a finite initial segment. □

Propositions 2 and 3 address different questions. The former rules out deterioration due to unused available capacity when durations remain unchanged. The latter compares distinct organizational configurations in which an increase in configured parallelism itself changes the overhead factors $C(P)$ and $\gamma(P)$. The specified increasing penalties guarantee a minimum at a finite P, but not an interior minimum: the optimum may occur at $P = 1$. For example, a single task with $A = H = 1$, $C(P) = 1$, and $\gamma(P) = P$ has $T^*(P) = 1 + P$ and is fastest at $P = 1$.

Proposition 4 (Human-Resource Speedup Ceiling)**.** For a fixed set of tasks and mode assignment ρ, if $H > 0$, then for every $P \geq 1$ and any $C(P) \geq 1$, $\gamma(P) \geq 1$:

$$S_E^{\text{ach}} := \frac{T_h}{T_{ai}} \;\leq\; \frac{T_h}{\gamma(P)\,H} \;\leq\; \frac{T_h}{H} \;=\; \frac{1}{\bar{h}_w}, \qquad \bar{h}_w := \frac{\sum_{i=1}^{N} Z_i h_i^{(\rho(i))}}{\sum_{i=1}^{N} Z_i}.$$

Proof. The human-attention intervals of all tasks are handled by a single developer and therefore do not overlap pairwise; their total duration is at least H (and at least $\gamma(P)H$ after accounting for attention switching). Consequently, $T_{ai} \geq \gamma(P)H \geq H$, which implies $S_E^{\text{ach}} = T_h/T_{ai} \leq T_h/H = 1/\bar{h}_w$. □

When $H = 0$, there are no mandatory human-attention phases, so the constraint is trivial and the human-resource ceiling is conventionally taken to be infinite. The other branches of B_4 and the scheduling constraints remain in force.

Remark 11 (Analogy with Amdahl's Law)**.** Proposition 4 is a structural analogue of Amdahl's law: the weighted-average normalized human component $\bar{h}_w$ plays the role of the sequential component [1]. The ceiling $1/\bar{h}_w$ is independent of P. If the target speedup exceeds this value, it cannot be achieved by adding agents; instead, the human component h must be reduced through more autonomous modes, automated verification, or formalized acceptance criteria. At level M4, the upper bound on speedup based on aggregate stream occupancy is $P/[C(P)\bar{a}_w + \gamma(P)\bar{h}_w]$, whereas the human-resource bound is $1/[\gamma(P)\bar{h}_w]$. These two aggregates are useful for rapid diagnosis, but they do not account for the critical path or the queue for developer attention.

4.8 Cross-Level Properties

Under a consistent parameterization, successive refinements preserve the constraints imposed at the preceding levels. Because $M_N \leq L$, $C(P) \geq 1$, and $\gamma(P) \geq 1$, we have $W_3(P) \geq W$, $L_3(P) \geq L$, $W_4(P) \geq W_3(P)$, and $L_4(P) \geq L_3(P)$. The lower bounds are therefore ordered as follows:

$$B_0 \leq B_1 \leq B_2 \leq B_3 \leq B_4.$$

Consequently, the upper bounds on achievable speedup, $\bar{S}^{(j)} := T_h/B_j$, decrease monotonically as j increases. At level M0, the completion time is exactly equal to the bound ($T_{ai}^{(0)} = B_0$), and Observation 1 gives an algebraically equivalent speedup criterion in the form of a necessary and sufficient condition. Starting at level M1, B_j serves only as a lower bound. The actual completion time is determined by the assignment of tasks to processors and may strictly exceed B_j even under an optimal schedule; an example of such a gap for B_1 is given in the calculation section. The speedup condition based on the linear component remains necessary but is no longer sufficient (Remark 4).

The correspondence with the notation of the probabilistic extension of the model is as follows: the linear component at level M3 corresponds to $T_{ai}^{\text{lin}} := W_3(P)/P$, while the correction for indivisibility requires the maximum weight $\max_i w_i^{(3)}(P)$. The dependency graph, phase queue, and shared human resource are not yet considered in the probabilistic extension.

4.9 Scale Invariance and Comparative Statics

Proposition 5 (Scale Invariance of Rankings)**.** Fix a project decomposition—a set of tasks with task sizes Z_i and a dependency graph G—and a configuration σ with an integer number P of agent streams. At level M4, also fix, for each task, the number, types, and order of phases, the precedence relations, the local-blocking rule, $C(P)$, and $\gamma(P)$. A scale transformation multiplies the duration of *every* agent and human-attention phase by the same common factor $\kappa > 0$; accordingly, both a_i and h_i are scaled, while their decomposition and phase profile are preserved. Let $T^*(\kappa)$ denote the optimal makespan after this transformation. Then:

1. $T^*(\kappa) = \kappa T^*(1)$ (homogeneity of degree one); the lower bounds B_0–B_4 have the same property if $C(P)$ and $\gamma(P)$ do not depend on κ. The optimal assignment of tasks to agent streams, the queue service order, the active term in $\max\{W_4(P)/P,\, L_4(P),\, \gamma(P)H\}$, and the composition of the critical path are independent of κ;

2. if, for two work-organization variants A and B, the complete duration vectors of the corresponding agent and human-attention phases are known up to the same common factor $\kappa > 0$, that is, $\mathbf{d}^A = \kappa\, \hat{\mathbf{d}}^A$ and $\mathbf{d}^B = \kappa\, \hat{\mathbf{d}}^B$, then the ratio T_A^*/T_B^* and hence the ranking of the variants are independent of κ. In particular, a_i, h_i, and $k_i = a_i + h_i$ must scale consistently; proportionality of the aggregate k_i values alone is insufficient when the phase decomposition changes.

Proof. When the durations of all phases are multiplied by κ, task durations, human service times, and the waiting intervals induced by the same event order are all multiplied by κ. Any feasible schedule is transformed into a feasible schedule for the scaled instance by multiplying all start and completion times by κ. The phase order, precedence relations, local slot retention, and resource constraints are preserved. The transformation is invertible and establishes a bijection between the sets of feasible schedules. Therefore, $T^*(\kappa) = \kappa T^*(1)$, and an optimal schedule is mapped to an optimal schedule. The homogeneity of B_j follows from the linearity of W, M_N, L, W_3, L_3, W_4, L_4, and H in κ. Claim (ii) follows from (i): $T_A^*(\kappa)/T_B^*(\kappa) = T_A^*(1)/T_B^*(1)$. □

Remark 12 (Comparative Statics without Exact Calibration)**.** Proposition 5 clarifies the capabilities of the model. An absolute prediction—the value of T_{ai} in hours or of S_E—requires knowledge of the scale of the durations and inherits calibration error on that same scale (Remark 14). However, the *choice among decomposition and work-organization variants* is independent of the common scale if the complete relative profiles of the agent and human-attention phases are known up to a single common factor.

The model also indicates the direction of beneficial changes without requiring exact calibration. The lower bound B_2 is piecewise linear in the vector $k = (k_1, \ldots, k_N)$. Its elasticities with respect to the individual coefficients can be computed explicitly; when the

branch $L > W/P$ is active, the expression assumes that the critical path π^* is unique:

$$\frac{\partial B_2}{\partial k_i} \cdot \frac{k_i}{B_2} = \begin{cases} \dfrac{Z_i k_i}{\sum_j Z_j k_j}, & \text{if } W/P > L, \\ \dfrac{Z_i k_i X}{L}, & \text{if } L > W/P,\ \pi^* \text{ is unique and } i \in \pi^*, \\ 0, & \text{if } L > W/P,\ \pi^* \text{ is unique and } i \notin \pi^* \end{cases}$$

At points where the active terms switch, and in the presence of multiple critical paths, B_2 may be nondifferentiable; in that case, directional derivatives determined by all active paths are used. When the L branch is active, increasing P does not reduce the lower bound B_2 itself, although the makespan T^* may decrease until it reaches this bound. Similarly, when the $\gamma(P)H$ branch is active, a task's share of H describes the sensitivity of B_4, but this sensitivity may be transferred to T^* only when tightness, $T^* = B_4$, has been established.

The invariance has three limitations. First, it applies only to a *common multiplicative* error across all phases. If estimation errors distort the relative task durations, the decomposition between a_i and h_i, or the within-task phase profile, the composition of the critical path and the resource-augmented critical path, as well as the ranking of the variants, may change. Second, the factor κ must be common to all phases and all variants being compared. In other words, the systematic bias of the estimation procedure (Remark 14) is assumed to be the same across configurations used by a given team under a common measurement protocol. Third, comparing different modes requires knowledge of the relative mode-specific profiles separately for agent and human-attention phases. This requirement is substantially weaker than knowing their absolute durations but stronger than knowing only the aggregate coefficients k_i.

4.10 Calibration and Uncertainty

Remark 13 (Stochastic Nature of the Coefficients k_i)**.** In practice, AI-assisted task execution is nondeterministic. It is therefore useful to treat the coefficient k_i as a random variable whose distribution reflects uncertainty in code generation, the probability of defects, and the time required to correct them. Then T_{ai} also becomes a random variable, and the speedup condition should be evaluated for the expectation, $\mathbb{E}[T_{ai}] < T_h$, or for selected quantiles of the distribution.

By linearity of expectation, note that $\mathbb{E}[T_{ai}^{(0)}] = \frac{X}{P} \sum_{i=1}^{N} Z_i \, \nu_i$, where $\nu_i = \mathbb{E}[k_i]$. Therefore, the equivalent level-M0 criterion for deterministic coefficients (Observation 1) carries over verbatim to expectations:

$$\mathbb{E}[T_{ai}^{(0)}] < T_h \iff P > \frac{\sum_{i=1}^{N} Z_i \nu_i}{\sum_{i=1}^{N} Z_i}.$$

A systematic treatment of the stochastic formulation, including variance and quantile guarantees, is deferred to a separate probabilistic model.

Remark 14 (Estimating k_i before Project Start)**.** The operational definition of $k_i^{(r)}$ (Definition 1) is retrospective: the coefficient is calculated from the observed duration $T_i^{ai}(r)$ and is not itself a forecast. Applying the speedup criteria (Observation 1, Remark 4, Theorem 1) *before* the project starts requires an external estimate $\hat{k}_i^{(r)}$. It can be obtained as follows:

1. tasks are classified by type (CRUD operations, integration with an external API, tests, infrastructure, etc.); let $c(i)$ denote the type of task i;

2. a sample of k values is accumulated from the team's tasks for "type $\times$ mode" pairs; separate time tracking also makes it possible to record the human component h (Definition 1). Failed runs, timeouts, and unaccepted results must be explicitly included either as observations or as censored cases: a sample containing only completed tasks estimates a conditional duration and is subject to survivorship bias;

3. the estimate $\hat{k}_i^{(r)}$ is taken to be the empirical mean for the pair $(c(i), r)$—for the expectation-based criterion (Remark 13)—or an upper sample quantile for a conservative decision.

At level M0, the sensitivity of the result to estimation error follows directly from $S_E = P/\bar{k}_w$: a relative error in $\bar{k}_w$ is transferred to S_E on the same multiplicative scale (overestimating $\bar{k}_w$ by a factor of $(1+\varepsilon)$ underestimates S_E by the same factor). For comparing configurations at M0, the absolute level of k is less important: a common multiplicative error does not change their ranking. At M4, the same conclusion requires the stronger premise of Proposition 5: a single factor must scale all agent and human-attention phases while preserving their types and order. Thus, the model has a dual status. In retrospective applications, it specifies an exact identity; when it is used for decision-making, the accuracy of the absolute predictions T_{ai} and S_E^{ach} is determined by the quality of the estimate $\hat{k}$ and of the phase profile. Comparative conclusions are robust only to a common systematic error in the sense of Remark 12.

5 Computational Verification Method

The computational experiments are designed to check the internal logic of M4, identify counterexamples to overly strong interpretations, and conduct sensitivity analysis. They do not estimate the performance of a particular team or AI tool: the durations, topologies, and overhead functions are predominantly synthetic. All sample sizes reported below refer either to fully enumerated frozen matrices or to a reproducible generator, rather than to a random sample of real-world projects.

5.1 Experimental Questions and Reporting Rules

Each series addresses a prespecified question and uses its own estimand. Table 2 establishes this correspondence before the results are presented and prevents a certified optimum, the makespan of a fixed policy, and an observation on a finite synthetic grid from being conflated.

Table 2: Relationship between the computational series and the claims being tested

Series	Question	Varied factors	Estimand and reporting rule	Method
EXP-00	Are the variants of the illustrative scenario reproduced?	Configurations 2–4	B_4, T^*; certificate of $T^* = B_4$	exact
EXP-01	Is the bound B_4 always attainable?	DAG, phases, P	$T^*/B_4 - 1$; certified positive gap	exact
EXP-02	How does available capacity differ from configured parallelism?	P, $C(P)$, $\gamma(P)$, human-share	Profile of $T^*(P)$	exact
EXP-03	Do the aggregates identify the exact makespan?	Phase profile with equal aggregates	Paired difference in T^*	exact
EXP-04–05	How does granularity interact with P and overhead?	Decomposition, P, overhead type and magnitude	Optimal granularity and interaction contrast	exact
EXP-06	Are the scale and comparative conclusions robust?	Common scale and frozen error families	Invariance and preservation of ranking	exact/stress
EXP-07	Do the directions of the effects carry over to synthetic DAGs?	N, topology, weights, placement of human-work	Paired contrasts for $T(\pi)$; separate exact audit	mixed
EXP-08	What changes when a fixed H is placed differently?	Concentration along a path, phases, P, γ/C	Paired changes in aggregates, paths, and $T(\pi)$; exact audit	mixed

All reports distinguish among four quantities: the structural lower bound B_4, the certified optimum T^*, a feasible solution found without proof of optimality, and the makespan $T(\pi)$ of a fixed policy. The notation T^* is used only for runs with status `OPTIMAL` in the computational reports. Independently of solver status, a verified feasible schedule with $T(\pi) = B_4$ certifies optimality through $B_4 \leq T^* \leq T(\pi) = B_4$. Equality must hold at the exact numerical precision of the instance; coincidence of rounded displayed values is insufficient.

5.2 Common Semantics and Experimental Quantities

All series use the same M4 semantics: once a task has started, it retains one agent slot through its final phase, including while waiting for developer attention; all human-attention phases use a single nonpreemptive resource of capacity 1; and a successor becomes available after all its predecessors have completed. The multiplier $C(P)$ is applied to agent phases, whereas $\gamma(P)$ is applied to human-attention phases. The solver minimizes only the

makespan; queueing and total blocking are reported as derived characteristics but do not serve as secondary objectives. Therefore, the reported Q pertains to one returned optimal schedule, not to the minimum Q among all makespan-optimal schedules. In the exact model, a task is assigned when its first phase begins; the fixed simulation policy reserves a slot in advance when it selects a ready task. Its queue and makespan are specific to that policy.

For series with an equal share of human work, define

$$r_h := \frac{h_i}{k_i}.$$

In the canonical instances of EXP-01 and EXP-03, $k_i = 1$ and r_h is the same for all tasks; in the illustrative instance of EXP-02, the original k_i is retained and $h_i = r_h k_i$. The `io` profile divides H_i equally around the single agent phase: $H_i/2, A_i, H_i/2$. The `front_loaded` profile uses $0.9H_i, A_i, 0.1H_i$. In the equal-alternation profile, the four human-attention phases have durations $H_i/4$, and the three agent phases have durations $A_i/3$. The historical label `alternating_sync` denotes precisely this within-task equality and *does not* require requests from different tasks to coincide in wall-clock time. In `alternating_staggered`, the human-attention phases are the same, while the agent fractions $\{1/6, 2/6, 3/6\}$ are cyclically permuted across tasks. In EXP-08, the neutral name `equal_alternating` is used for equal alternation. Only in EXP-08 are the nominal halves, quarters, and thirds implemented on an allocation grid of 0.001: an integer number of ticks is divided with a remainder, which is distributed according to a seeded permutation. The sum of the phases is preserved exactly, and parts of the same type may differ by at most one tick.

The experimental granularity m is the number of parallel subtasks that replace the selected task. Under the canonical EXP-04–EXP-05 operator, its original initial human-attention phase and useful agent work are divided equally among the m subtasks; all of them inherit the predecessors of the original task. They are followed by a join that contains the original final acceptance, preserves the original quality gate, and becomes the predecessor of all former successors. At $m = 1$, this represents semantically the same amount of useful work, although the DAG already contains an explicit join. Therefore, the EXP-04–EXP-05 contrasts refer to the transformed "parts plus join" operator; C3 in EXP-07 uses the original task when $m = 1$, and the numerical values from these two series are not directly comparable. Let E_0 be the original total duration of the selected task, r_o the overhead rate per additional boundary, and λ_h the human share of this overhead. Then

$$O(m, r_o) = (m-1) r_o E_0, \qquad O_h = \lambda_h O, \qquad O_a = (1-\lambda_h) O.$$

Half of O_h and O_a is distributed equally among the inputs to the subtasks, and the other half is added to the join. When $r_o = 0$, the useful work components A and H are preserved exactly. For two levels of parallelism, the complementarity of decomposition is measured by the contrast

$$I(P, m) = \big[T^*(P, 1) - T^*(P, m)\big] - \big[T^*(1, 1) - T^*(1, m)\big]. \tag{7}$$

A positive I means that the decomposition gain at P is greater than at $P = 1$; it does not by itself imply that either gain is positive. The contrast is computed only when all four solver runs return `OPTIMAL`. For a prespecified fixed policy π, define separately

$$I_\pi(P, m) = \big[T_\pi(P, 1) - T_\pi(P, m)\big] - \big[T_\pi(1, 1) - T_\pi(1, m)\big],$$

where T_π is the makespan of this policy, not a certified optimum. In the frozen EXP-04–EXP-05 matrices, tolerance $= 10^{-4}$ is used: a contrast is classified as positive when $I >$ tolerance, negative when $I < -$tolerance, and zero otherwise. On the finite EXP-04 grid, a curve is classified as U-shaped if all its points have been certified as optimal, its minimum is attained at least at one interior value $m \notin \{1, 8\}$, and both endpoints $T^*(1)$ and $T^*(8)$ exceed the minimum by more than tolerance. This operational definition makes no claim of U-shaped behavior outside the integer values of m examined.

Two concentration measures are used for the synthetic DAGs. The quantity $\rho_L = L/W_4$ is the fraction of structural work on the longest path when $C = \gamma = 1$, whereas $\rho_H^{cp} = H_{cp}/H$ is the fraction of all human work located on the selected structural path. They are not interchangeable: the former characterizes the sequential structure of the DAG, whereas the latter characterizes the placement of a fixed or approximately fixed amount of human work.

5.3 Frozen Exact Matrices EXP-00–EXP-05

EXP-00 solves three fully specified variants of the illustrative scenario (variants 2–4). EXP-01 uses the following full factorial design of 90 exact grid points:

$$5 \text{ topologies} \times N{=}4 \times P \in \{2, 4\} \times r_h \in \{0.10, 0.25, 0.50\} \times 3 \text{ profiles} = 90$$

The topologies are chain, independent, fork–join, diamond, and two-layer; the task weights cycle through $6, 12, 18, 24$. The original frozen matrix also included $N = 8, 12$ (270 points), but a prespecified rule required a global reduction in size if any point failed to receive status `OPTIMAL` within the common 60-second time limit. The first such point at $N = 8$ received `FEASIBLE`, after which the complete $N = 4$ matrix was run without individually adjusting the time limits; the original matrix and the decision record were retained.

EXP-02 contains two objects—the DAG of illustrative variant 4 and the canonical fork–join with $N = 4$—and the grid $P \in \{1, 2, 3, 4, 6, 8, 12\}$. Five unique settings are run for each object: three values $r_h \in \{0.10, 0.25, 0.50\}$ with $C = \gamma = 1$, and, separately at $r_h = 0.25$,

$$C(P) = 1 + 0.05(P - 1) + 0.005P(P - 1) \quad \text{and} \quad \gamma(P) = 1 + 0.10(P - 1).$$

The shared setting $r_h = 0.25, C = \gamma = 1$ is not duplicated, so a total of $2 \times 5 \times 7 = 70$ points are solved.

EXP-03 crosses two DAGs (fork–join and two-layer), $P \in \{2, 4\}$, $r_h \in \{0.25, 0.50\}$, and four profiles (`io`, `front_loaded`, equal alternation, and staggered alternation), for a total of 32 exact problems. For each of the eight "DAG–P–r_h" sets, three profiles are compared with `io`, yielding 24 paired contrasts at identical $A, H, W_4, L_4, B_4, C, \gamma$.

EXP-04A reuses the certified illustrative pair of variants 3–4. EXP-04B applies the operator described above to the sink task $E_0 = 24$ in the canonical two-layer DAG with $N = 4$, $P = 4$, $r_h = 0.25$, and the `io` profile. The grid

$$m \in \{1, 2, 3, 4, 6, 8\}, \quad r_o \in \{0, 0.025, 0.05, 0.10, 0.20\}, \quad \lambda_h \in \{0, 0.5, 1\}$$

contains 90 formal combinations and 66 semantically distinct scenarios: when $m = 1$, overhead is zero for all r_o, λ_h, and when $r_o = 0$, the values of λ_h coincide. After the shared points are restored, 13 substantively distinct curves are analyzed.

EXP-05 uses the same object and operator but crosses $P, m \in \{1, 2, 3, 4, 6, 8\}$, $r_o \in \{0, 0.05, 0.10\}$, and $\lambda_h \in \{0, 0.5\}$. The 216 formal positions yield 156 unique problems; their solutions restore 180 analytical positions after identical zero-overhead slices are merged. The first common pass with a 120-second limit produced 155 `OPTIMAL` results and one `FEASIBLE` result; the single increase in the limit to 300 seconds allowed by the matrix specification was then applied uniformly to all 156 points, rather than only to the difficult one.

5.4 Robustness, Synthetic DAGs, and Confirmatory Freeze

EXP-06 is divided into three parts. EXP-06A scales four prespecified instances (illustrative Variants 3–4 and two canonical points) by $\kappa \in \{0.25, 0.5, 1, 2, 4\}$, jointly scaling X, the phases, the time unit, and the tolerance; this yields 20 exact-solver points with an unchanged integer model. EXP-06B compares Variants 3–4 at five levels $u \in \{0.05, 0.10, 0.20, 0.30, 0.50\}$ and 1000 seeds per level: 5000 pairs, or 10000 exact-solver instances. The agent and human multipliers are sampled independently and uniformly in log space over $[\log(1-u), \log(1+u)]$; tasks common to both variants receive identical multipliers, while the decomposed test tasks inherit the multiplier of the original task with an independent bounded log-residual of scale 0.25. EXP-06C tests two prespecified adverse directions over $u = 0, 0.005, \ldots, 0.95$: the original test task in Variant 3 receives the multiplier $1-u$, whereas the corresponding decomposed tasks in Variant 4 receive $1+u$ on the selected agent or human resource. This yields 382 pairs, or 764 exact-solver instances. These error families are synthetic and do not represent confidence intervals for empirical calibration.

EXP-07 is a stratified pilot comprising 72 cells:

$$N \in \{8, 16, 32, 64\}, \qquad \text{topology} \in \{\text{wide-layered,mixed,chain-like}\},$$
$$\text{weights} \in \{\text{homogeneous,lognormal}\}, \qquad \rho_H^{cp} \in \{\text{low,medium,high}\}.$$

In each cell, 20 sequential observation seeds are accepted, for a total of 1440 scenarios: 480 common "graph plus weights" structural bases are each repeated for three levels of human-work placement. Rejected attempts are retained but are not counted as observations. By construction, the three topology families correspond to the low, medium, and high bins of ρ_L, so the effect of topology family is not separately identifiable from ρ_L. On the full sample, the four frozen directions C1–C4 evaluate a fixed policy or branches of B_4: the contrast between equal and permuted alternation; the shift in policy-optimal P over the grid $\{1, 2, 4, 8\}$ as agent or human cost increases; the policy contrast $I_\pi(4, 3)$ for $m = 3$, $P = 1/4$, $r_o = 0/0.10$, $\lambda_h = 0.5$; and the frequency with which the human branch is active at different values of ρ_H^{cp}. When minimum makespans are tied, policy-optimal parallelism is defined as $P_{\text{opt}} := \min \arg\min_P T_\pi(P)$.

The expected directions were fixed before computation. For C1, the median equal-minus-permuted difference in policy-tightness is expected to be nonnegative. The frozen rule also required a nonnegative upper bootstrap limit, but the implemented decision uses only the sign of the point estimate. Neither rule establishes a positive population median: that would require a lower confidence limit above zero. C1 is therefore reported as a descriptive sign check; its full-sample estimate and interval are both negative. For C2, the median shift in policy-optimal P is expected to be nonpositive; in addition, the proportion of positive shifts must not exceed 0.05. For C3, the medians of $I_\pi(4, 3)$ and of the attenuation of the gain when moving from $r_o = 0$ to $r_o = 0.10$ are expected to be

nonnegative. For C4, both the paired mean of the high-minus-low differences between the human-active-branch indicators and the high-minus-low difference in their frequencies are expected to be positive. These pilot decisions are descriptive checks of the specified directions, not tests controlling a stated type-I error rate. Individual exceptions are retained and reported, so a median direction does not imply a universal sign on every DAG. The intervals use 2000 cluster bootstrap resamples with seed 707071; the common structural base is the resampling unit, so the three human-work placements are not treated as independent.

After 20 accepted seeds in each cell, the frozen matrix prescribed a pilot-extension rule for two key cells. In the first, $N = 16$, the topology family is mixed, the weights are lognormal, and ρ_H^{cp} is medium; in the second, $N = 64$ with the same topology family and weight type, but ρ_H^{cp} is high. Under the protocol, the sample was to be increased to 50 seeds if the bootstrap interval for the primary median crossed zero or the Monte Carlo standard error of the primary proportion exceeded 0.05. The implemented runner checked only whether the global bootstrap intervals for the medians crossed zero; the key-cell and MCSE branches were not executed. The global trigger did not fire, so the extension was not run. EXP-07 should therefore be interpreted as a 20-seed pilot with a partially implemented extension rule, not as a fully completed confirmatory procedure. The prespecified exact audit comprises 12 DAGs with $N \in \{8, 16\}$, homogeneous weights, medium concentration, and seeds 0–1 for the three topology families. Following a single resource adjustment adopted before inspection of the claim outcomes, the exact scope is restricted to the baseline-gap and the two C1 profiles: 36 scenarios with a 10-second limit. C2 and C3 in EXP-07 are policy-only, whereas C4 is lower-bound-only. `FEASIBLE` incumbents are excluded from the certified pairs. EXP-08 resolves the conflation of total H with its placement by using independent development and confirmatory streams. The common grid contains 24 cells defined by $N \times$ topology $\times$ weight type. The development namespace provides 10 bases per cell (240), and the confirmatory namespace provides 20 (480); the raw seed is the first 64 bits of the SHA-256 hash of the namespace, cell, observation number, and attempt number. Confirmatory graphs are generated only when a freeze file is present, and overlaps between their raw seeds and those used for development and EXP-07 are checked automatically.

For each base, a reference path is first selected with $C = \gamma = 1$; ties among longest paths are resolved by a seeded ranking that is independent of task ID, and their multiplicity is retained as a separate indicator. The total share of human work is fixed exactly at $H/W_4 = 0.12$, after which exactly 0.30 or 0.70 of H is placed on the reference path. Allocation within and outside the path is proportional to the structural weights, is subject to task-level share bounds $[0.025, 0.85]$, and is performed by a bounded largest-remainder allocator on a 0.001 grid with seeded tie-breaking. The schedule-effect analysis uses the profiles `io` and `equal_alternating`, $P \in \{1, 4, 8\}$, and $C = \gamma = 1$; the paired "high minus low" difference is normalized by the common B_4. The engineering equivalence margin is 0.01, and the sensitivity margins are 0.005 and 0.02; 2000 bootstrap resamples are performed at the graph-base level with seed 808071. A separate differentiated-slowdown slice uses $C = 1$, $\gamma = 1.3$, and $P = 4$.

The scope of the EXP-08 exact audit was selected solely from the statuses and wall-clock times of the development runs, without retaining their objective values. After screening under common limits of 5 and 30 seconds, the following scope was frozen before confirmatory generation: $N = 8$, homogeneous weights, the mixed and chain-like families, observation indices 0–1, $P \in \{4, 8\}$, and both allocations. This yields 16 confirmatory

scenarios with a 30-second limit; wide-layered and $N = 16$ were excluded uniformly under the timing rule, not on the basis of the effect value.

5.5 Synthetic DAG Generator

EXP-07 and EXP-08 use the same type of layered generator. The number of layers is the rounded value of Nf, clipped to $[2, N]$. With two layers, the source forms the first layer and all other vertices form the second; with three or more layers, the first and last layers each contain one vertex, and the remaining vertices are distributed among the intermediate layers. Every vertex outside the first layer has at least one predecessor in the immediately preceding layer, and every vertex outside the last layer has at least one successor in the immediately following layer. The parameters (f, p_{adj}, p_{skip}) are $(0.20, 0.30, 0)$ for wide-layered, $(0.45, 0.25, 0.04)$ for mixed, and $(0.75, 0.15, 0)$ for chain-like. Only acyclic, weakly connected graphs in which every vertex is reachable from the source are accepted, with $\rho_L \in [0.10, 0.30)$, $[0.30, 0.60)$, or $[0.60, 0.90]$, respectively.

Homogeneous weights are equal to 12. For lognormal weights, the procedure first generates $Y \sim \text{LogNormal}(0, 0.55)$. The multiplier $\min\{6, \max\{1, \text{round}(2Y)\}\}$ is then multiplied by 12. In EXP-07, two task-level human-share values—on and off the structural critical path—are selected from the grid $0.025, 0.05, \ldots, 0.80$: first, the deviation of ρ_H^{cp} from the target 0.18, 0.45, or 0.75 in the corresponding bin is minimized, followed by the deviation of the overall human share from 0.25. This provides approximate, rather than exact, alignment of total H; strictly fixed H is introduced only in EXP-08.

5.6 Exact Solver, Time Grid, and Fixed Policy

The exact instances are formulated as a CP-SAT interval model in OR-Tools 9.15.6755 [28]. Integer start and end times are specified for the phases; the phase order and DAG arcs are imposed through precedence constraints, the human resource through a global `NoOverlap`, and task assignment and local blocking through optional intervals spanning the entire task span on the selected agent stream. A `NoOverlap` constraint is also specified for each stream; the objective is the maximum of the end times of the final phases.

Each scenario specifies a positive `time_unit`. The effective duration of every phase must be exactly divisible by it; otherwise, the scenario validator terminates with an error. Thus, CP-SAT receives an exact integer model rather than rounding continuous durations within the solver. The inverse transformation multiplies integer ticks by `time_unit`. In EXP-06B/C, the perturbed phases are quantized in advance using `ROUND_HALF_UP`, to a minimum of one tick, after which the final phase for each resource is adjusted so that the task total remains exactly representable. EXP-08 separately allocates H on a 0.001 allocation grid and solves the schedule on a 0.0001 grid.

All exact runs use one search worker and solver seed 0. The status, objective, best bound, relative gap, and wall time are retained. A time limit does not constitute a proof of optimality: rows with status `FEASIBLE` retain an incumbent but are not assigned the notation T^*.

For the large samples, the prespecified deterministic policy `bottom_level_fcfs_v1` is used. Let $w_i^{(4)}$ be the effective task weight and

$$b_i = w_i^{(4)} + \max_{j \in \text{succ}(i)} b_j$$

Table 3: Integer grids and limits for the exact solver

Series	Time unit	Common per-scenario limit
EXP-00	0.05	300 s
EXP-01	0.025	60 s
EXP-02	0.00005	60 s
EXP-03	0.025	60 s
EXP-04	0.00625	120 s
EXP-05	0.00625	120 s, followed by a single frozen increase to 300 s
EXP-06A	original grid of the instance, multiplied by κ	60 s
EXP-06B/C	0.05	10 s
EXP-07 exact audit	0.001	10 s
EXP-08 exact audit	0.0001	30 s

its bottom level, where the maximum over the empty set is zero. At each event time, ready unassigned tasks are sorted by decreasing b_i, then by decreasing $w_i^{(4)}$, and finally lexicographically by task ID; available streams are taken in increasing numerical order. A task releases its stream only upon completion of its final phase. Requests for developer attention are handled in order of arrival (FCFS); ties are broken by decreasing b_i, task ID, and phase index. At a common timestamp, the next phases of already assigned tasks are continued first, newly ready tasks are then assigned, and the first human request is served afterward. The policy is nonpreemptive and uses no future information beyond the fixed DAG and durations. Its results pertain to $T(\pi)$, not to T^*; the small exact subsamples serve as an audit and do not replace the primary estimand.

5.7 Reproducibility Environment and Integrity

The EXP-01–EXP-08 matrices, the EXP-00 scenarios, the decision rules, and the seeds are stored with the code. The manifests record the SHA-256 hashes of the input, implementation, and output files; dependency versions; solver settings; and statuses. The package requires Python $\geq 3.11, < 3.14$; the reproduction commands request Python 3.13, while `uv.lock` and `pyproject.toml` pin OR-Tools 9.15.6755, PyYAML 6.0.3, and Matplotlib 3.11.1. The preserved local environment in which this revision was verified reports Python 3.13.14 and macOS 26.5 arm64.

However, the original manifests do not retain the Python patch version, the operating system, the processor, or the amount of memory. Consequently, the specified Python patch version and platform characterize the available local verification but are not established metadata for every original run; wall time cannot be reproduced as a platform-invariant quantity. The numerical conclusions are based on statuses and objectives, not on comparisons of wall time. The commands, EXP-08 order, and complete list of artifacts are provided in Appendix C.

6 Computational Results

6.1 Illustrative Planning Scenario and Reproduction by EXP-00

Consider a synthetic illustrative scenario involving the development of a small request-processing service. The fixed workload comprises an API contract, a persistence layer, business logic, background processing, observability, tests, and packaging. On a notional scale, it equals 38 story points, or 19 units of Z; the normalization $X = 4$ hours yields a sequential no-AI baseline time of $T_h = 76$ hours. This relationship is used solely to make the scenario transparent and is not an empirical conversion from story points to hours.

The task-DAG contains the sequential core $1 \to 2 \to 3 \to 4 \to 6$ and two side branches: observability with packaging, and work that becomes available once the interfaces have been fixed. Four variants of work organization are compared. In Variants 1–3, the operating mode and configured parallelism change, and the coefficients k_i, h_i therefore change as well. Variant 4 retains the configuration of Variant 3 but decomposes the monolithic testing task into three parts with exact dependencies. Only the comparison of Variants 3 and 4 isolates the effect of decomposition with all other factors held constant.

Table 4: Condensed comparison of the illustrative-scenario variants

Metric	Variant 1	Variant 2	Variant 3	Variant 4
P	1	2	4	4
Mode	interactive	mixed	delegated	delegated
Decomposition	coarse	coarse	coarse	split tests
W_4, h	65.0	59.8	53.6	53.6
W_4/P, h	65.0	29.9	13.4	13.4
L_4, h	51.8	47.8	43.2	36.7
H, h	65.0	38.4	19.0	19.0
B_4, h	65.0	47.8	43.2	36.7
T^*, h	65.0	47.8	43.2	36.7
Speedup	1.17×	1.59×	1.76×	2.07×

In EXP-00, the exact solver returned status `OPTIMAL` for Variants 2–4. In each case, a feasible schedule with $T(\pi) = B_4$ was exhibited; hence, the chain $B_4 \le T^* \le T(\pi)$ proves $T^* = B_4$ specifically for these scenarios. The equality is not assumed to be a universal property of M4.

6.1.1 Local Blocking and the Human Resource

In Variant 2, two agent slots are served by a single developer. A naive schedule that is feasible with respect to the DAG and the number of agents contains overlapping human-attention phases and is therefore infeasible under M4. The corrected schedule in Figure 1 sequences the task-specification and review phases. Task 7 waits 22.2 hours for review and retains the second slot, but this wait proceeds in parallel with the critical branch and does not increase the project makespan. This example shows why $Q(\pi)$ alone does not determine the makespan: the placement of the wait in the resource schedule matters.

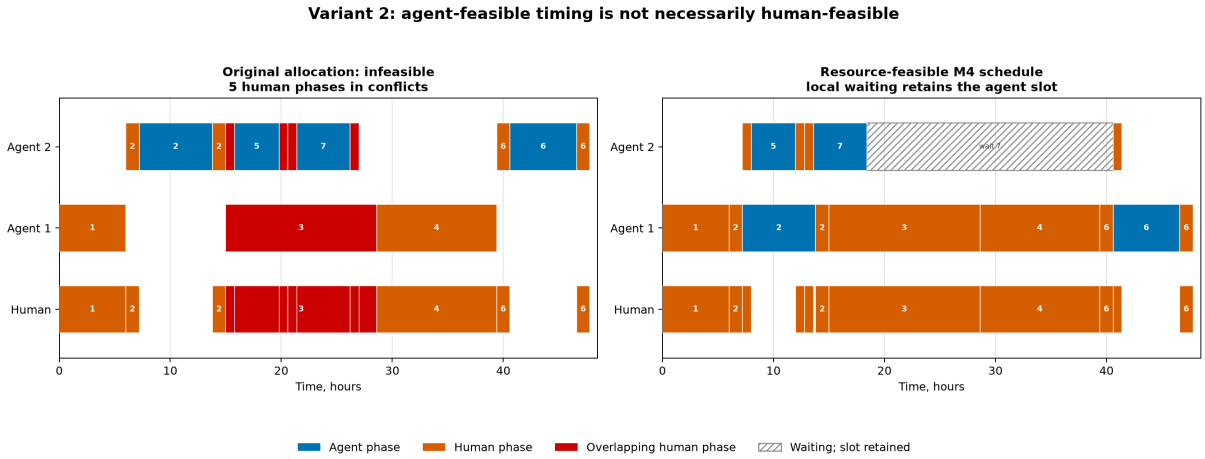

Figure 1: Variant 2 before and after explicitly accounting for the human resource. On the left, the original schedule contains overlapping human-attention phases and is infeasible under M4. On the right, the developer's service is serialized; the waiting task retains its agent slot but does not increase the overall makespan.

6.1.2 Critical-Path Decomposition

In Variant 3, the entire testing task waits for completion of the background worker. In Variant 4, unit and integration tests begin once their respective predecessors have completed, while only retry testing remains at the end of the critical path. With $A = 34.6$, $H = 19.0$, and $W_4 = 53.6$ unchanged, the critical path and exact makespan decrease from 43.2 to 36.7 hours. The effect is 6.5 hours, or approximately 15% of the Variant 3 makespan; speedup relative to the no-AI baseline increases from 1.76× to 2.07×.

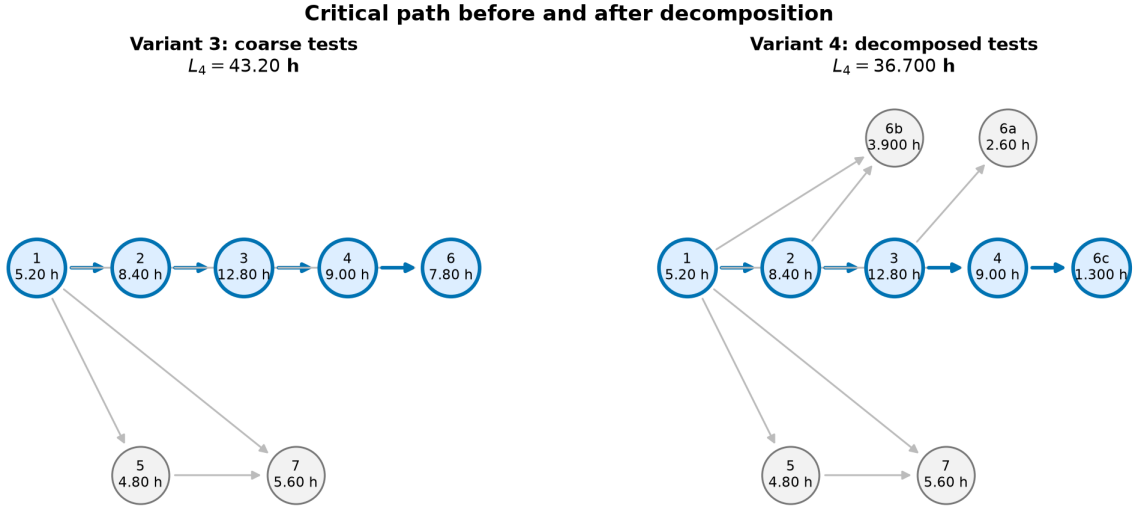

Figure 2: Task-DAG and critical path before and after refining the dependencies of the testing task. Total work is preserved, while unnecessary serialization is reduced.

Complete task parameters, phase intervals, the human-resource check, and elementary M0–M1 examples are provided in Appendix B. The series below are organized by substantive question rather than by numerical identifier: attainability of the bound and

the role of phase ordering are examined first, followed by parallelism and decomposition, and then by scale robustness, extension to synthetic DAGs, and controlled placement of human work. A numerical identifier denotes a frozen protocol, not the position of a series in the logic of the exposition.

6.2 Attainability of B_4

The equality obtained for the illustrative scenario in EXP-00 is not a general property of the model. In EXP-01, all 90 scenarios were solved to optimality, and in 30 of them a strict gap $T^* > B_4$ was found; the maximum relative gap on this grid was 18.333%. The selected four-task counterexample on the EXP-01 grid is the diamond at $P = 2$: $B_4 = 48.00$, whereas $T^* = 49.25$ hours. The additional 1.25 hours arise from a resource chain of repeated human-attention phases that is absent from the three aggregate terms defining B_4.

EXP-03 directly tests the sufficiency of the aggregates. In 14 of the 24 pairs, identical DAGs and identical values of A, H, W_4, L_4, B_4, C, and γ yielded different values of T^*. In the most pronounced selected pair, with a common $B_4 = 36.0$, the two phase profiles yielded 40.5 and 37.75 hours. Thus, B_4 correctly provides a structural lower bound, whereas the exact value is determined by the resource-augmented phase graph.

Figure 3 shows such a chain explicitly at one EXP-05 point with $P = 2$, $m = 4$, overhead rate $r_o = 0.05$, and $\lambda_h = 0$ (all overhead is assigned to agent work). In this chain, the ordinary phase sequence and DAG dependencies are interleaved with the service order of the single human resource and the task order on an agent slot. Its length of 36.45 hours therefore equals the certified T^* and exceeds the aggregate lower bound $B_4 = 31.80$ hours.

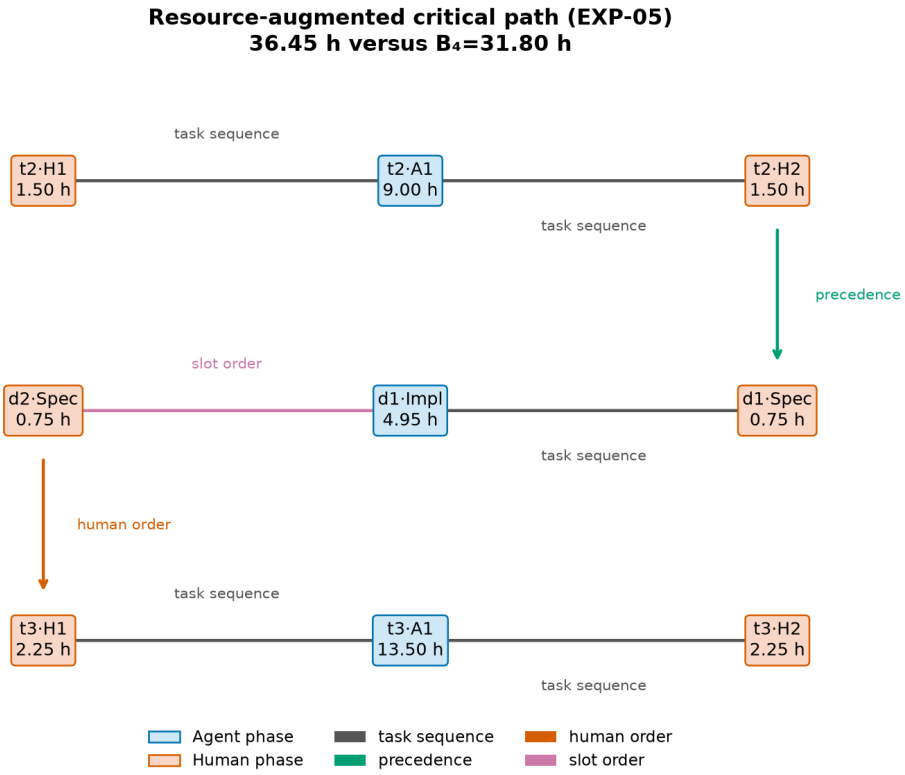

Figure 3: Resource-augmented critical path: task sequence — the phase order within a task, precedence — a DAG edge, human order — the order on the human resource, and slot order — the task order on one agent slot. The labels $d1, d2$ refer to the two decomposed subtasks.

6.3 Configured Parallelism

EXP-02 comprises 70 exact scenarios, including both constant-overhead and increasing-overhead regimes. On the curves with $C = \gamma = 1$ throughout, increasing P solely as available capacity did not worsen the optimum and quickly produced a plateau. This plateau need not attain B_4: for the four-task fork–join with $r_h = 0.50$, every tested $P \geq 2$ yields $T^* = 49.5$ hours, whereas $B_4 = L_4 = 48.0$ and $H = 30.0$ hours. The residual gap is imposed by a mixed resource chain and persists despite unused agent capacity. When instead an increase in P was accompanied by an increase in $C(P)$ or $\gamma(P)$, an interior minimum occurred at $P = 2$ in all four corresponding curves examined. This is consistent with Propositions 2 and 3: the deterioration is caused not by an additional unused stream but by a change in the process configuration.

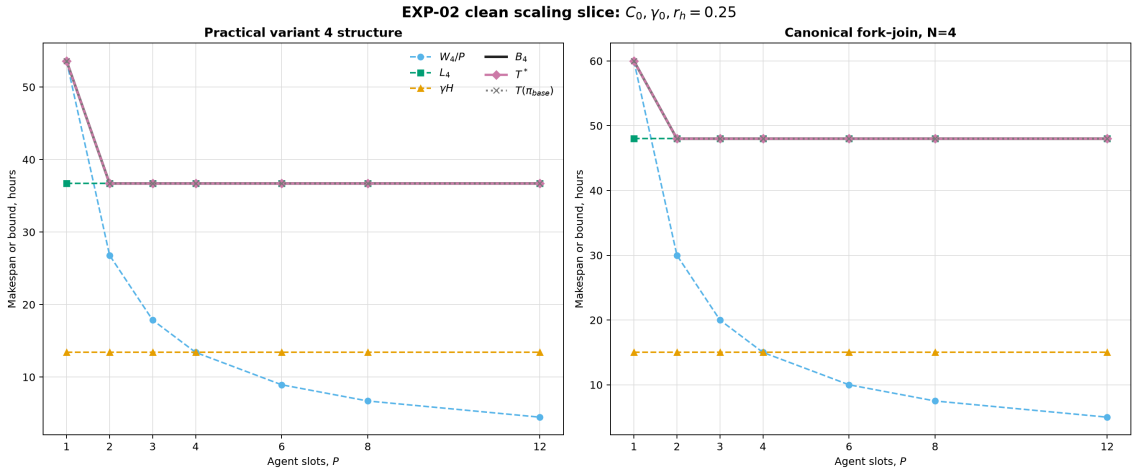

Figure 4: Exact makespan as the number of agent streams varies. Constant overhead produces saturation, whereas increasing $C(P)$ or $\gamma(P)$ produces an interior minimum on the grid examined.

6.4 Decomposition and Its Interaction with P

In EXP-04, the exact $T^*(m)$ profile on the finite grid was U-shaped in 12 of the 13 curves—in every case with positive decomposition overhead. The common zero-overhead slice reached a plateau once serialization was removed. With low overhead, the optimum was usually attained at $m = 3$; as overhead increased, it shifted toward $m = 2$. Decomposition is therefore not monotonically beneficial: it reduces structural serialization but adds task-specification, acceptance, and integration overhead.

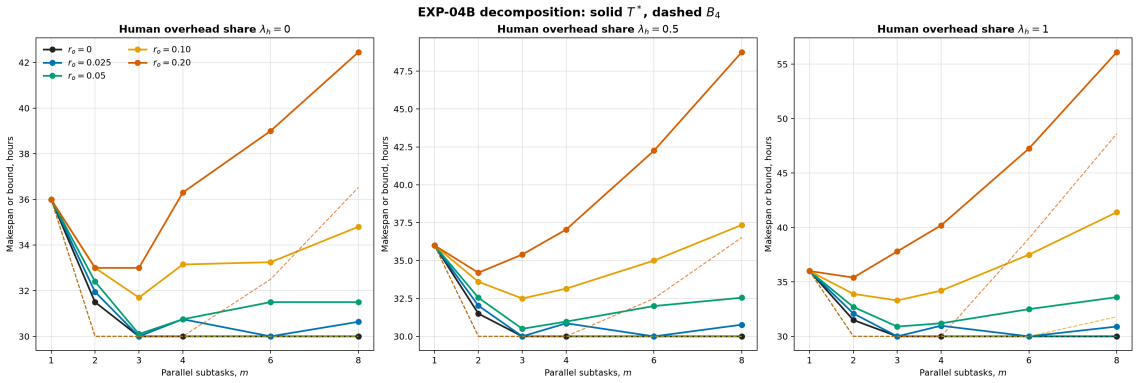

Figure 5: Exact decomposition curves. On the finite grid examined, positive overhead produces a finite optimal granularity; zero overhead produces a plateau.

EXP-05 examines the interaction effect. Of the 100 interior interaction contrasts with positive overhead, 96 are positive, four are zero, and none are negative. However, the absolute gain from decomposition at $P > 1$ is positive at only 74 points and negative at 26; at 22 points, a positive interaction contrast coincides with a negative absolute gain. Complementarity therefore means that the effect is more favorable than at $P = 1$, not that decomposition is unconditionally beneficial. Increasing the human component of overhead attenuates the gain. This is a result on the selected finite grid, not a universal theorem.

6.5 Scale and Parameter Robustness: EXP-06

As an implementation check, EXP-06 reproduced scale invariance at all 20 of 20 exact-solver points. The advantage of illustrative Variant 4 over Variant 3 was preserved in all 5000 random pairs and all 382 prespecified adverse pairs. This result establishes robustness only within the frozen error families, not under arbitrary perturbations of all parameters.

6.6 Transfer across Synthetic DAGs: EXP-07

Across 1440 scenarios constructed from 480 synthetic DAGs with three placements of human work, EXP-07, in its pilot mode, supported two prespecified median directions for `bottom_level_fcfs_v1`. In C2, the median shift in $P_{\mathrm{opt}} := \min \arg\min_P T_\pi(P)$ after introducing agent and human costs was zero; shifts in the opposite, positive direction occurred in 0.42% and 0.63% of DAGs, respectively, below the prespecified 5% threshold. In C3, the median $I_\pi(4, 3)$ was 6.4, with a 95% bootstrap interval of $[6.0, 6.8]$, while the median attenuation of the gain due to overhead was 2.0, with an interval of $[2.0, 2.0]$. Negative individual contrasts nevertheless accounted for 6.53% and 3.47%, respectively. Thus, the pilot supported aggregate directions, not universal signs for every DAG or claims about a certified T^*. Because the pilot-extension rule was only partially implemented, these decisions pertain to the resulting 20-seed pilot and require reassessment under the fully implemented gate.

The hypotheses concerning the "synchronous" profile and simple concentration of human work were not supported: the former did not, in fact, model synchronization in time, while the latter conflated a change in total H with its placement. The retrospective interpretation of the former is therefore limited to a comparison of equal and permuted asynchronous phase profiles, and the conclusion regarding the placement of H is retested in EXP-08. In the EXP-07 exact audit, only 17 of 36 runs returned `OPTIMAL`; four complete C1 pairs yielded a small estimate of the opposite sign. This audit did not cover C2–C3 and does not replace the primary fixed-policy estimand.

6.7 Placement of Fixed Human Work: EXP-08

The corrected paired design of EXP-08 preserved the DAG, A, H, and total work. With $C = \gamma = 1$, moving human work between tasks changed neither H, W_4, L_4, B_4 nor the active branches in any of the 2880 checks. When $\gamma > C$, moving an amount ΔH onto the prespecified path changes the length of that path by exactly $(\gamma - C)\Delta H$; this identity was confirmed in all 480 pairs. The change in the fixed-policy makespan, however, depended on the phases and on P. For the primary `io` profile, the normalized paired median "high minus low" at $P = 4$ was -0.0077, with a 95% bootstrap interval of $[-0.0090, -0.0065]$, and was classified as practically equivalent under the prespecified margin of 0.01. At $P = 8$, the estimate was -0.0117, with an interval of $[-0.0120, -0.0101]$, and the decision was negative.

All 16 exact-audit scenarios had status `OPTIMAL`, but this does not make the statistical conclusion determinate: the eight certified pairs yielded the same median, 0.0010, for $P = 4$ and $P = 8$, with an interval of $[-0.0150, 0.0080]$, so both decisions remained inconclusive. Thus, two distinct characteristics matter: total human work determines the resource ceiling; when $\gamma \neq C$, its placement changes the path weights and L_4, whereas when $C = \gamma$, phase order can still change the queue and the fixed-policy makespan while the aggregates remain unchanged.

7 Discussion

The results support interpreting the model not as a formula in which "the number of agents divides the completion time," but as a planning model for a capacity-constrained human–agent cell.

7.1 Bound, Optimum, and a Concrete Schedule

B_4 identifies a time below which completion is impossible because of the aggregate workload of the streams, the critical path induced by the original precedence constraints, or the single human resource. T^* answers a different question: the best makespan after accounting for all resource orderings. Finally, $T(\pi)$ characterizes a selected policy or an actual schedule. The equality $T^* = B_4$ requires a certificate; otherwise, an equal lower bound does not imply an equal makespan.

7.2 Resource Chains as a Gap Mechanism

Repeated requests from multiple tasks to a single developer link phases that do not lie on the same path in the original task-DAG. Retention of an agent slot while waiting propagates this competition further through the schedule. The resource-augmented phase graph makes the mechanism visible: its maximum-weight path contains not only technological arcs but also resource-order arcs. This explains why A, H, W_4, L_4, B_4 alone do not identify T^*.

7.3 Parallelism as a Configuration Choice

Available capacity must be distinguished from configured parallelism. With durations held constant, an additional stream may be left idle, so the optimum cannot worsen. An organizational transition to a larger P, however, may increase the integration cost $C(P)$ and the switching cost $\gamma(P)$. The comparison then concerns different processes, and a finite optimum is possible. In practical terms, P cannot be selected independently of the mode assignment and the overhead functions. The formal configuration $\sigma = (P, \rho)$ fixes the number of streams and the task modes; decomposition, the DAG, the phase profiles, and the acceptance criterion constitute the remaining conditions of a fully specified scenario. If at least one of them changes together with P, the observed difference pertains to a comparison of the scenarios as a whole, not to the isolated effect of an additional stream.

7.4 Decomposition and Parallelism

Decomposition is beneficial while it shortens long sequential segments and creates ready work for available streams. Beyond that point, additional boundaries increase task-specification and integration overhead and the number of human interventions. Under the decomposition operator studied here, the optimal granularity therefore depends on P: without exploitable parallelism, small independent tasks provide little benefit, whereas at large P, granularity itself may constrain capacity utilization. The experiments support this complementarity on the selected grids but do not establish it for an arbitrary DAG or splitting rule; its direction and magnitude require calibration on real processes.

7.5 Volume and Placement of Human Work

Total H imposes an unconditional sequential workload and a ceiling on speedup. Its placement is a separate characteristic. When agent and human-attention phases are scaled equally, moving H among tasks need not change the aggregate lower bound; when $\gamma > C$, human work on the critical path is more costly than agent work and lengthens that path. Even when L_4 remains unchanged, phase order may change the queue and T^*. Thus, process design requires more than reducing the hours spent on review: it also requires deciding when and in which tasks those hours are needed.

7.6 Relation to Prior Work

The human-resource ceiling is a structural analogue of the logic of Amdahl's law: the sequential component here is not a segment of a computational program but the measured involvement of the developer [1]. An interior minimum over configured P is consistent with the Universal Scalability Law in the limited sense that increasing contention and coordination can turn saturation into negative returns [19]. The numerical form of $C(P)$ for agents nevertheless remains a hypothesis of the model, not a consequence of these classical results.

The gap between B_4 and T^* connects the formulation to scheduling theory and the RCPSP: the original task-dependency graph and aggregate load are insufficient when a single service resource creates additional chains induced by resource constraints [17, 8]. Resource-order arcs and a shared server are standard mechanisms in scheduling theory [29, 20]. Loading and unloading models already include service before and after processing and may retain a machine until unloading [15]; the reentrant model routes work back to the same primary machine after a remote server but releases that machine during service [10]. Accordingly, no novelty is claimed for DAGs, a shared resource, repeated service, or a resource-augmented graph as such.

A related concept of a finite number of robots that can be controlled effectively appears in models of the number of robots per operator, in which service intervals must fit within the autonomous-work windows of the other systems [26, 13]. The experiment by Cummings and Mitchell further shows that queueing, delays in regaining situational awareness, and retasking reduce the operator's predicted throughput, but its UAV-simulation results cannot be transferred numerically to software development [14]. Among coding-agent systems, MAGIS implements an automated "development—quality control—revision" cycle, while CAID uses concurrent worktrees with a dependency plan, merging, and testing [32, 16]. Their quality and success metrics do not demonstrate a reduction in makespan; in CAID's main comparisons, elapsed time and cost increased together with the quality metric.

The present model combines known constructs in a mapping to the software process: a task- and mode-dependent coefficient k, separate a_i and h_i, repeated human-attention phases, local retention of an agent slot, integration and attention-switching overhead, and the makespan of an accepted result. The main reviewed corpus, assembled under a search protocol with a cutoff date of 2026-08-09, contained no joint empirical calibration of these elements on software tasks; this is not a universal claim about the entire literature. The present article likewise does not close this empirical gap: the computational results clarify the mechanism of the process constraint in synthetic scenarios but do not replace measurements of specific tools and teams.

7.7 Using the Model

First, tasks and operating modes should be estimated on a common scale, and the task-DAG and phase profiles should be constructed; B_4 can then be computed as a rapid bottleneck diagnostic. For several competing configurations, an exact or heuristic schedule is constructed, and whether a certified T^* or only $T(\pi)$ was obtained is recorded explicitly. Finally, sensitivity to k, h, C, and γ is evaluated. The model is more useful for comparing variants of work organization than for promising a precise elapsed time without local calibration. The active branch of B_4 characterizes the constraint that determines the lower bound, but does not by itself prove that T^* is governed by the same branch: such a conclusion requires either attainability of the bound or analysis of the resource-augmented schedule. Similarly, an autonomous agent phase means only that the developer is not involved during that interval; a task containing such a phase remains delegated if task specification or acceptance requires human involvement.

8 Limitations and Threats to Validity

The formal model assumes that the directed acyclic graph $G = (V, E)$ is known before planning begins and remains unchanged during execution. This assumption is reasonable for a prespecified workload but limits the model's use for research tasks, architectural explorations, and other R&D processes in which new tasks and dependencies are discovered as results become available. In such processes, the estimates $L_4(P)$ and B_4 apply only to the current snapshot G_t; a critical path computed once cannot be interpreted as a fixed property of the entire project.

All tasks are assumed to be available when planning begins, their phases are assumed to be nonpreemptive, and the only constrained resources are the P agent streams and one developer. Release dates, separate CI/API capacities, and fully manual tasks require a different formulation; M4 describes only a prespecified AI-assisted task set.

The model also does not describe the mechanism by which new vertices are revealed, the cost of replanning, or the choice of how to decompose the work further. Splitting a task can expose only substantively feasible parallelism: it does not eliminate genuine precedence relations and usually increases the cost of task specification, review, and integration. The reported results can therefore be used to evaluate a given graph or its current snapshot, but do not by themselves determine an optimal policy for constructing a dynamic graph.

An absolute prediction of completion time requires calibrating X, k_i, h_i, $C(P)$, and $\gamma(P)$ for the specific tools and operating mode; when these change, the calibration may quickly become obsolete. The requirements are weaker for a comparative choice between two variants: workloads and durations may be expressed in notional units of a single baseline task if both configurations are evaluated on a common scale. Proposition 5 guarantees ranking robustness only to a single common multiplier applied equally to all agent and human-attention phases of the variants being compared. Unequal errors across tasks or modes, distortion of the decomposition between a_i and h_i or of the within-task phase profile, and different values of $C(P)$ and $\gamma(P)$ can change the resource paths and the ranking of the variants.

Construct validity. The coefficients k, a, and h aggregate the quality of task specification, the capabilities of the tool, and the review and correction of the result. The same numerical profile may conceal different mechanisms, and makespan does not measure code quality, understanding of the system, computational cost, or downstream defects.

The terms "synchronous" and "concentration" in the early experimental hypotheses proved insufficiently operationalized; the associated conclusions are not treated as supported.

Internal validity. The exact series use deterministic durations, an integer grid, and a single solver. The risk of implementation error is reduced through independent checks of the constraints, optimality statuses, frozen inputs, and reproducible manifests; however, a computational experiment verifies properties of the implementation of the formal model, not the truth of its assumptions about the real process.

Conclusion validity. Finite exact grids do not prove universal patterns beyond the selected ranges. The counts 30/90, 12/13, and 96/100 describe the matrices examined, not frequencies in the population of projects. In the large EXP-07 series and part of EXP-08, a fixed policy is evaluated; its effects cannot be transferred to T^* without a separate exact proof.

External validity. Most of the DAGs, durations, and functions $C(P), \gamma(P)$ are synthetic and have not been calibrated on longitudinal data from teams. The formulation considers one developer, homogeneous agent streams, and local blocking with slot retention. Other schedulers may reuse capacity held by a waiting task, multiple people may handle requests in parallel, and real tasks may reveal dependencies and rework dynamically.

Parameter uncertainty. EXP-06 tests only prespecified random and adverse error families. The observed robustness of the ranking does not guarantee robustness to an arbitrary joint error in the estimates. Before a practical decision is made, local calibration, interval analysis, and replanning are required as actual data become available.

9 Future Work

For a fixed graph, the proposed model solves the forward problem: given the work structure and the parameters of the human–agent cell, it estimates the achievable speedup and constructs a schedule. The inverse problem is a natural extension. If only part of the graph G_t is known at time t, further decomposition can be selected in light of the critical path, the required parallelism, and the limited human-attention resource. The model would thereby serve not only to evaluate a completed plan but also as a criterion for constructing the as-yet unrevealed part of the project.

Such a policy can follow a rolling-horizon approach. After a result reveals new work or dependencies, G_t, the set of ready tasks, and the phase estimates are updated, and the unfinished part is rescheduled. Research tasks can then be represented as exploratory vertices whose results reveal a set of feasible continuations $\mathcal{D}_t$. For each $d \in \mathcal{D}_t$, the following surrogate estimate can be used:

$$\Phi_t(d) = \max\left\{\frac{W_{4,t}(d) + \widehat{Q}_t(d)}{P}, L_{4,t}(d), \gamma(P)H_t(d)\right\}, \qquad d_t^* \in \arg\min_{d\in\mathcal{D}_t} \Phi_t(d),$$

where $\widehat{Q}_t(d)$ estimates future waiting that is not captured by the structural bounds already known. A decomposition variant must preserve the original workload and acceptance criterion, while its additional task specification, verification, and integration overhead must be included in $W_{4,t}(d)$ and $H_t(d)$. If an empirically established manageable limit $P_{\max}^{\mathrm{attn}}$ on the number of streams that can be supervised concurrently is available, exceeding that limit should be prohibited, and the proximity of useful parallelism to $P_{\max}^{\mathrm{attn}}$ should be used as a secondary criterion after minimizing Φ_t. Constructing feasible graph transformations,

estimating $\widehat{Q}_t(d)$, and deriving guarantees for such an adaptive policy constitute a separate joint decomposition-and-scheduling problem; in this article, the criterion is presented only as a sketch of a future theory.

A separate direction is to extend the model to other multi-agent and swarm architectures. The closest extension of the current formulation is a hierarchical variant in which an orchestrator agent replaces the developer in the inner loop: it decomposes the objective, assigns tasks to worker agents, services their requests, and accepts their results. The structural analogy is direct: the orchestrator is likewise a shared resource of finite capacity, and requests from worker agents form a queue and may locally block their streams. Human attention may then remain only at the boundaries of the overall process—when setting the overall objective and performing final acceptance.

A different formulation arises under algorithmic orchestration with auditing and self-checking. The macro-level task graph may remain deterministic, but each vertex expands into a local loop of "execution—audit—conditional rework—repeated audit." If such loops are self-contained and do not use a shared orchestrator, they do not block one another, and the internal human resource of capacity 1 disappears. In the terms of the present model, audit and rework become agent phases: h_i decreases for the internal vertices, while a_i increases; the sign of the change in $k_i = a_i + h_i$ depends on the balance between the human effort saved and the additional agent work. With a fixed number of passes, such a loop can be unfolded into an ordinary phase DAG; conditional termination and an unknown number of rework passes require a stochastic model of rework.

A more general extension should describe an architecture by a graph of resources and interactions rather than by the number of agents. Hierarchical orchestrators, decentralized swarms, mutual auditing, specialized planners and verifiers, shared memory, and heterogeneous resources create different bottlenecks and different forms of coordination overhead. Comparing such systems requires replacing the star-shaped "one developer—P agents" scheme with a heterogeneous multilevel scheduling model and evaluating the configuration as a whole: its topology, routing rules, audit resources, result quality, and the cost of additional self-checking cycles. Multiple human–agent cells require additional cross-cell resources and therefore cannot be reduced to an increase in P at M4.

Another direction is relative and interval calibration for organizations that lack a history of precise measurements. Under cold-start conditions, the unit X can be interpreted not as an hour but as a notional baseline task, after which Z_i, k_i, and h_i can be elicited from experts relative to that unit. If two configurations A and B are described on the same scale and all of their agent and human-attention phases contain the same unknown common factor $\kappa > 0$, Proposition 5 implies

$$\frac{T_A^*(\kappa)}{T_B^*(\kappa)} = \frac{T_A^*(1)}{T_B^*(1)}.$$

Thus, even coarse quantitative relative estimates can be useful for selecting a faster work-organization scheme without promising an exact completion time in hours. Further research should replace point estimates with intervals for k_i and h_i, identify conditions under which one variant robustly dominates another, and characterize cases in which uncertainty in the relative profile can alter the ranking.

Another direction is a stochastic and robust formulation with rework, failures, learning, and interval estimates of k, h, C, and γ. It should provide not only the expected completion time but also quantiles, the probability of missing a deadline, and conditions under which one configuration robustly dominates another.

Finally, empirical calibration on traces of real software work is necessary. It should measure autonomous intervals, human-attention intervals, waiting, rework, and result quality separately in order to test the forms of $C(P)$ and $\gamma(P)$ and distinguish effective parallelism from the nominal number of running agents.

10 Conclusion

This article examined the transition from the duration of an isolated AI-assisted task to the makespan of a fixed set of interdependent software tasks. For the human–agent cell studied here, the answer to the research question is conditional: a configuration reduces makespan relative to sequential no-AI work only when a feasible schedule completes the same workload to the same acceptance criterion faster than the baseline. Achievable speedup depends jointly on task- and mode-specific durations, dependencies and phase profiles, the number of agent streams and the rules governing their use, integration overhead, and the sequential resource of human attention. It is therefore a property of a fully specified process, not an isolated characteristic of the AI tool or the nominal agent count.

The M0–M4 hierarchy makes this conclusion progressively less idealized. It moves from divisible aggregate work to indivisible tasks, precedence constraints, configuration-dependent overhead, and an exact phase schedule in which autonomous-agent work alternates with requests for one developer's attention. At M0, the exact aggregate speedup condition is $P > \bar{k}_w$; under the stated identical-stream and zero-overhead assumptions, the M1–M2 list-scheduling bounds give sufficient speedup guarantees accounting for task indivisibility and precedence. At M4, the structural lower bound B_4 combines agent-stream load, the original critical path, and total human work. The bound is a necessary diagnostic, but it need not be attainable: repeated human service and local retention of a waiting task's agent slot can create mixed resource chains that are absent from its three branches. The maximum-weight paths in feasible resource-augmented phase graphs capture those chains, and minimizing their length gives the optimal makespan exactly. Sequential human work also imposes a speedup ceiling; after it becomes active, additional agents cannot substitute for shorter specification, review, correction, or acceptance phases.

The reproducible EXP-00–EXP-08 package computationally checks these distinctions within the formal model rather than calibrating a particular tool or team. Most importantly, in the frozen 90-scenario EXP-01 grid, a strict certified gap $T^* > B_4$ occurred in 30 cases, so the lower bound cannot be used as a universal estimate of achievable makespan. The sensitivity experiments also separate available capacity from configured parallelism: an unused additional stream cannot worsen the optimum when phase durations and the overhead multipliers C and γ are held fixed, whereas a change in configuration may increase integration or attention overhead and create an interior optimum over P. Likewise, finer decomposition can expose parallelism while adding specification and integration boundaries, producing nonmonotonic makespan profiles. Complementarity between parallelism and granularity therefore does not imply that either more streams or more tasks is unconditionally beneficial. All reported frequencies describe prespecified synthetic matrices or generated samples, not the prevalence of these effects in real projects.

In practice, the model should be used to compare complete work-organization scenarios rather than to select the largest available agent count. Tasks and operating modes must be placed on a common scale, the workload and acceptance criterion must be fixed, and the DAG, phase profiles, and overhead assumptions must be stated. The active branch of B_4 identifies the constraint governing the structural lower bound, but only a resource-feasible

schedule establishes an attainable makespan. Sensitivity analysis is therefore part of the comparison, not an optional addition. When candidate interventions are encoded as complete alternative scenarios, sensitivity comparisons can indicate whether makespan is more responsive to task execution time, critical-path length, human-attention demand, coordination overhead, or decomposition.

These conclusions remain limited to the assumed architecture: a fixed, known DAG; one developer; homogeneous agent streams; nonpreemptive phases; and local slot retention. Quality and computational cost are represented only when they alter time to an accepted result, and the model does not replace a causal study of AI effectiveness. Empirical use requires local measurement of agent and human-attention intervals, waiting, rework, and the dependence of overhead on configuration. Extensions to dynamic graph revelation, adaptive decomposition, stochastic durations, and other orchestration architectures remain future work. Until such validation is available, the contribution is structural and operationally interpretable: within the model, useful parallelism must be assessed through the coordinated design of tasks, dependencies, phases, resources, and acceptance rules for the human–agent process as a whole.

11 Code and Data Availability

The research artifact is distributed with the current working version of the manuscript in the `experiments` directory. It includes the source code for the simulator and exact solver, tests, frozen EXP-00–EXP-08 scenario matrices, checksum manifests, raw CSV/JSON results, reports, and figure-generation code. The `pyproject.toml` and `uv.lock` files pin the Python dependencies of the computational package. The literature-review search protocol and evidence matrix are included in the artifact as `../tmp_docs/simple_model_full_literature_search_protocol.md` and `../tmp_docs/simple_model_full_evidence_matrix.md`.

In addition to the reproducibility package, we developed an interactive companion artifact, the M4 Human–Agent Workbench, available at `https://m4.articles.rexarrior.online/`. Its source code and deployment configuration are located in the `calculator` directory. The interface allows a user to import an AI-generated project decomposition in JSON format, edit the DAG and M4 parameters, and then compute the lower bound, a resource-feasible schedule, and scenario-based speedup locally in the browser. The Russian and English versions, as well as automatic, light, and dark themes, are available within the same deployment. The configuration and result are not sent to the server unless the user explicitly opts in to sharing the calculation.

The public calculator is a demonstration and practical interface to the model, but it does not replace the reproducibility package in `experiments`. The code for the computational package and calculator is distributed under the MIT License; the manuscript text, documentation, author-created figures, scenarios, and results are distributed under the CC BY 4.0 License. The precise scope and exclusions of each license are specified in `LICENSE.md`. The public repository is available at `https://github.com/Rexarrior/simple_model_preprint`. A versioned release and DOI will be added after the version accompanying the preprint has been archived. The calculator's live URL is not an archival persistent identifier.

Verification and reproduction commands are provided in Appendix C. If the article is distributed separately from the repository, the `experiments` directory and the two review files listed above must be distributed as a single supplementary artifact: without the input

matrices, manifests, lockfile, and search protocol, the PDF manuscript alone is insufficient for independent verification of the computations and literature search.

Declarations

The author is employed by Yandex. This research was conducted independently in the author's personal time and received no dedicated funding from Yandex. Some computational work used Yandex equipment and computing infrastructure. The views expressed are solely those of the author and do not necessarily represent those of Yandex. The author declares no other competing interests.

References

[1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference*, pages 483–485, 1967. DOI: https://doi.org/10.1145/1465482.1465560.

[2] Nishant Balepur, Connor Baumler, Valerie Chen, Eunsol Choi, Rachel Rudinger, and Jordan Lee Boyd-Graber. (Im)Paired Programming: Coding agents improve productivity but harm understanding, 2026. Version 1, submitted 29 July 2026; DOI: https://doi.org/10.48550/arXiv.2607.26375.

[3] Shraddha Barke, Michael B. James, and Nadia Polikarpova. Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):78:1–78:27, 2023. DOI: https://doi.org/10.1145/3586030.

[4] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv preprint arXiv:2507.09089, 2025. DOI: https://doi.org/10.48550/arXiv.2507.09089.

[5] Joel Becker, Nate Rush, Tom Cunningham, David Rein, and Khalid Mahamud. We are changing our developer productivity experiment design. METR, February 2026. https://metr.org/blog/2026-02-24-uplift-update/; accessed 2026-08-03.

[6] Barry Boehm, Chris Abts, and Sunita Chulani. Software development cost estimation approaches—a survey. *Annals of Software Engineering*, 10(1–4):177–205, 2000. DOI: https://doi.org/10.1023/A:1018991717352.

[7] Frederick P. Brooks, Jr. *The Mythical Man-Month: Essays on Software Engineering.* Addison-Wesley, Boston, anniversary edition, 1995.

[8] Peter Brucker, Andreas Drexl, Rolf Möhring, Klaus Neumann, and Erwin Pesch. Resource-constrained project scheduling: Notation, classification, models, and methods. *European Journal of Operational Research*, 112(1):3–41, 1999. DOI: https://doi.org/10.1016/S0377-2217(98)00204-5.

[9] Erik Brynjolfsson, Danielle Li, and Lindsey R. Raymond. Generative AI at work. *The Quarterly Journal of Economics*, 140(2):889–942, 2025. DOI: https://doi.org/10.1093/qje/qjae044.

[10] Konstantin Chakhlevitch and Celia A. Glass. Scheduling reentrant jobs on parallel machines with a remote server. *Computers & Operations Research*, 36(9):2580–2589, 2009. DOI: https://doi.org/10.1016/j.cor.2008.11.007.

[11] Valerie Chen, Ameet Talwalkar, Robert Brennan, and Graham Neubig. Code with me or for me? how increasing AI automation transforms developer workflows. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems*, pages 122:1–122:19. Association for Computing Machinery, 2026. CHI 2026; author manuscript version 2 submitted 13 September 2025; DOI: https://doi.org/10.1145/3772318.3790850.

[12] Jacob W. Crandall, Mary L. Cummings, Mauro Della Penna, and Paul M. A. de Jong. Computing the effects of operator attention allocation in human control of multiple robots. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 41(3):385–397, 2011. DOI: https://doi.org/10.1109/TSMCA.2010.2084082.

[13] Jacob W. Crandall, Michael A. Goodrich, Dan R. Olsen, Jr., and Curtis W. Nielsen. Validating human-robot interaction schemes in multitasking environments. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 35(4):438–449, 2005. DOI: https://doi.org/10.1109/TSMCA.2005.850587.

[14] Mary L. Cummings and Paul J. Mitchell. Predicting controller capacity in supervisory control of multiple UAVs. *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans*, 38(2):451–460, 2008. DOI: https://doi.org/10.1109/TSMCA.2007.914757.

[15] Abdelhak Elidrissi, Rachid Benmansour, Keramat Hasani, and Frank Werner. Scheduling on parallel machines with a common server in charge of loading and unloading operations, 2023. Version 1, submitted 29 June 2023; DOI: https://doi.org/10.48550/arXiv.2306.16669.

[16] Jiayi Geng and Graham Neubig. Effective strategies for asynchronous software engineering agents, 2026. Version 2, revised 8 July 2026; accepted at COLM 2026; DOI: https://doi.org/10.48550/arXiv.2603.21489.

[17] R. L. Graham. Bounds for certain multiprocessing anomalies. *The Bell System Technical Journal*, 45(9):1563–1581, 1966. DOI: https://doi.org/10.1002/j.1538-7305.1966.tb01709.x.

[18] R. L. Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969. DOI: https://doi.org/10.1137/0117039.

[19] Neil J. Gunther. A general theory of computational scalability based on rational functions, 2008. DOI: https://doi.org/10.48550/arXiv.0808.1431.

[20] Nicholas G. Hall, Chris N. Potts, and Chelliah Sriskandarajah. Parallel machine scheduling with a common server. *Discrete Applied Mathematics*, 102(3):223–243, 2000. DOI: https://doi.org/10.1016/S0166-218X(99)00206-1.

[21] Magne Jørgensen. Forecasting of software development work effort: Evidence on expert judgement and formal models. *International Journal of Forecasting*, 23(3):449–462, 2007. DOI: https://doi.org/10.1016/j.ijforecast.2007.05.008.

[22] Magne Jørgensen and Martin Shepperd. A systematic review of software development cost estimation studies. *IEEE Transactions on Software Engineering*, 33(1):33–53, 2007. DOI: https://doi.org/10.1109/TSE.2007.256943.

[23] Yubin Kim, Ken Gu, Chanwoo Park, Chunjong Park, Samuel Schmidgall, A. Ali Heydari, Yao Yan, Zhihan Zhang, Yuchen Zhuang, Yun Liu, Mark Malhotra, Paul Pu Liang, Hae Won Park, Yuzhe Yang, Xuhai Xu, Yilun Du, Shwetak Patel, Tim Althoff, Daniel McDuff, and Xin Liu. Towards a science of scaling agent systems, 2025. Version 3, revised 8 April 2026; DOI: https://doi.org/10.48550/arXiv.2512.08296.

[24] S. A. Kravchenko and F. Werner. Parallel machine scheduling problems with a single server. *Mathematical and Computer Modelling*, 26(12):1–11, 1997. DOI: https://doi.org/10.1016/S0895-7177(97)00236-7.

[25] Banruo Liu, Haoran Qiu, Íñigo Goiri, Rodrigo Fonseca, Ricardo Bianchini, and Esha Choukse. Agentic coding in the wild: Characterizing GitHub Copilot traces at production scale, 2026. Version 1, submitted 30 July 2026; DOI: https://doi.org/10.48550/arXiv.2608.00101.

[26] Dan R. Olsen, Jr. and Stephen Bart Wood. Fan-out: Measuring human control of multiple robots. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 231–238, 2004. DOI: https://doi.org/10.1145/985692.985722.

[27] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv preprint arXiv:2302.06590, 2023. DOI: https://doi.org/10.48550/arXiv.2302.06590.

[28] Laurent Perron, Frédéric Didier, and Steven Gay. The CP-SAT-LP solver. In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 3:1–3:2. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2023. DOI: https://doi.org/10.4230/LIPIcs.CP.2023.3.

[29] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, 3 edition, 2008. DOI: https://doi.org/10.1007/978-0-387-78935-4.

[30] Dario D. Salvucci and Niels A. Taatgen. Threaded cognition: An integrated theory of concurrent multitasking. *Psychological Review*, 115(1):101–130, 2008. DOI: https://doi.org/10.1037/0033-295X.115.1.101.

[31] Ningzhi Tang, Chaoran Chen, Gelei Xu, Yiyu Shi, Yu Huang, Collin McMillan, Tao Dong, and Toby Jia-Jun Li. How coding agents fail their users: A large-scale analysis of developer–agent misalignment in 20,574 real-world sessions, 2026. Version 1, submitted 28 May 2026; DOI: https://doi.org/10.48550/arXiv.2605.29442.

[32] Wei Tao, Yucheng Zhou, Yanlin Wang, Wenqiang Zhang, Hongyu Zhang, and Yu Cheng. MAGIS: LLM-based multi-agent framework for GitHub issue resolution. In *Advances in Neural Information Processing Systems*, volume 37, 2024. NeurIPS 2024; DOI: `https://doi.org/10.52202/079017-1647`.

[33] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *CHI Conference on Human Factors in Computing Systems Extended Abstracts*, pages 1–7, 2022. DOI: `https://doi.org/10.1145/3491101.3519665`.

A Full Glossary of Terms and Notation

This appendix consolidates the terms and notation used throughout the article. The explanations provide a concise reference; the formal definitions and assumptions are introduced in Section 4.

A.1 Core Terms

Work Unit
An abstract amount of work whose no-AI effort is equal to X. In the illustrative scenario, work units are linked to estimates in story points, although the model does not require a particular task-size scale.

Task
A part of the project that is indivisible at the selected level of decomposition. A task has size Z_i, operating mode r, and normalized duration $k_i^{(r)}$. When the level of decomposition changes, one task may be replaced by several subtasks.

Effort
The amount of work expressed in terms of execution time without AI. In the no-AI baseline with one developer, effort coincides with elapsed time; under parallel execution, total effort and the project's elapsed time differ.

Elapsed and Active Time
Elapsed time is measured from the start to the completion of a task and includes waiting. Active time counts only intervals during which the worker is directly engaged in execution. These metrics, as well as task-flow throughput, are not interchangeable.

Accepted Result
A result that satisfies the specified acceptance criterion. The time spent on review, correction, and repeated review is included in the task duration; merely completing agent generation does not constitute completion of the work.

No-AI Baseline
Sequential execution of the specified workload by one developer without AI. Its duration is denoted by T_h and is used as the baseline for calculating speedup. The term used in the English-language literature is *baseline*.

Normalized AI-Assisted Duration
The ratio of the AI-assisted task duration to the baseline estimate $Z_i X$. It is denoted

by $k_i^{(r)}$ and depends on the task, tool, and mode. This is a descriptive quantity, not a universal property of an AI model or a causal estimate in the absence of a separate identification design.

AI Coding Agent
A software system based on an AI model that can receive a task, work with code and tools, and return a result for automated or human acceptance. The short form used in the article is *agent*.

Human–Agent Cell
The organizational unit considered in the article: one developer assigns tasks to one or more agents and accepts their results. Modeling multiple interacting cells is left for future work.

Operating Mode
The manner in which a human and an agent interact while performing a task. The interactive, delegated, and fully autonomous modes are distinguished. The mode characterizes not the product itself but the sequence of task specification, execution, review, and correction.

Interactive Mode
The developer and agent perform a single task jointly, sequentially exchanging requests and results. In the limiting case, the human component occupies the entire task duration: $h_i = k_i$.

Delegated Mode
The developer specifies a task, the agent performs it autonomously, and the developer then reviews the result and either accepts it or returns it for revision.

Fully Autonomous Mode
A limiting mode in which a prespecified task has no mandatory human-attention phase. The model does not represent discovery of new tasks or automatic acceptance of results, and this mode is not examined in detail. It must not be conflated with an autonomous agent phase of a delegated task: such a phase frees the developer during execution, but task specification and acceptance of the result still require human-attention phases.

Process Configuration
A complete description of how the work is organized: configured parallelism P, the assignment of modes to tasks, the tool used, and the task-specification and acceptance protocol. When the tool and protocol are fixed, the abbreviated notation $\sigma = (P, r)$ is used for a common mode, or $\sigma = (P, \rho)$ for different modes. Configurations should be compared as a whole because changing any of these elements may change the coefficients $k_i^{(r)}$, phase profiles, and overhead.

Decomposition
The division of a project into tasks and subtasks with explicit boundaries and dependencies. Decomposition determines the parallelism that can be exploited and the composition of the critical path; excessive fragmentation may increase task-specification and acceptance overhead.

Parallelism
The number of AI-assisted execution streams configured to be available concurrently. It is denoted by P. In scheduling problems, P is an integer number of processors, whereas in analytical estimates it may be interpreted as an effective value averaged over a period. The number of agents launched need not coincide with the parallelism actually attainable.

Dependency Graph
A directed acyclic graph $G = (V, E)$ whose vertices correspond to tasks and whose arcs specify precedence relations. The English abbreviation is DAG (*directed acyclic graph*).

Schedule
An assignment of tasks to processors and of their start and completion times. A schedule is called feasible if it respects precedence relations, phase order, the number of agent streams, local retention of the assigned agent slot by a task once started, and the human-resource constraint.

Task Phase
A continuous stage of task execution with a uniquely specified resource type. The model distinguishes autonomous agent phases and human-attention phases; the latter require the developer's attention while simultaneously retaining the corresponding agent slot.

Critical Path
The longest path by duration in the dependency graph. Its length L provides a lower bound on the project duration: increasing the number of agents does not shorten tasks connected sequentially along this path.

Makespan
The time from the start of the project until the completion of the final task with an accepted result. This is the principal time metric in the article. In scheduling theory, the corresponding term is *makespan*.

Total Work
The sum of the durations of all AI-assisted tasks when they are executed in separate streams. It is denoted by W. The quantity W/P provides an idealized estimate under uniform load distribution.

Agent Component
Intervals of task execution during which the agent works autonomously and the developer is free. The normalized duration is denoted by $a_i^{(r)}$.

Human Component
Intervals during which the developer is engaged in task specification, reading, review, or directing corrections for the task. The normalized duration is denoted by $h_i^{(r)}$.

Human Resource
The time and attention of a single developer, shared by all streams. The resource has unit capacity: human-attention intervals of different tasks cannot overlap.

Coordination Overhead
: Additional costs arising from the parallel operation of multiple streams. Cross-stream integration overhead is captured by the function $C(P)$, while the developer's cost of switching among streams is captured by the multiplier $\gamma(P)$.

Local Blocking of an Agent Slot
: The level-M4 rule under which a task, once started, retains its assigned agent until its result is accepted. If a task requests the attention of a busy developer, the agent stops working and remains occupied while waiting; the other agents continue their tasks. Blocking one stream does not imply that the entire system stops.

Queue for Developer Attention
: The set of agent requests awaiting service by a single developer. Waiting is not included in the productive human time H, but it increases the occupancy of retained agent slots and may cause the actual completion time to exceed the lower bound B_4.

Reuse of a Blocked Stream
: Starting another task in place of an agent that is waiting for the developer. Neither this arrangement nor the launch of an additional priority stream beyond P is considered in the article.

Speedup
: The ratio of the no-AI baseline time to the time of the selected AI-assisted configuration. A value greater than one indicates a reduction in project duration, a value equal to one indicates no effect on time, and a value less than one indicates a slowdown.

Lower Bound on Time
: A value below which the makespan cannot fall. At levels M1–M4, the lower bound need not be attainable even under an optimal schedule.

Bottleneck
: A task, path, or resource that determines the lower bound on time in the configuration under consideration. In the model, the bottleneck may be the aggregate load W/P, the longest task M_N, the critical path L, or the human resource $\gamma(P)H$.

Levels M0–M4
: Successive refinements of the model. Level M0 assumes ideal divisibility of work; M1 accounts for task indivisibility, M2 for the dependency graph, M3 redefines durations to account for coordination overhead, and M4 introduces phase ordering, a capacity-constrained human resource, and local blocking.

A.2 Notation

X Baseline effort per unit of work without AI.

N Number of tasks in the project; $i \in \{1, \ldots, N\}$ is the task index.

Z_i Size of task i in units of X.

R, r Set of admissible operating modes and the selected mode.

ρ Mapping that assigns a mode to each task: $\rho\colon \{1,\dots,N\} \to R$.

$T_i^{ai}(r)$
Observed time required to complete task i with AI in a single stream operating in mode r when the developer is fully available.

$k_i^{(r)}$ Normalized AI-assisted duration of task i in mode r: $k_i^{(r)} = T_i^{ai}(r)/(Z_i X)$. Values less than, equal to, and greater than one correspond to durations shorter than, equal to, and longer than the baseline, respectively.

$\hat{k}_i^{(r)}$ Prospective estimate of $k_i^{(r)}$ obtained before the project begins from comparable tasks and operating modes.

$a_i^{(r)}$, $h_i^{(r)}$
Agent and human components of the coefficient $k_i^{(r)}$, where $k_i^{(r)} = a_i^{(r)} + h_i^{(r)}$.

P, P_h
Configured parallelism of the AI-assisted process and parallelism of the no-AI baseline, respectively. This article assumes $P_h = 1$.

σ Shorthand for the process configuration when the tool and acceptance protocol are fixed: $\sigma = (P, r)$ or $\sigma = (P, \rho)$.

T_h Time required for a single developer to complete the entire project sequentially without AI.

T_{ai}, $T_{ai}^{(0)}$
AI-assisted project completion time and its idealized value at level M0.

W Total AI-assisted work: $W = X \sum_{i=1}^{N} Z_i k_i$.

w_i Weight, or duration, of an individual AI-assisted task: $w_i = Z_i k_i X$.

A Total autonomous agent work: $A = X \sum_i Z_i a_i$.

M_N Duration of the longest task: $M_N = X \max_i(Z_i k_i)$.

$G = (V, E)$
Directed acyclic graph of task dependencies.

L Length of the critical path in G under the weights w_i.

$W_3(P)$, $L_3(P)$
Total agent-stream workload and critical-path length at level M3 after applying $C(P)$ only to autonomous agent phases.

$W_4(P)$, $L_4(P)$
The analogous level-M4 quantities after additionally accounting for the multiplier $\gamma(P)$ in the durations of human-attention phases.

$C(P)$
Multiplier for cross-stream coordination and integration overhead; $C(1) = 1$.

H Total human time: $H = X \sum_{i=1}^{N} Z_i h_i^{(r)}$.

$\gamma(P)$
: Multiplier for the developer's attention-switching overhead across parallel streams; $\gamma(1) = 1$.

$Q(\pi)$
: Total time for which assigned agents are blocked in the queue for developer attention under a particular schedule π. It is an endogenous property of the schedule and is included in neither k_i nor H.

$T(\pi)$, T^*
: Makespan of a particular feasible schedule π and the minimum makespan over all feasible schedules, respectively.

$\bar{k}_w$ Weighted-average normalized duration: $\bar{k}_w = \sum_i Z_i k_i / \sum_i Z_i$.

$\bar{a}_w$ Weighted-average normalized agent work: $\bar{a}_w = \sum_i Z_i a_i^{(r)} / \sum_i Z_i$.

$\bar{h}_w$ Weighted-average normalized human work: $\bar{h}_w = \sum_i Z_i h_i^{(r)} / \sum_i Z_i$.

$B_0, \ldots, B_4$
: Lower bounds on the makespan under the successive refinements M0–M4. A transition between levels may add a resource constraint and redefine the effective durations.

S_E, S_E^{ach}
: Idealized and achievable speedup factors.

S_{target}
: Target speedup factor.

μ Ratio of the bottleneck duration to the no-AI baseline time T_h: $\mu = M_N/T_h$ at level M1 and $\mu = L/T_h$ at level M2. The symbol m is reserved for the number of decomposition parts in the computational experiments.

ν_i Expected value of the random normalized duration: $\nu_i = \mathbb{E}[k_i]$.

b, b_0 Ratio of the bottleneck duration to the average workload W/P, and a specified upper bound on this ratio used as a decomposition rule.

κ Common positive multiplier applied to the durations of all agent and human-attention phases in the scale-invariance analysis. Scaling only the aggregate k_i values is insufficient for the level-M4 conclusion.

B Detailed Calculation Examples

B.1 Elementary M0–M1 Examples

We first consider four self-contained numerical examples. In every case, the baseline effort per unit of work is set to $X = 4$ hours.

The idealized AI-assisted completion time is calculated as

$$T_{ai}^{(0)} = \frac{1}{P} \sum_{i=1}^{N} Z_i k_i X.$$

The speedup condition in the idealized model can be written as a parallelism threshold:

$$P > \frac{\sum_{i=1}^{N} Z_i k_i}{\sum_{i=1}^{N} Z_i} = \bar{k}_w.$$

If the configured parallelism P does not exceed the threshold $\bar{k}_w$, no speedup is achieved. The condition $P > \bar{k}_w$ is necessary in all variants of the model; it is sufficient only in the idealized model of perfectly divisible tasks.

For comparison with the more realistic model of indivisible tasks, we also use the following lower bound on the makespan:

$$T_{ai} \geq \max\left\{ \frac{1}{P} \sum_{i=1}^{N} Z_i k_i X,\ \max_{i \in \overline{1,N}} (Z_i k_i X) \right\}.$$

The calculations assume $C(P) \equiv 1$ and $\gamma(P) \equiv 1$. Examples 1–4 cover levels M0, where work is treated as perfectly divisible, and M1, where independent indivisible tasks are considered. The illustrative scenario additionally considers level M2 with a dependency graph and level M4 with the developer as a sequential resource. At level M4, each task has a human-attention component h_i in the decomposition $k_i = a_i + h_i$; the total human-attention time $H = X \sum_i Z_i h_i$ and the speedup ceiling $1/\bar{h}_w$ are calculated. A delegated task is divided into initial task specification, an autonomous agent phase, and final review. From task specification through acceptance, it retains its assigned agent slot, including while it waits for developer attention. The schedules are checked to ensure that human-attention intervals do not overlap and blocked slots are not reused; their sensitivity to $\gamma(P) > 1$ is discussed when the variants are compared. Level M3, which describes cross-stream integration overhead $C(P)$, is not used in the calculations.

B.1.1 Example 1

Table 5: Parameters of Example 1

Parameter	Value
N (number of tasks)	3
$\{Z_i\}$ (task sizes)	$\{2, 3, 1\}$
$\{k_i\}$ (normalized-duration coefficients)	$\{0.5, 0.7, 0.4\}$
P (parallelism)	2
X (baseline effort)	4 h

Parameters.

Calculations.

$$\sum Z_i = 2 + 3 + 1 = 6,$$
$$\sum Z_i k_i = 2{\cdot}0.5 + 3{\cdot}0.7 + 1{\cdot}0.4 = 3.5,$$
$$T_h = 6 \cdot 4 = 24 \text{ h},$$
$$T_{ai}^{(0)} = \frac{1}{2} \cdot 3.5 \cdot 4 = 7 \text{ h},$$
$$S_E = \frac{24}{7} \approx 3.43.$$

For the indivisible-task model, the AI-assisted task weights are

$$\{Z_i k_i X\} = \{4.0, 8.4, 1.6\} \text{ h}.$$

Under an optimal assignment, the first processor receives the 8.4-hour task, and the second receives two tasks with a combined duration of $4.0 + 1.6 = 5.6$ hours. The attainable makespan is therefore 8.4 hours, and the speedup is

$$S_E^{\text{ach}} = \frac{24}{8.4} \approx 2.86.$$

Interpretation. All three tasks have $k_i < 1$, so AI assistance reduces their modeled effort, while parallelism $P = 2$ amplifies this effect. The idealized model yields a 3.43× speedup. After task indivisibility is taken into account, the speedup decreases to 2.86× because the makespan is bounded by the longest task, whose duration is 8.4 hours.

B.1.2 Example 2

Table 6: Parameters of Example 2

Parameter	Value
N (number of tasks)	4
$\{Z_i\}$ (task sizes)	$\{5, 2, 4, 3\}$
$\{k_i\}$ (normalized-duration coefficients)	$\{0.8, 0.6, 0.9, 0.5\}$
P (parallelism)	3
X (baseline effort)	4 h

Parameters.

Calculations.

$$\begin{aligned}
\sum Z_i &= 5 + 2 + 4 + 3 = 14, \\
\sum Z_i k_i &= 5{\cdot}0.8 + 2{\cdot}0.6 + 4{\cdot}0.9 + 3{\cdot}0.5 = 10.3, \\
T_h &= 14 \cdot 4 = 56 \text{ h}, \\
T_{ai}^{(0)} &= \frac{1}{3} \cdot 10.3 \cdot 4 \approx 13.73 \text{ h}, \\
S_E &= \frac{56}{13.73} \approx 4.08.
\end{aligned}$$

For the indivisible-task model, the AI-assisted task weights are

$$\{Z_i k_i X\} = \{16.0, 4.8, 14.4, 6.0\} \text{ h}.$$

Under an optimal workload assignment, the processors receive 16.0, 14.4, and $6.0 + 4.8 = 10.8$ hours of work, respectively. The attainable makespan is 16.0 hours, and the speedup is

$$S_E^{\text{ach}} = \frac{56}{16.0} = 3.50.$$

Interpretation. In Example 2, $k_i < 1$ for every task and the parallelism factor is $P = 3$. The idealized speedup of 4.08× is the largest among the first three examples. Once task indivisibility is taken into account, the makespan is bounded by the 16.0-hour task, so the attainable speedup decreases to 3.50×.

B.1.3 Example 3

Table 7: Parameters of Example 3

Parameter	Value
N (number of tasks)	3
$\{Z_i\}$ (task sizes)	$\{8, 5, 6\}$
$\{k_i\}$ (normalized-duration coefficients)	$\{1.1, 0.7, 0.8\}$
P (parallelism)	2
X (baseline effort)	4 h

Parameters.

Calculations.

$$\begin{aligned}
\sum Z_i &= 8 + 5 + 6 = 19, \\
\sum Z_i k_i &= 8{\cdot}1.1 + 5{\cdot}0.7 + 6{\cdot}0.8 = 17.1, \\
T_h &= 19 \cdot 4 = 76 \text{ h}, \\
T_{ai}^{(0)} &= \frac{1}{2} \cdot 17.1 \cdot 4 = 34.2 \text{ h}, \\
S_E &= \frac{76}{34.2} \approx 2.22.
\end{aligned}$$

For the indivisible-task model, the AI-assisted task weights are

$$\{Z_i k_i X\} = \{35.2, 14.0, 19.2\} \text{ h}.$$

Under an optimal assignment, one processor receives the 35.2-hour task and the other receives two tasks with a combined duration of $19.2 + 14.0 = 33.2$ hours. The attainable makespan is 35.2 hours, and the speedup is

$$S_E^{\text{ach}} = \frac{76}{35.2} \approx 2.16.$$

Interpretation. In Example 3, speedup is retained even though AI assistance increases the modeled effort of one task ($k_i = 1.1$). This same 35.2-hour task is the schedule bottleneck. Consequently, the idealized speedup of 2.22× is only slightly greater than the attainable value of 2.16×; both are lower than in Examples 1 and 2.

B.1.4 Example 4

Parameters.

Table 8: Parameters of Example 4

Parameter	Value
N (number of tasks)	3
$\{Z_i\}$ (task sizes)	$\{4, 3, 3\}$
$\{k_i\}$ (normalized-duration coefficients)	$\{2.4, 2.1, 2.3\}$
P (parallelism)	2
X (baseline effort)	4 h

Calculations.

$$\begin{aligned}
\sum Z_i &= 4 + 3 + 3 = 10, \\
\sum Z_i k_i &= 4{\cdot}2.4 + 3{\cdot}2.1 + 3{\cdot}2.3 = 22.8, \\
\bar{k}_w &= \frac{22.8}{10} = 2.28, \\
T_h &= 10 \cdot 4 = 40 \text{ h}, \\
T_{ai}^{(0)} &= \frac{1}{2} \cdot 22.8 \cdot 4 = 45.6 \text{ h}, \\
S_E &= \frac{40}{45.6} \approx 0.88.
\end{aligned}$$

The threshold condition for speedup is not satisfied:

$$P = 2 < \bar{k}_w = 2.28.$$

Consequently, the configured parallelism is insufficient to offset the increase in modeled task effort under AI-assisted execution.

For the indivisible-task model, the AI-assisted task weights are

$$\{Z_i k_i X\} = \{38.4, 25.2, 27.6\} \text{ h}.$$

For $P = 2$, the optimal assignment is $\{38.4\}$ and $\{25.2{+}27.6 = 52.8\}$ hours. The attainable makespan is therefore 52.8 hours, and the speedup is lower than the idealized estimate:

$$S_E^{\text{ach}} = \frac{40}{52.8} \approx 0.76 < 1.$$

The makespan lower bound is $\max\{45.6,\ 38.4\} = 45.6$ hours, but it is not attained: the optimum is 52.8 hours. This is the only one of the four examples in which $\max\{W/P,\ \max_i Z_i k_i X\}$ is strictly less than the optimum. The expression is thus a lower bound rather than a result guaranteed to be attainable. The gap remains within Graham's guarantee: $52.8 \le (2 - 1/2) \cdot 45.6 = 68.4$ hours.

The sufficient speedup criterion is not satisfied either. The longest-task share is $\mu = M_N/T_h = 38.4/40 = 0.96$, and the threshold is $P - (P-1)\,\mu = 2 - 0.96 = 1.04$, whereas $\bar{k}_w = 2.28 > 1.04$. Moreover, even the necessary condition $\bar{k}_w < P = 2$ is violated, consistently with the absence of speedup.

Interpretation. In Example 4, all coefficients k_i are substantially greater than one, so AI assistance markedly increases the modeled effort of every task. With $\bar{k}_w = 2.28$, the configured parallelism $P = 2$ is insufficient to offset this slowdown. Even the idealized model shows an increase in completion time from 40 to 45.6 hours; after task indivisibility and load imbalance are taken into account, the makespan increases to 52.8 hours.

B.2 Summary Comparison

Table 9: Summary of the Calculation Results

Metric	Ex. 1	Ex. 2	Ex. 3	Ex. 4
T_h (without AI), h	24.0	56.0	76.0	40.0
$T_{ai}^{(0)}$ (with AI), h	7.0	13.73	34.2	45.6
Idealized speedup S_E	3.43×	4.08×	2.22×	0.88×
Attainable makespan, h	8.4	16.0	35.2	52.8
Attainable speedup S_E^{ach}	2.86×	3.50×	2.16×	0.76×
Speedup condition met?	Yes	Yes	Yes	No

Remark 15 (Final Interpretation)**.**

1. Examples 1–3 achieve a speedup both under the idealized formula and under the more realistic assignment of indivisible tasks.
2. The difference between the idealized and attainable speedups is due to the longest task: increasing parallelism cannot reduce the makespan below its duration.
3. Example 2 yields the greatest speedup because it combines $k_i < 1$ for every task with greater parallelism, $P = 3$.
4. Example 3 exhibits a more moderate effect because one of the largest tasks has $k_i > 1$ and becomes the critical bottleneck in the schedule.
5. Example 4 illustrates an adverse scenario: even with $P = 2$, the parallelism threshold is not satisfied ($P < \bar{k}_w$), so $T_{ai}^{(0)} > T_h$ and no speedup is achieved.

B.3 Full Parameters of the Illustrative Scenario

We now move from abstract task sets to an illustrative scenario involving the development of a small microservice. This is a synthetic scenario with author-specified parameters, not an empirical study of a particular project. The same project is considered in four variants to separate the influence of two factors. In Variants 1–3, the task set remains unchanged while the *process configuration* changes: the parallelism and the agents' operating mode. Variant 4 retains the best of these configurations but changes the *decomposition*: a large task on the critical path is split into subtasks. This comparison makes it possible to assess separately the effect of work organization and the effect of a finer project decomposition.

As before, the baseline effort of one unit of work is set to $X = 4$ hours.

B.3.1 Scenario Definition

Consider a project to develop a small microservice for processing user requests. The service accepts a request through an HTTP API, validates the input, persists the state, initiates asynchronous processing, and provides observability facilities. The project is compact enough to permit a complete analysis, yet includes typical elements of production software development: an API contract, data persistence, business rules, background processing, tests, and infrastructure configuration.

For concreteness, assume that the service must support the following minimum set of capabilities:

- creating a request through the HTTP API;
- validating required fields and using a uniform error format;
- persisting the request and its status in a database;
- moving a request through the following states: created, accepted for processing, processed, and completed with an error;
- background processing with retries after transient errors;
- structured logs, metrics, a health check, and basic configuration;
- a suite of unit and integration tests;
- packaging and a minimal configuration for execution in a continuous integration and deployment environment.

For transparent parameterization of the illustrative scenario, we use notional story points (SP) as units of task size. Let 2 SP correspond to one unit of Z_i. One story point then corresponds to $X/2 = 2$ hours, while the baseline effort associated with one unit of Z_i remains $X = 4$ hours. Subtasks with an odd number of story points have fractional task sizes ($1\ \text{SP} = 0.5\,Z$), which is consistent with the condition $Z_i \in \mathbb{Q}^+$ in Definition 1. Calculations in Variants 1–3 use the aggregate tasks, whereas Variant 4 directly uses the decomposition into subtasks.

The baseline decomposition of the project into aggregate tasks is shown in Table 10.

Table 10: Baseline decomposition of the project into tasks

i	Task	Estimate, SP	Task size Z_i
1	API contract and request validation	4	2
2	Persistence layer (model, migrations, repository)	6	3
3	Business logic (primary use cases and state transitions)	8	4
4	Background worker (asynchronous processing and retries)	6	3
5	Observability and configuration (logs, metrics, health check)	4	2
6	Tests and integration checks	6	3
7	Packaging and deployment configuration (Docker, CI)	4	2

The composition of the aggregate tasks is detailed in Table 11. This detail is important for planning: at the subtask level, it becomes apparent which work can be delegated to agents independently and which work requires a contract to have been fixed in advance or the result of another task. The detailed decomposition is used below to identify dependencies introduced by graph coarsening and in Variant 4, where the testing task is split into separate components.

The total project size in terms of the aggregate tasks is

$$\sum_{i=1}^{7} Z_i = 2 + 3 + 4 + 3 + 2 + 3 + 2 = 19.$$

Table 11: Subtasks and story points

i	Task	Subtask	SP
1	API	Specify endpoints, DTOs, and the error format	2
1	API	Implement data validation and error handling	2
2	Persistence	Design database tables and migrations	2
2	Persistence	Implement the repository and data-access methods	3
2	Persistence	Add basic test data for local verification	1
3	Business logic	Specify states and valid transitions	2
3	Business logic	Implement the primary use cases	4
3	Business logic	Handle domain-level errors	2
4	Background worker	Implement retrieval of tasks for processing	2
4	Background worker	Add retries and failure handling	3
4	Background worker	Synchronize statuses with the primary model	1
5	Observability	Add structured logs and metrics	2
5	Observability	Configure settings and the health check	2
6	Tests	Cover the business logic with unit tests	2
6	Tests	Add integration tests for the API and persistence layer	3
6	Tests	Verify background processing and retries	1
7	Packaging	Prepare the Docker/service configuration	2
7	Packaging	Add continuous-integration commands and a smoke test	2

The equivalent total is 38 SP, which, at 2 hours per story point, also yields 76 hours of baseline work.

The no-AI completion time (one developer working sequentially) is

$$T_h = 19 \cdot 4 = 76 \text{ h}.$$

The tasks are linked by precedence relations. To plan parallel work, define a graph $G = (V, E)$ whose vertices correspond to the aggregate tasks in Table 10. The dependencies are listed in Table 12.

In edge notation, the graph is

$$E = \{1{\to}2,\ 1{\to}3,\ 1{\to}5,\ 1{\to}7,\ 2{\to}3,\ 2{\to}4,\ 3{\to}4,\ 3{\to}6,\ 4{\to}6,\ 5{\to}7\}.$$

This graph implies a natural wave-based schedule:

1. first, the API contract and basic DTOs are fixed;
2. the persistence layer and observability can then be undertaken in parallel;
3. once the persistence layer is complete, the full business logic becomes available, while packaging can be prepared in parallel;
4. the background worker depends on the business logic and data model;
5. integration tests and background-processing checks complete the critical path.

Table 12: Dependency graph among the aggregate tasks

Task	Depends on	Work that can proceed in parallel
1. API contract and request validation	—	Initial task; establishes interfaces for the remaining work
2. Persistence layer	1	Can proceed in parallel with observability once the API is fixed
3. Business logic	1, 2	Can be partially designed after the API is fixed; implementation requires the persistence layer
4. Background worker	2, 3	Requires the data model and state-transition rules
5. Observability and configuration	1	Can proceed in parallel with the persistence layer and business logic
6. Tests and integration checks	1, 2, 3, 4	Unit tests can begin earlier; integration tests depend on the core components
7. Packaging and deployment	1, 5	Can be prepared in parallel once the interfaces and configuration are fixed

The project thus combines parallelizable portions with strict dependencies. In Variants 1–3, the aggregate values Z_i and coefficients k_i are used, but completion time is determined not by balancing independent tasks, but by a schedule on the graph in Table 12.

Actual Dependencies and Coarsening Artifacts. The coarsened graph conceals an important distinction. Some edges in Table 12 represent *substantive* dependencies: the business logic cannot rely on a data model that has not yet been created, nor can the background worker rely on undefined state transitions. Other edges arise solely from *task coarsening*: they connect entire tasks even though only individual subtasks are actually dependent. For example, the dependence of Task 6 on Tasks 1–4 means that the test suite contains checks for each component. Nevertheless, unit tests for the business logic can begin immediately after Task 3, integration tests can begin once the API and persistence layer are complete, and only the retry tests require a completed background worker. Similarly, at the subtask level, the dependency $3 \to 4$ pertains only to the 2-SP state-and-transition model within the 8-SP task; the background worker does not require the remaining business-logic use cases. A substantive dependency cannot be eliminated by rescheduling, whereas a dependency induced by coarsening can be refined by replacing one vertex with several subtasks. The dominance of the critical path in the calculations below is therefore determined in part by the chosen graph granularity. Variant 4 quantifies this effect.

In Variants 1–3, the project and aggregate task set remain unchanged, but the process configurations $\sigma = (P, r)$ differ: the number of parallel streams P, the operating-mode assignment to tasks, and the stream assignment. Because the coefficient $k_i = k_i^{(r)}$ depends on the "task × tool × mode" triple, interactive work in Variant 1, mixed-mode work in Variant 2, and delegated work from detailed specifications in Variant 3 define different vectors $\{k_i\}$ for the same task set. The tool and acceptance rules are fixed within the scenario. Entire configurations are therefore compared, rather than an isolated change in a single parameter. Variant 4 reproduces the configuration of Variant 3 and changes only

the decomposition, allowing its effect to be assessed separately.

At level M4, competition among streams for developer attention is modeled explicitly. Each task has a human component h_i that includes task specification, review, and the direction of corrections. A schedule is considered feasible only if the human-attention intervals of different tasks do not overlap and one agent remains continuously assigned to every task that has started but has not yet been accepted. If a human-attention phase cannot begin immediately, the agent remains blocked in the queue; other agents continue working. In interactive mode, $h_i = k_i$ because the developer retains the task context throughout its execution; in delegated mode, h_i is substantially smaller than k_i but is nonzero because task specification and acceptance are still required. The values of h_i, like those of k_i, are assigned by expert judgment. Two simplifying assumptions are retained: $C(P) \equiv 1$, meaning that cross-stream integration overhead is not modeled, and $\gamma(P) \equiv 1$. The effect of attention switching is considered separately when the variants are compared.

B.3.2 Variant 1. Interactive Work in a Single Stream

The developer completes the tasks sequentially using an AI assistant. For each task, the developer formulates a prompt, receives the result, reviews it, and corrects it if necessary. There is no parallelism ($P = 1$). The configuration is $\sigma_1 = (P{=}1,\ \text{interactive mode})$.

Table 13: Parameters of Variant 1

i	Task	Z_i	k_i	h_i
1	API contract and request validation	2	0.80	0.80
2	Persistence layer	3	0.90	0.90
3	Business logic	4	0.85	0.85
4	Background worker	3	0.95	0.95
5	Observability and configuration	2	0.75	0.75
6	Tests	3	0.80	0.80
7	Packaging and deployment	2	0.90	0.90

Parameters. The coefficients k_i encode moderate acceleration: AI assists in generating boilerplate code and tests, but every result undergoes manual review. For tasks with high architectural complexity (business logic and the background worker), the coefficient is closer to one. The human component satisfies $h_i = k_i$: in interactive mode, the developer retains the task context throughout its execution (the agent component is $a_i = 0$).

Calculations.

$$\sum_{i=1}^{7} Z_i k_i = 2{\cdot}0.80 + 3{\cdot}0.90 + 4{\cdot}0.85 + 3{\cdot}0.95 + 2{\cdot}0.75 + 3{\cdot}0.80 + 2{\cdot}0.90$$
$$= 1.60 + 2.70 + 3.40 + 2.85 + 1.50 + 2.40 + 1.80 = 16.25.$$

The weighted-average normalized duration is

$$\bar{k}_w = \frac{16.25}{19} \approx 0.855.$$

The task weights in the graph are $w_i = Z_i k_i X$:

$$\{w_i\}_{i=1}^{7} = \{6.4,\ 10.8,\ 13.6,\ 11.4,\ 6.0,\ 9.6,\ 7.2\} \text{ h}.$$

The total work is

$$W = \sum_{i=1}^{7} w_i = 16.25 \cdot 4 = 65.0 \text{ h}.$$

The idealized completion time for independent tasks, that is, the lower bound based on total work when $P = 1$, is

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{65.0}{1} = 65.0 \text{ h}.$$

The idealized speedup without accounting for dependencies is

$$S_E = \frac{76}{65.0} \approx 1.17.$$

The critical path through the graph comprises Tasks $1 \to 2 \to 3 \to 4 \to 6$:

$$L = 6.4 + 10.8 + 13.6 + 11.4 + 9.6 = 51.8 \text{ h}.$$

The lower bound on the makespan accounting for the dependency graph is

$$B_4 = \max\left\{\frac{W}{P},\ L,\ H\right\} = \max\{65.0,\ 51.8,\ 65.0\} = 65.0 \text{ h}.$$

When $P = 1$, the schedule is sequential. For example, the tasks may be completed in the topological order $1, 2, 3, 4, 5, 6, 7$. The makespan equals the total work:

$$T(\pi_1) = T^* = B_4 = 65.0 \text{ h}.$$

The achievable speedup accounting for the graph is

$$S_E^{\text{ach}} = \frac{76}{65.0} \approx 1.17.$$

At level M4, the human-resource calculation is as follows because $h_i = k_i$:

$$H = X \sum_{i=1}^{7} Z_i h_i = 65.0 \text{ h} = W, \qquad \bar{h}_w = \bar{k}_w \approx 0.855, \qquad \frac{1}{\bar{h}_w} \approx 1.17.$$

Interpretation. Interactive mode provides direct control over the result. AI reduces the effort required for each task ($\bar{k}_w \approx 0.855$), but the absence of parallelism limits the overall effect. The critical path is not the active constraint: when $P = 1$, $W/P = 65.0$ h exceeds $L = 51.8$ h. The speedup is $1.17\times$, corresponding to a savings of approximately 11 hours relative to the 76-hour no-AI baseline.

At level M4, the achieved speedup of $1.17\times$ exactly equals the human-resource ceiling $1/\bar{h}_w$. In interactive mode, the makespan equals the total human time H. Consequently, further process reorganization cannot yield an improvement without reducing h, that is, without transferring part of the work to agents for autonomous execution.

B.3.3 Variant 2. Interactive and Delegated Tasks

In this variant, two agent streams are available ($P = 2$). Tasks that require architectural decisions are performed interactively, whereas more formalized tasks are delegated to an agent and accepted by the developer after an autonomous phase. The operating mode applies to a task rather than being fixed to a particular agent: any available agent may receive the next ready task in either of the prescribed modes.

Tasks are assigned to operating modes as follows:

- **Interactive mode**: API contract, business logic, and background worker—tasks that require context and architectural decisions.
- **Delegated mode**: persistence layer, observability, tests, and packaging—tasks with more formalized requirements.

The configuration is $\sigma_2 = (P{=}2,\ \rho)$ with the mixed mode assignment ρ: tasks in Stream 1 are performed in interactive mode, and tasks in Stream 2 are performed in delegated mode.

Table 14: Parameters of Variant 2

i	Task	Z_i	k_i	h_i	Mode
1	API contract and request validation	2	0.75	0.75	interact.
2	Persistence layer	3	0.75	0.20	deleg.
3	Business logic	4	0.85	0.85	interact.
4	Background worker	3	0.90	0.90	interact.
5	Observability and configuration	2	0.70	0.20	deleg.
6	Tests	3	0.70	0.20	deleg.
7	Packaging and deployment	2	0.80	0.20	deleg.

Parameters. In delegated mode, the coefficients k_i are slightly lower than in Variant 1. This difference results from the change in operating mode: the agent works independently and does not wait for intermediate responses from the developer. This effect arises even in a single stream and is consistent with the definition of $k_i^{(r)}$ as a characteristic of the "task $\times$ tool $\times$ operating mode" triple; the tool is held fixed in the scenario. For interactive tasks, the coefficients are close to those in Variant 1, and $h_i = k_i$ because the developer is occupied throughout their execution. For delegated tasks, $h_i = 0.20$ is assumed; this value accounts for task specification and review of the result. In the schedule, the human time is divided equally between initial task specification and final review, and the agent slot is retained over the entire interval from the beginning of specification through completion of review.

Calculations.

$$\sum_{i=1}^{7} Z_i k_i = 2{\cdot}0.75 + 3{\cdot}0.75 + 4{\cdot}0.85 + 3{\cdot}0.90 + 2{\cdot}0.70 + 3{\cdot}0.70 + 2{\cdot}0.80$$

$$= 1.50 + 2.25 + 3.40 + 2.70 + 1.40 + 2.10 + 1.60 = 14.95.$$

The weighted-average normalized duration is

$$\bar{k}_w = \frac{14.95}{19} \approx 0.787.$$

The task weights in the graph are $w_i = Z_i k_i X$:

$$\{w_i\}_{i=1}^{7} = \{6.0,\ 9.0,\ 13.6,\ 10.8,\ 5.6,\ 8.4,\ 6.4\} \text{ h}.$$

The total work is

$$W = \sum_{i=1}^{7} w_i = 14.95 \cdot 4 = 59.8 \text{ h}.$$

The idealized completion time for independent tasks, that is, the lower bound based on total work when $P = 2$, is

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{59.8}{2} = 29.9 \text{ h}.$$

The idealized speedup without accounting for dependencies is

$$S_E = \frac{76}{29.9} \approx 2.54.$$

The critical path through the graph comprises Tasks $1 \to 2 \to 3 \to 4 \to 6$:

$$L = 6.0 + 9.0 + 13.6 + 10.8 + 8.4 = 47.8 \text{ h}.$$

The human-attention time at level M4 is

$$\sum_{i=1}^{7} Z_i h_i = 2{\cdot}0.75 + 3{\cdot}0.20 + 4{\cdot}0.85 + 3{\cdot}0.90 + 2{\cdot}0.20 + 3{\cdot}0.20 + 2{\cdot}0.20 = 9.6,$$

$$H = 9.6 \cdot 4 = 38.4 \text{ h}, \qquad \bar{h}_w = \frac{9.6}{19} \approx 0.505, \qquad \frac{1}{\bar{h}_w} \approx 1.98.$$

The lower bound on the makespan accounting for the dependency graph and the human resource is

$$B_4 = \max\left\{\frac{W}{P},\ L,\ H\right\} = \max\{29.9,\ 47.8,\ 38.4\} = 47.8 \text{ h}.$$

The assignment to streams from Table 14 that respects the dependencies is

Stream	Task intervals, h	Idle time, h
1	1: $[0, 6.0]$; 3: $[15.0, 28.6]$; 4: $[28.6, 39.4]$	$[6.0, 15.0]$, waiting for the persistence layer
2	2: $[6.0, 15.0]$; 5: $[15.0, 20.6]$; 7: $[20.6, 27.0]$; 6: $[39.4, 47.8]$	$[27.0, 39.4]$, waiting for dependencies

If only the two agent streams and the precedence relations are considered, this assignment has a makespan of 47.8 hours, numerically equal to the lower bound. However, as the following check shows, it is not a feasible level-M4 schedule.

$$T_{\text{agents}} = B_4 = 47.8 \text{ h}.$$

Human-Resource Check (Level M4). The schedule above is feasible with respect to the streams and dependencies but *infeasible with respect to the human resource*. The developer is continuously occupied by interactive tasks over the interval [15.0, 39.4]: first by Task 3 and then by Task 4. At the same time, delegated Tasks 5 [15.0, 20.6] and 7 [20.6, 27.0] each require 1.6 hours of human attention for task specification and review. These intervals overlap the interactive work, which is impossible with a single developer. Consequently, the schedule that is formally feasible on the agent streams does not provide the resource capacity needed for the timely acceptance of the delegated results.

The conflict can be eliminated by restructuring the schedule: delegated tasks and their human-attention phases are placed in intervals when the developer is available. Once started, a task continuously retains its assigned agent slot, including while it waits for final review. The corrected schedule is as follows:

- Agent 1: Task 1 [0, 6.0]; Task 2 [6.0, 15.0]; Task 3 [15.0, 28.6]; Task 4 [28.6, 39.4]; Task 6 [39.4, 47.8].
- Agent 2: Task 5 [7.2, 12.8]; Task 7 [12.8, 41.4]. For Task 7, specification occupies [12.8, 13.6], autonomous work occupies [13.6, 18.4], waiting for review occupies [18.4, 40.6], and review occupies [40.6, 41.4].
- Developer: Task 1 [0, 6.0]; specification of Task 2 [6.0, 7.2]; specification of Task 5 [7.2, 8.0]; review of Task 5 [12.0, 12.8]; specification of Task 7 [12.8, 13.6]; review of Task 2 [13.8, 15.0]; Task 3 [15.0, 28.6]; Task 4 [28.6, 39.4]; specification of Task 6 [39.4, 40.6]; review of Task 7 [40.6, 41.4]; review of Task 6 [46.6, 47.8]. The developer intervals are pairwise nonoverlapping, and their total duration is 38.4 hours $= H$.

All dependencies are satisfied. Task 3 starts at time 15.0, after Task 2 has been fully accepted; Task 7 starts after Task 5 has been accepted; and Task 6 starts after Task 4. The waiting interval of Task 7 is not used for other work on Agent 2. Despite this local blocking, the makespan remains 47.8 hours because Task 7 is not on the critical path. Thus, the human resource does not increase the makespan only when the phases and the queue are planned explicitly; the original schedule was infeasible.

The corrected schedule is feasible and attains the lower bound; it is therefore optimal:

$$T(\pi_2) = T^* = B_4 = 47.8 \text{ h}.$$

The attainable speedup accounting for the graph and the human resource is

$$S_E^{\text{ach}} = \frac{76}{47.8} \approx 1.59.$$

Interpretation. If the tasks were treated as independent, the lower bound $W/P = 29.9$ hours would correspond to a speedup of approximately 2.54×. However, the dependency graph substantially reduces the exploitable parallelism: Tasks $1 \to 2 \to 3 \to 4 \to 6$ form a critical path of length 47.8 hours. After the graph is taken into account, the attainable speedup decreases to 1.59×. Observability and deployment do proceed in parallel, but they are outside the critical path and have almost no effect on the overall makespan.

The key observation is that parallelism $P = 2$ helps only where the graph permits independent work. In this variant, most of the effort lies on a sequential chain, so balancing the streams without accounting for dependencies overstates the effect of the configuration in the illustrative scenario.

The human resource is not yet the active constraint: $H = 38.4$ hours $< L = 47.8$ hours, and the attained speedup remains below the ceiling $1/\bar{h}_w \approx 1.98$ ($1.59 < 1.98$). This residual capacity can be used, however, only when review intervals are explicitly scheduled. The three interactive tasks with $h = k$ occupy the developer for a total of 30.4 hours, so reviews of delegated tasks must be placed in the remaining intervals.

B.3.4 Variant 3. Delegated Agents with Detailed Specifications

In this variant, four AI agents work in parallel ($P = 4$). The configuration is specified as $\sigma_3 = (P{=}4,$ delegated mode with detailed specifications). The coefficients k_i are lower than in the preceding variants because the operating-mode profile changes, not because parallelism increases: the agents receive detailed specifications and execute the autonomous agent phases, after which the developer accepts the result. The dependency graph among the aggregate tasks remains unchanged and limits the exploitable parallelism.

Table 15: Parameters of Variant 3

i	Task	Z_i	k_i	h_i
1	API contract and request validation	2	0.65	0.25
2	Persistence layer	3	0.70	0.25
3	Business logic	4	0.80	0.25
4	Background worker	3	0.75	0.25
5	Observability and configuration	2	0.60	0.25
6	Tests	3	0.65	0.25
7	Packaging and deployment	2	0.70	0.25

Parameters. The same human component, $h_i = 0.25$, is assumed for all tasks. The developer's involvement consists of preparing a detailed specification and subsequently reviewing the result; implementation is delegated entirely to the agent.

Calculations.

$$\sum_{i=1}^{7} Z_i k_i = 2{\cdot}0.65 + 3{\cdot}0.70 + 4{\cdot}0.80 + 3{\cdot}0.75 + 2{\cdot}0.60 + 3{\cdot}0.65 + 2{\cdot}0.70$$
$$= 1.30 + 2.10 + 3.20 + 2.25 + 1.20 + 1.95 + 1.40 = 13.40.$$

The weighted-average normalized duration is

$$\bar{k}_w = \frac{13.40}{19} \approx 0.705.$$

The task weights in the graph are $w_i = Z_i k_i X$:

$$\{w_i\}_{i=1}^{7} = \{5.2,\ 8.4,\ 12.8,\ 9.0,\ 4.8,\ 7.8,\ 5.6\} \text{ h}.$$

The total work is

$$W = \sum_{i=1}^{7} w_i = 13.40 \cdot 4 = 53.6 \text{ h}.$$

The idealized completion time for independent tasks, that is, the lower bound based on total work when $P = 4$, is

$$T_{ai}^{(0)} = \frac{W}{P} = \frac{53.6}{4} = 13.4 \text{ h}.$$

The idealized speedup without accounting for dependencies is

$$S_E = \frac{76}{13.4} \approx 5.67.$$

The critical path through the graph comprises Tasks $1 \to 2 \to 3 \to 4 \to 6$:

$$L = 5.2 + 8.4 + 12.8 + 9.0 + 7.8 = 43.2 \text{ h}.$$

The human time at level M4 is

$$\sum_{i=1}^{7} Z_i h_i = 0.25 \cdot 19 = 4.75, \qquad H = 4.75 \cdot 4 = 19.0 \text{ h}, \qquad \bar{h}_w = 0.25, \qquad \frac{1}{\bar{h}_w} = 4.0.$$

The lower bound on the makespan accounting for the dependency graph and the human resource is

$$B_4 = \max\left\{\frac{W}{P},\ L,\ H\right\} = \max\{13.4,\ 43.2,\ 19.0\} = 43.2 \text{ h}.$$

One feasible assignment to four agents that respects the dependencies is

- Agent 1: Task 1 $[0, 5.2]$, Task 2 $[5.2, 13.6]$, Task 3 $[13.6, 26.4]$, Task 4 $[26.4, 35.4]$, and Task 6 $[35.4, 43.2]$.
- Agent 2: Task 5 $[6.7, 11.5]$ and Task 7 $[15.6, 21.2]$.
- Agents 3 and 4 are idle because, at this coarse graph granularity, the critical path dominates the exploitable parallelism.

This schedule attains the lower bound:

$$T(\pi_3) = T^* = B_4 = 43.2 \text{ h}.$$

Human-Resource Check (Level M4). To prevent an agent from starting work before task specification, the human time is divided equally between preparation of the input specification and final review; the agent performs the implementation between these stages. The developer's blocks are scheduled as follows: Task 1—[0, 1.0] and [4.2, 5.2]; Task 2—[5.2, 6.7] and [12.1, 13.6]; Task 5—[6.7, 7.7] and [10.5, 11.5]; Task 3—[13.6, 15.6] and [24.4, 26.4]; Task 7—[15.6, 16.6] and [20.2, 21.2]; Task 4—[26.4, 27.9] and [33.9, 35.4]; and Task 6—[35.4, 36.9] and [41.7, 43.2]. The intervals are pairwise nonoverlapping, and each input specification precedes the agent component of the corresponding task. Total human utilization is 19.0 h = H over a project duration of 43.2 h, so the schedule is feasible without modification.

The achievable speedup accounting for the graph and the human resource is

$$S_E^{\text{ach}} = \frac{76}{43.2} \approx 1.76.$$

Interpretation. With four agents, total work decreases to $W = 53.6$ h, and the estimate for independent tasks is $W/P = 13.4$ h. Under the coarsened graph, however, this value is unattainable: the chain $1 \to 2 \to 3 \to 4 \to 6$ has length 43.2 h and becomes the primary constraint. After dependencies are taken into account, the speedup is therefore not $\times 5.67$ but $\times 1.76$.

The improvement relative to Variant 2 should be interpreted with caution. The critical path decreases from 47.8 to 43.2 h because the coefficients k_i decrease upon switching to delegated mode with detailed specifications. The increase in P from 2 to 4 does not itself affect L: additional agents do not accelerate the sequential chain of aggregate tasks and are idle in this schedule. The next change to the scenario is therefore not a further increase in the number of agents, but the decomposition of large tasks on the critical path into smaller, independently verifiable components. Variant 4 considers precisely this change.

Level M4 imposes an additional upper bound on speedup. In this variant, the human-resource ceiling $1/\bar{h}_w = 4.0$ is substantially higher than the achieved speedup of $\times 1.76$, so L remains the active constraint. Nevertheless, under the selected mode ($\bar{h}_w = 0.25$), neither decomposition nor any number of agents can provide speedup greater than $\times 4.0$: the 19 hours of sequential human time cannot be eliminated. As the critical path is shortened, H becomes the active constraint.

B.3.5 Variant 4. The Same Configuration with Finer Decomposition

This variant differs from Variant 3 only in its decomposition. The number of agents $P = 4$, delegated mode with detailed specifications, the coefficients k, and the human components h remain unchanged. Only the graph changes: the aggregate Task 6, "Tests," is replaced by the three subtasks from Table 11, each with its exact dependencies. In Variants 1–3, the configuration changed while the decomposition remained fixed; here, by contrast, the decomposition changes while the configuration remains fixed. The difference from Variant 3 is therefore attributable solely to decomposition.

Parameters. Tasks 1–5 and 7 remain unchanged (Table 15). Task 6 is decomposed as specified in Table 11:

Table 16: Subtasks of Task 6 in Variant 4

i	Subtask	SP	Z_i	k_i / h_i	Depends on
6a	Unit tests for the business logic	2	1	0.65 / 0.25	3
6b	Integration tests for the API and persistence layer	3	1.5	0.65 / 0.25	1, 2
6c	Verification of background processing and retries	1	0.5	0.65 / 0.25	4

The decomposition is assumed *not to change the amount of work*: the subtasks inherit $k = 0.65$ and $h = 0.25$ from the original task. The quantities W, $\bar{k}_w$, and H are therefore unchanged, so the comparison with Variant 3 isolates the structural effect. In practice, decomposition entails additional overhead for preparing specifications, issuing tasks, and accepting each subtask. Under the adopted accounting rule, this overhead is included in $k^{(r)}$ and increases it when decomposition becomes excessive. Conversely, k may decrease for small tasks with clear boundaries. Thus, holding k and h constant is a neutral scenario assumption, and the sensitivity of the result to decomposition overhead requires separate calibration.

The subtask sizes are fractional ($Z_{6b} = 1.5$, $Z_{6c} = 0.5$): an odd number of story points under the conversion 2 SP = 1 Z. The subtask weights are

$$w_{6a} = 1 \cdot 0.65 \cdot 4 = 2.6 \text{ h}, \qquad w_{6b} = 1.5 \cdot 0.65 \cdot 4 = 3.9 \text{ h}, \qquad w_{6c} = 0.5 \cdot 0.65 \cdot 4 = 1.3 \text{ h};$$

together, they total 7.8 h, the weight of the original Task 6.

Calculations. The aggregate quantities are the same as in Variant 3:

$$W = 53.6 \text{ h}, \qquad \bar{k}_w \approx 0.705, \qquad \frac{W}{P} = 13.4 \text{ h}, \qquad H = 19.0 \text{ h}, \qquad \frac{1}{\bar{h}_w} = 4.0.$$

The critical path changes. The monolithic vertex 6 and its incoming edges $3 \to 6$ and $4 \to 6$ are replaced by three vertices with the exact dependencies $3 \to$ 6a, $\{1, 2\} \to$ 6b, and $4 \to$ 6c, and the longest path becomes

$$1 \to 2 \to 3 \to 4 \to 6c: \qquad L = 5.2 + 8.4 + 12.8 + 9.0 + 1.3 = 36.7 \text{ h}$$

(the competing paths are shorter: $1 \to 2 \to 3 \to$ 6a yields 29.0 h, $1 \to 2 \to$ 6b yields 17.5 h, and $1 \to 5 \to 7$ yields 15.6 h). The tail of the critical path decreases from 7.8 h (all of Task 6) to 1.3 h (only the component that actually waits for the background worker).

The lower bound on the makespan is

$$B_4 = \max\left\{\frac{W}{P},\ L,\ H\right\} = \max\{13.4,\ 36.7,\ 19.0\} = 36.7 \text{ h}.$$

A feasible assignment that respects the dependencies is

- Agent 1: Task 1 [0, 5.2], Task 2 [5.2, 13.6], Task 3 [13.6, 26.4], Task 4 [26.4, 35.4], and Task 6c [35.4, 36.7].
- Agent 2: Task 5 [6.7, 11.5]; for Task 7, the input specification occupies [19.5, 20.5], agent implementation occupies [20.5, 24.1], and the result then waits in the queue for developer attention until the review at [28.4, 29.4]. During the waiting interval [24.1, 28.4], the agent remains assigned to Task 7 and cannot receive other work.
- Agent 3: Task 6b [15.6, 19.5] and Task 6a [27.9, 30.5]. Agent 4 is idle.

This schedule attains the lower bound:

$$T(\pi_4) = T^* = B_4 = 36.7 \text{ h}.$$

Human-Resource Check (Level M4). As in Variant 3, the human time for each task is divided equally between the input specification and final review. The developer's blocks are as follows: Task 1—[0, 1.0] and [4.2, 5.2]; Task 2—[5.2, 6.7] and [12.1, 13.6]; Task 5—[6.7, 7.7] and [10.5, 11.5]; Task 3—[13.6, 15.6] and [24.4, 26.4]; Task 6b—[15.6, 16.35] and [18.75, 19.5]; Task 7—[19.5, 20.5] and [28.4, 29.4]; Task 4—[26.4, 27.9] and [33.9, 35.4]; Task 6a—[27.9, 28.4] and [30.0, 30.5]; and Task 6c—[35.4, 35.65] and [36.45, 36.7]. The intervals are pairwise nonoverlapping, each input specification precedes agent implementation, and total human utilization is 19.0 h = H. The schedule is therefore feasible with respect to the human resource and does indeed attain 36.7 h.

The achievable speedup accounting for the graph and the human resource is

$$S_E^{\text{ach}} = \frac{76}{36.7} \approx 2.07.$$

Interpretation. The makespan decreases from 43.2 to 36.7 h, while speedup increases from ×1.76 to ×2.07, an increase of 18%, without adding agents, changing the operating mode, or improving the tool. This effect is obtained by replacing one aggregate vertex with three subtasks already identified in Table 11. The dependency "testing after completion of the entire core" was an artifact of graph coarsening; refining it shortens the critical path by 6.5 h. In the comparison considered here, the result is determined by decomposition, not by the number of agents.

Only the transition from Variant 3 to Variant 4 is an all-else-equal comparison: the decrease from 43.2 to 36.7 h is attributable to decomposition with P, k, and h held constant. In the transition from Variant 2 to Variant 3, both P and the operating mode change, so their effects are not separated. The calculation nevertheless shows that the reduction in the active bound L from 47.8 to 43.2 h is associated with lower k values along the critical path. The value of P does not itself enter the formula for L, and the additional agents are idle.

The comparative conclusion for Variants 3 and 4 also holds without an exact absolute calibration. By Proposition 5, multiplying all agent and human-attention phases in both variants by a common factor $\kappa > 0$ multiplies both makespans by κ but leaves the ratio 43.2/36.7 and the ranking of the variants unchanged. This robustness does not extend to task-specific errors of unequal magnitude or to changes in the relative profiles of k_i and h_i.

Decomposition can be continued iteratively. The next candidate is the dependency $3 \to 4$, which at the subtask level reduces to the 2-SP state model within the 8-SP task. If it is separated into an early subtask, the background worker can begin before the remaining business-logic use cases are complete. The limit to such decomposition is already apparent: the path $1 \to 2 \to 3 \to 4$, of length 35.4 h, almost completely determines $L = 36.7$ h, while the makespan is bounded below by the human resource $H = 19.0$ h and speedup is bounded above by ×4.0. As the critical path is shortened further, H becomes the active constraint, as predicted by level M4.

B.3.6 Comparison of Variants

Table 17: Comparison of work-organization variants accounting for the dependency graph

Metric	Variant 1	Variant 2	Variant 3	Variant 4
Configured parallelism P	1	2	4	4
Mode	interactive	mixed	delegated w/spec.	delegated w/spec.
Decomposition	coarse	coarse	coarse	split tests
W, h	65.0	59.8	53.6	53.6
W/P, h	65.0	29.9	13.4	13.4
L, h	51.8	47.8	43.2	36.7
H, h	65.0	38.4	19.0	19.0
B_4, h	65.0	47.8	43.2	36.7
$T(\pi)$, h	65.0	47.8	43.2	36.7
S_E^{ach}	×1.17	×1.59	×1.76	×2.07
Ceiling $1/\bar{h}_w$	×1.17	×1.98	×4.00	×4.00

For these illustrative variants, the table shows that, once dependencies are accounted for, the final speedup is determined not only by the total work W and the number of

agents P, but also by the critical-path length L—and L itself depends on the selected decomposition.

1. **The W/P lower bound is informative but optimistic as a makespan estimate.** It equals 29.9 h for Variant 2 and 13.4 h for Variant 3. They are unattainable under the stated M4 parameters even if all DAG edges are removed, because $H = 38.4$ h and $H = 19.0$ h, respectively, already exceed W/P. Queueing may further increase stream occupancy.

2. **The critical path determines the result in Variants 2–4.** In Variants 2 and 3, the chain $1 \to 2 \to 3 \to 4 \to 6$ determines makespans of 47.8 and 43.2 h, respectively. After the tests are split in Variant 4, the critical path decreases to 36.7 h but remains the active constraint. The achievable speedups are therefore $\times 1.59$, $\times 1.76$, and $\times 2.07$, rather than the values calculated under the assumption of fully independent tasks.

3. **Increasing P alone is insufficient in these illustrative variants.** In the transition from $P = 2$ to $P = 4$, the independent-task estimate W/P decreases sharply, but the overall makespan changes only moderately: additional agents cannot begin tasks on the critical path until their predecessors have been completed.

4. **The decomposition contrast is isolated by an all-else-equal comparison.** The transitions $1 \to 2$ and $2 \to 3$ change several configuration parameters simultaneously. Only the transition $3 \to 4$ preserves P, the operating mode, k, and h; therefore, within this illustrative scenario, the increase in speedup from $\times 1.76$ to $\times 2.07$ pertains to the change in decomposition and requires no additional resources.

5. **The human resource imposes a ceiling and constrains the schedule.** As the human component decreases, the ceiling $1/\bar{h}_w$ increases: $\times 1.17 \to \times 1.98 \to \times 4.00$. It is already attained in Variant 1, where the process is human-constrained. Variants 2–4 retain headroom, but exploiting it requires explicit scheduling of the service intervals. In the revised Variant 2 schedule, the agent assigned to Task 7 is blocked in the queue for developer attention for 22.2 h; this does not increase the makespan because the task is not on the critical path.

6. **Moderate attention-switching overhead does not change the ranking in this example.** With $\gamma(P) = 1 + 0.1(P - 1)$, exact recomputation on the refined time grid yields the following values (hours):

variant	W_4	L_4	γH	B_4	T^*
2	63.640	51.320	42.240	51.320	51.36
3	59.300	47.700	24.700	47.700	47.70
4	59.300	40.450	24.700	40.450	40.45

   Thus, the ordering $4 < 3 < 2$ is preserved, and the human-resource branch remains inactive. Under the more conservative assumption $h_i = 0.5k_i$ for Variants 3–4, holding k_i fixed also changes $a_i = k_i - h_i$ and every effective path weight. With $C = 1$ and $\gamma(4) = 1.3$, recomputation gives $W_4 = 61.64$ h and $\gamma H = 34.84$ h in both variants, while $L_4 = B_4 = T^*$ is 49.68 h in Variant 3 and 42.205 h in Variant 4. These schedules retain the equal division of human work between input specification and final review. The ranking is preserved; the human-resource branch remains inactive but may become active if L_4 is reduced further.

Implications of the Illustrative Scenario.

1. **Dependencies should be modeled explicitly.** Balancing independent tasks overstates speedup when the work is linked by a dependency graph. For the illustrative scenario considered here, at level M4 the lower bound is $\max\{W/P, L, H\}$ when $C(P) = \gamma(P) = 1$.

2. **The critical path becomes the principal constraint.** In Variants 2–4, the makespan equals L. The composition of the critical path depends on the granularity of the graph: after the testing task is split, only retry testing remains at the end of the path, whereas unit and integration tests are performed earlier and in parallel.

3. **In this illustrative scenario, increasing the number of agents does not yield proportional speedup.** The mechanism illustrated here is that, if the graph contains large sequentially dependent tasks, increasing P quickly ceases to affect the result: some agents remain idle while waiting for mandatory predecessors to be completed.

4. **Decomposition produces a quantifiable difference in this illustrative scenario.** Splitting one testing task reduced the makespan by 6.5 h and increased speedup from $\times 1.76$ to $\times 2.07$ without changing P, k, or h. Further candidates should be sought among critical-path edges that connect entire large tasks even though the actual dependency pertains to only one subtask.

5. **Both agents and human involvement must be scheduled.** A single developer reviews delegated tasks sequentially, so the corresponding intervals and the queue for developer attention must be included explicitly in the M4 schedule. A waiting agent retains its assigned agent slot; that slot cannot be reassigned to another task until the current task is accepted. Prolonged continuous interactive work blocks the processing of parallel results, and the achievable speedup is bounded above by $1/\bar{h}_w$ independently of the number of agents.

C Additional Computational Results

C.1 Additional Visual Results

Figure 6 shows the distribution of the gaps between the structural lower bound, the optimum, and the baseline policy. A zero gap means that B_4 is attainable, not that no queue is present.

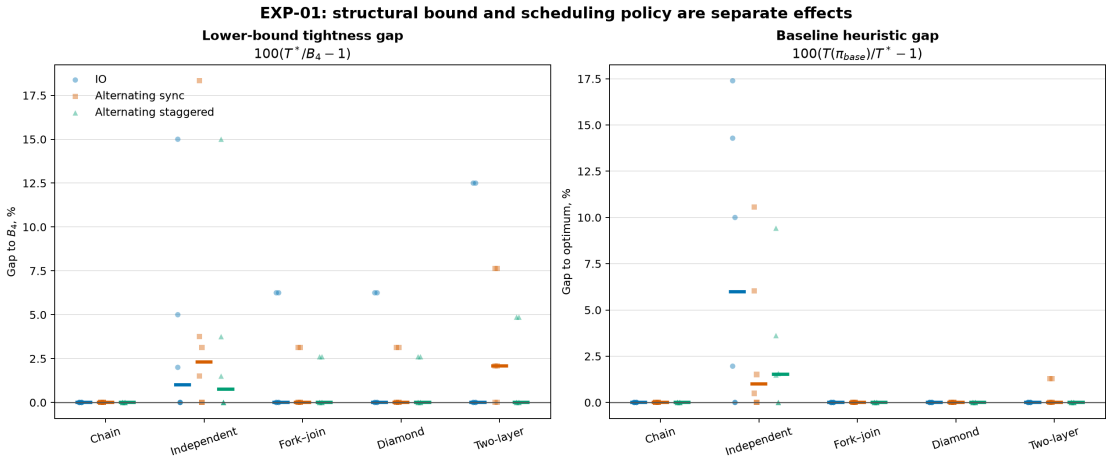

Figure 6: EXP-01: the $T^*/B_4 - 1$ gaps and the baseline-policy gaps.

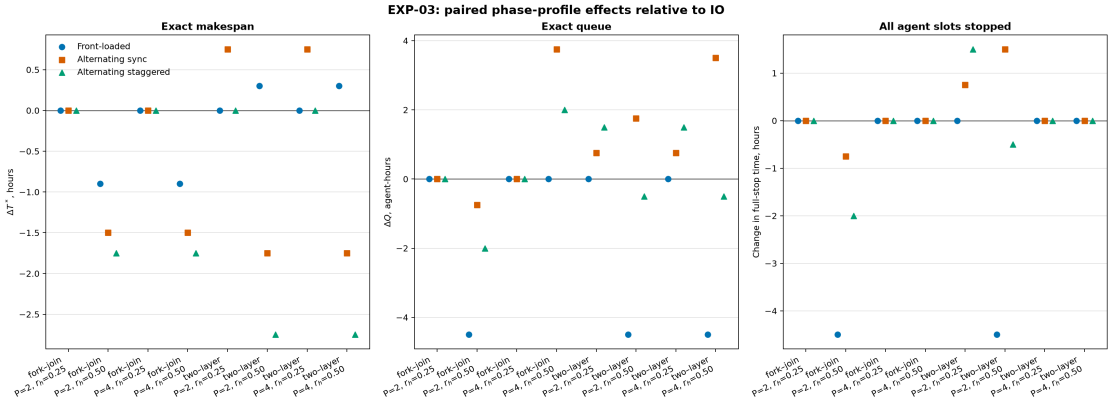

Figure 7: EXP-03: change in T^* for identical aggregates and different phase profiles. The historical label sync does not denote synchronization of wall-clock intervals here.

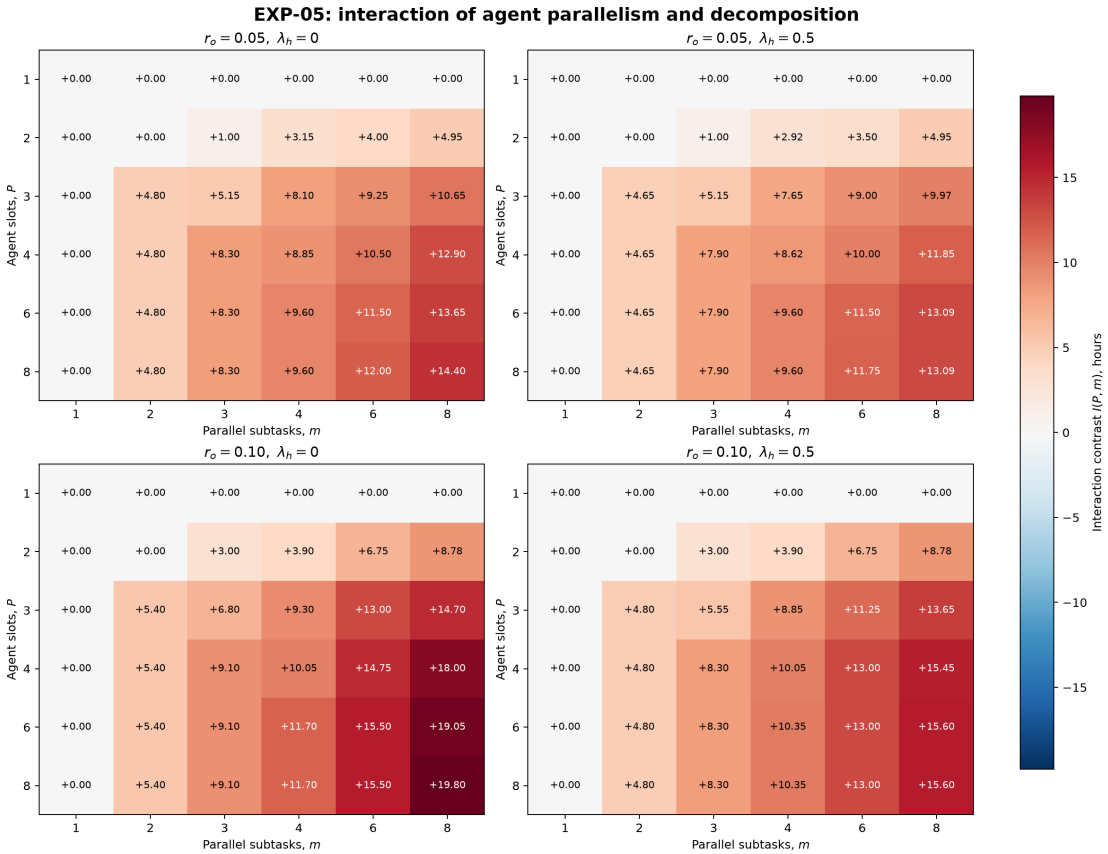

Figure 8: EXP-05: interaction between configured parallelism and granularity on the canonical exact grid.

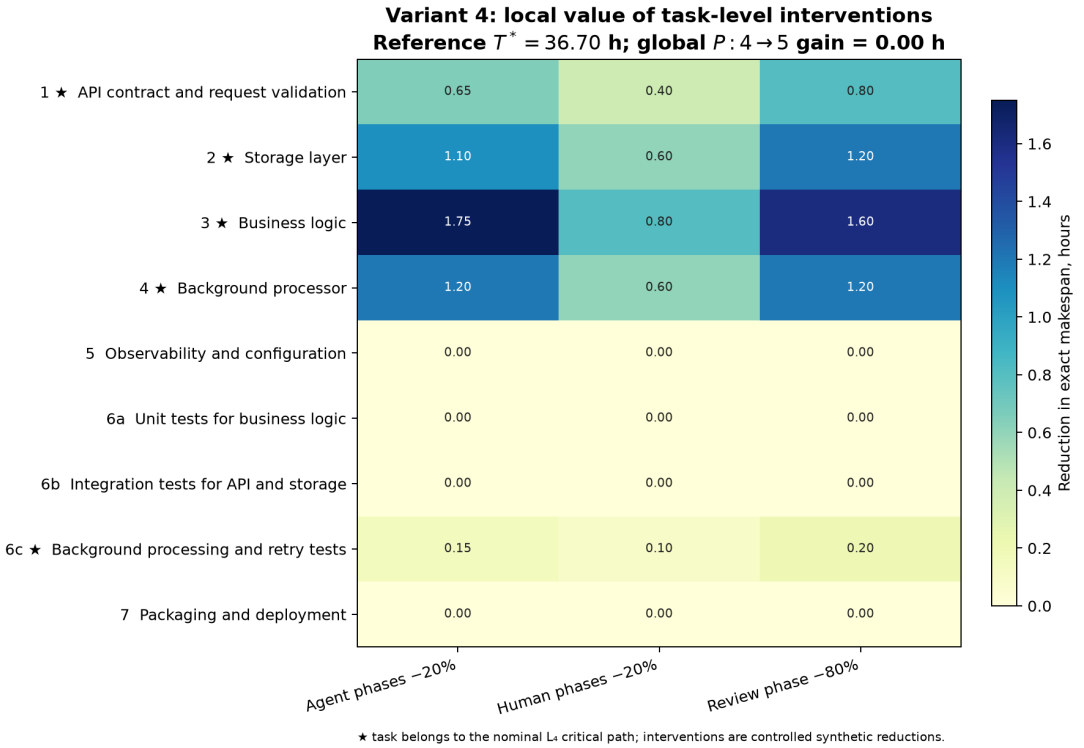

Figure 9: Illustrative local sensitivity analysis for Variant 4. Each cell reports the reduction in certified T^* after a controlled reduction in the phases of one task; asterisks identify tasks on the nominal critical path. Intervention magnitudes are synthetic and are not empirical estimates.

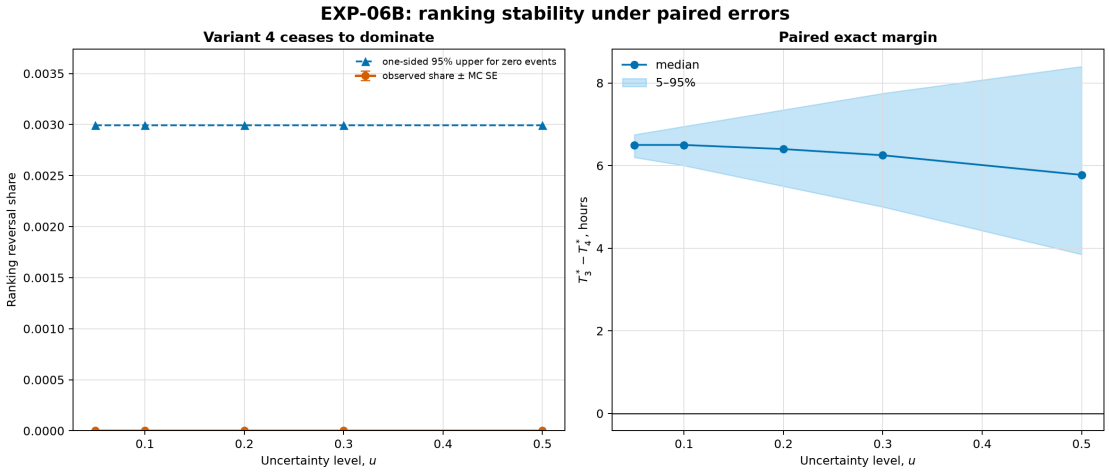

Figure 10: EXP-06: stability of the ranking of the illustrative variants within the frozen error families.

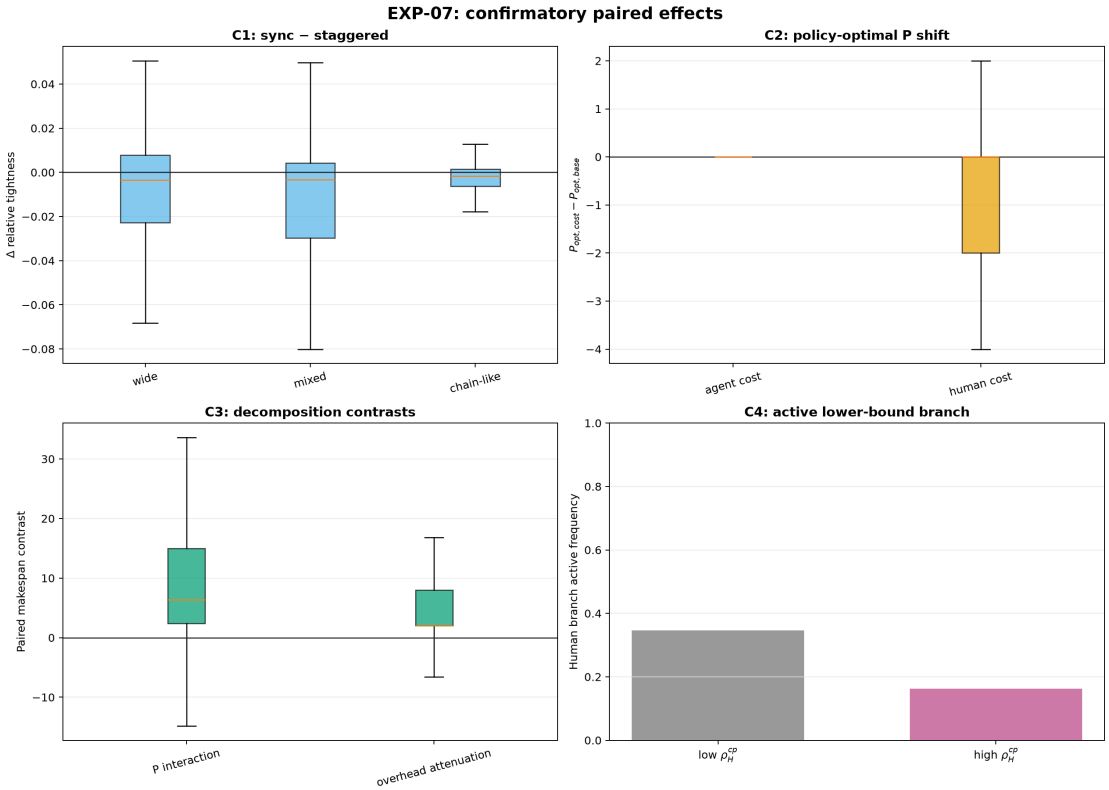

Figure 11: EXP-07: effects on synthetic DAGs. C1 and C4 are not supported; C2 and C3 pertain primarily to the fixed policy.

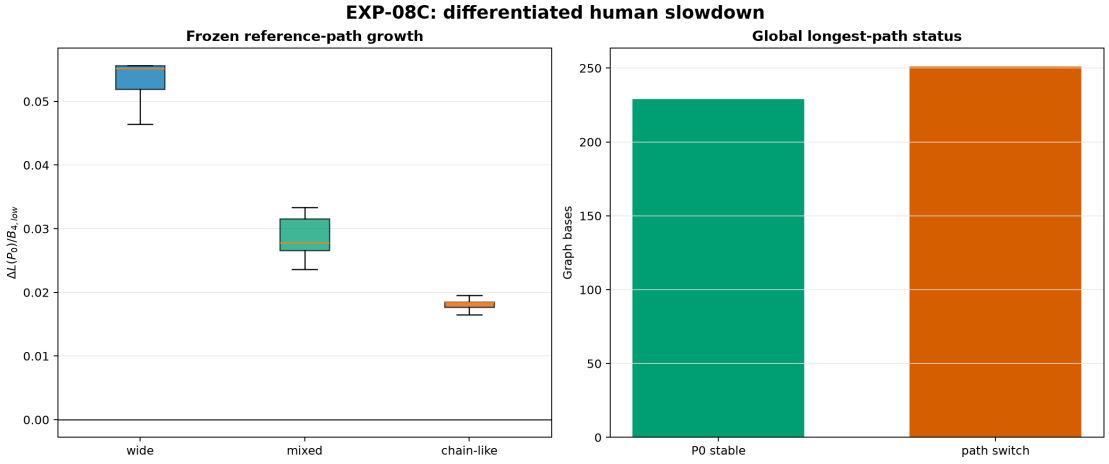

Figure 12: EXP-08: change in the prespecified path when $\gamma > C$ and switches in the global critical path.

C.2 Reproducibility

The source code, frozen inputs, and saved results are located in the `experiments` directory. From the project root, the complete set of commands is:

```
cd article/experiments
uv sync --python 3.13
```

```
uv run pytest -q
uv run python -m experiments validate scenarios/article
uv run python -m experiments run --experiment EXP-00
uv run python -m experiments run --experiment EXP-01
uv run python -m experiments run --experiment EXP-02
uv run python -m experiments run --experiment EXP-03
uv run python -m experiments run --experiment EXP-04
uv run python -m experiments run --experiment EXP-05
uv run python -m experiments run --experiment EXP-06
uv run python -m experiments run --experiment EXP-07
uv run python -m experiments run --experiment EXP-08 --stage development
uv run python -m experiments run --experiment EXP-08 --stage confirmatory
uv run python -m experiments figures --set candidates
```

EXP-00–EXP-07 are independent and can be reproduced separately. For EXP-08, the order is material: the development stage creates `exp08_freeze.json`, whereas the confirmatory command refuses to run without a matching freeze file and frozen matrix. The `-stage all` shortcut executes the same two stages sequentially, but the explicit commands above make the data boundary auditable.

Path	Contents
`scenarios/article/`	fully specified EXP-00 illustrative variants
`scenarios/canonical/`	frozen EXP-01–EXP-08 matrices, the original EXP-01 pilot, and the record of its global reduction
`src/experiments/`	schema, validator, fixed policy, exact solver, series generators, and figure-generation code
`tests/`	checks of the schema, series generation, invariants, and regression tests for the values reported in the article
`results/exp??_manifest.json`	SHA-256 checksums of inputs, implementation, and outputs; dependency versions; settings; and statuses
`results/exp??_report.md`	complete design, result tables, limitations, and links to raw CSV files
`results/full_experiments_report.md`	consolidated EXP-00–EXP-08 summary and artifact map
`figures/`	PNG/SVG pairs and their generation code

The principal raw tables contain the scenario parameters, B_4, the fixed-policy makespan, solver status, objective, best bound, and gap; separate phase and event tables permit an independent recheck of resource feasibility. The manifests contain the Git commit and checksums of the listed inputs, implementation files, and outputs, so a change to any listed file is detected before results are compared. The lockfile pins the packages. The manifests do not record the operating system or hardware used for the original runs, which limits the reproducibility of wall-clock running time but does not affect the `OPTIMAL` criterion or the saved integer objective values.

Data and code availability. Code (MIT), text and research materials (CC BY 4.0), and the public repository `https://github.com/Rexarrior/simple_model_preprint` are available. A versioned release and DOI will follow archival; the supplementary artifact is described in Section 11.